Fragstore Webshop

Visegrádi Tamás &
Kelemen Gábor

Modern Game Webshop

The premier destination for games across all platforms and genres

Release Date:
2026-01-22

Version: v1.1.0

Tartalom

Tartalom
Bevezetés 7
1. Felhasználói dokumentáció 9
1.1. Munka megosztás 9
1.1.1. A felosztás 9
1.1.2. Skill alapján való felosztás 9
1.2. A home weboldal 11
1.2.1. A navbar, hero és felső banner 11
1.2.2. About Us (Rólunk oldal) 13
1.2.3. Logo körhinta és értékelések 15
1.2.4. eGift Cards 17
1.2.5. Genre 19
1.2.6. FAQ és az emberektől kommentek 21
1.2.7. Footer (lábléc) 22
1.3. Games main weboldal 23
1.3.1. A Games main weboldal navbarja, termékek megjelenítése, a szűrők, a wishlist és a kosár 23
A grid változása, a lapozó és a footer avagy lábléc 27
1.4 Log in weboldal 29
1.4.1. Bejelentkezési felület (Login) 29
1.4.2. Biztonsági ellenőrzés (CAPTCHA) 29
1.4.3. Elfelejtett jelszó funkció 30
1.4.4. Regisztrációs lehetőség 30
1.4.5. OTP Verification (OTP ellenőrzés) 31
1.4.6. OTP kód megadása és ellenőrzése a 31
1.5. Regisztrációs felület (Sign Up) 32

1.5.1. Fiók létrehozása és visszajelzés 32
1.6. Cart 33
1.6.1 Fizetési folyamat (Checkout) 34
1.6.2 Szállítási mód kiválasztása 35
1.6.3 Fizetési mód megadása 36
1.6.4 Rendelés összegzése (Order Summary) 37
1.6.5 Vásárlás véglegesítése 38
1.7 Pac Man 38
1.7.1 A Pac Man menü 38
1.7.2. Pac Man játék 39
1.7.3 Halál utáni menü 40
1.7.4 Achievement lista 41
1.8 Snake 42
1.8.1 A menü 42
1.8.2 A snake játék maga 43
1.9 Tetris 45
1.9.1 A játék 45
1.9.2 Game over menü 46
1.10. Az admin felület 48
2. Fejlesztői dokumentáció – első szintű címsor 49
2.1. Felhasznált technológiák 49
2.1.1. PHP 49
2.1.2. HTML 49
2.1.3. CSS 49
2.1.4. JavaScript 50
2.1.5. TypeScript 50
2.1.6. JSON 50

2.1.7. SQL ... 51
2.1.8. Bash ... 51
2.1.9. Twig ... 51
2.2. Keretrendszerek ... 51
2.2.1. React ... 51
2.2.2. Angular ... 52
2.2.3. Alpine.js ... 52
2.2.4. p5.js ... 52
2.2.5. Sketch ... 52
2.2.6. GSAP ... 53
2.2.7. Bootstrap ... 53
2.2.8. Tailwind CSS ... 53
2.2.9. Bulma ... 53
2.2.10. Butterfly ... 53
2.3. API-k ... 54
2.3.1. Chart.js ... 54
2.3.2. Flags API ... 54
2.3.3. Currency Exchange API ... 54
2.3.4. EmailJS ... 54
2.3.5. RAWG API ... 55
2.3.6. Google Translate API ... 55
2.4. Cloud service ... 55
2.4.1. Firebase ... 55
2.5. Fejlesztési eszközök ... 56
2.5.1. Visual Studio Code ... 56
2.5.2. Figma ... 56
2.5.3. XAMPP ... 56

2.5.4. GitHub ... 57
2.5.5. Vite ... 57
2.5.6. Discord ... 57
2.6. Adatbázis ... 58
2.6.1. Adatbázis tervezése ... 58
2.6.2. Adatbázis megvalósítása ... 64
2.6.3. Teszt adatok kóddal ... 69
2.6.4. Továbbfejlesztési lehetőség ... 75
2.6.5. N:M-es kapcsolatok felbontása ... 75
2.6.6. A többikapcsolat ... 75
2.6.7. Firebase database ... 77
2.7. Figma használata az oldal teszteléséhez ... 81
2.7.1. Home weboldal terve ... 81
2.7.2. Login/Sign up ... 83
2.7.3. Main (webshop) ... 84
2.8. Kódrészletek ... 84
2.8.1. Bevásárlókosár és kívánságlista (Wishlist) ... 84
2.8.2. Játékok megjelenítése ... 89
2.8.3. Fordítás a Home oldalon ... 93
2.8.4. Giftcard kiíratás a home weboldalon ... 97
2.8.5. Log in backendje ... 101
2.8.6. Regisztráció (Sign up) backend folyamata ... 104
2.8.7. OTP kétlépcsős azonosító ... 107
2.8.8. Easter egg játék load (Pac Man) ... 112
2.8.9. Szellemek viselkedése (Pac Man) ... 117
2.8.10. Snake easter egg game ... 120
2.8.11. Snake control ... 126

2.8.12. Tetris kirájzolás 129
2.8.13. Általános ütközésvizsgáló 132
2.8.14. Alakzatok 137
2.8.15. Admin panel 141
3. Felmerült akadályok 145
4. Közös munka és Platformok 151
4.1. Trello 151
4.2. Github 152
4.3. Discord 153
5. Továbbfejlesztések 154
6. Összefoglalás 156
7. Források 158
8. Ábrajegyzék: 159

Bevezetés

A Fragstore webalkalmazás célja digitális termékek – elsősorban videojátékokhoz kapcsolódó gift cardok (például Call of Duty játékokhoz, illetve PlayStation gift cardok) – értékesítése egy modern, átlátható webes felületen. Az oldal egy webshopként működik, ahol a felhasználók böngészhetik a termékeket, kosárba helyezhetik őket, wishlistet hozhatnak létre, valamint regisztráció és bejelentkezés után különböző funkciókat érhetnek el. Bár az oldalon valódi fizetés nem valósul meg, a kosár működése teljes mértékben implementálva van.

A frontend megvalósításához JavaScriptet és annak keretrendszereit használtuk, míg a backend oldalon PHP és SQL technológiákra építettük fel a rendszert. Az alkalmazás egyik kiemelkedő funkciója egy AI chatbot, amely segíti a felhasználókat az oldalon való eligazodásban, valamint a wishlist rendszer, amely lehetővé teszi a kinézett termékek elmentését.

A projektet kétfős csapatban készítettük el. Kelemen Gábor (én) elsősorban az adatbázis tervezéséért és megvalósításáért, a fejlesztői dokumentáció elkészítéséért, az oldal reszponzivitásáért, valamint a tervezési folyamatok nagy részéért voltam felelős. Emellett jellemzően én készítettem el az egyes oldalak alap vázát (frame-jeit). A backend fejlesztésében mindketten részt vettünk: Visegrádi Tamással közösen dolgoztunk a bejelentkezési, regisztrációs és egyéb funkcionális elemek megvalósításán. Az easter egg játékok fejlesztésében kisebb mértékben vettem részt, ezek kidolgozásával főként csapattársam foglalkozott. A végső, igazán dizájnos megjelenés nagyban az ő munkájának köszönhető.

Az ötlet onnan származott, hogy mindketten szeretünk videojátékokkal játszani, és szerettünk volna egy összetettebb, kihívást jelentő projektet, amelyben bizonyíthatjuk a megszerzett tudásunkat. A fejlesztés során mentoraink – Drenkovics Péter, Barnáné Kiss Éva és Juhász Zoltán – folyamatos támogatást és motivációt nyújtottak számunkra, ami nagy segítséget jelentett a projekt megvalósítása során.

A projekt fejlesztése végig Git verziókezelő rendszerrel történt, a forráskód egy publikus GitHub repositoryban érhető el. A repository a https://github.com/VTom21/Fragstore-Webshop címen található meg, ahol a fejlesztési folyamat nyomon követhető, valamint a projekthez kapcsolódó dokumentáció is elérhető.

A Fragstore webalkalmazás fejlesztése során számos modern webes technológiát, keretrendszert és fejlesztői eszközt alkalmaztunk annak érdekében, hogy egy stabil, jól

skálázható és felhasználóbarát rendszert hozzunk létre. A backend megvalósításához PHP-t és SQL adatbázist használtunk, amelyek lehetővé tették a felhasználói adatok, termékek, kosár- és wishlist funkciók biztonságos kezelését, valamint az üzleti logika hatékony megvalósítását. A frontend oldalon HTML, CSS és JavaScript alkotta az alapot, amelyet különböző JavaScript-alapú megoldásokkal egészítettünk ki annak érdekében, hogy az oldal dinamikus és interaktív működést biztosítson.

A JavaScript ökoszisztémán belül több különböző megközelítést is alkalmaztunk. A komplexebb, komponensalapú felületek kialakításában React és Angular keretrendszerek segítették a strukturált fejlesztést, míg az egyszerűbb interakciók és kisebb dinamikus elemek megvalósításához Alpine.js bizonyult hatékony megoldásnak. A TypeScript használata hozzájárult a kód minőségének javításához, mivel a típusosság révén csökkentette a futásidejű hibák számát. Az adatok strukturált továbbítását JSON formátum biztosította a frontend és a backend között.

A megjelenés és a felhasználói élmény fejlesztése során különös hangsúlyt fektettünk a reszponzivitásra és a vizuális élményre. Ebben nagy segítséget nyújtott a Bootstrap és a Tailwind CSS, amelyek lehetővé tették a különböző képernyőméretekhez való gyors és hatékony alkalmazkodást. Az animációk és látványos átmenetek megvalósításához GSAP könyvtárat használtunk, míg a kreatívabb, játékosabb elemek és easter egg funkciók fejlesztésében a p5.js játszott fontos szerepet. A felhasználói felület megtervezése és az oldal vizuális koncepciójának kialakítása Figma és Sketch segítségével történt, amelyek elősegítették az egységes dizájn kialakítását már a fejlesztés korai szakaszában.

Az alkalmazás funkcionalitását külső szolgáltatások és API-k is bővítették. A Chart.js lehetőséget biztosított adatok grafikus megjelenítésére, amely elsősorban statisztikai vagy adminisztrációs nézetekben volt hasznos. A Flags API segítségével országokhoz kötődő vizuális elemeket tudtunk megjeleníteni, míg a Currency Exchange API lehetővé tette az árak különböző pénznemek közötti átváltását. Az EmailJS integrálása biztosította az értesítések és kapcsolatfelvételi funkciók egyszerű megvalósítását, külön backend email szerver konfigurálása nélkül.

A fejlesztési folyamat során kiemelt szerepet kaptak a modern fejlesztői eszközök is. A Visual Studio Code biztosította az egységes fejlesztői környezetet, míg a Git és a GitHub lehetővé tették a verziókövetést és a hatékony csapatmunkát. A helyi fejlesztéshez XAMPP környezetet használtunk, amely megkönnyítette a backend tesztelését. A frontend buildelési és fejlesztési folyamatokat Vite gyorsította fel, míg egyes fejlesztési és automatizálási feladatok során Bash parancsokat alkalmaztunk.

Ezen technológiák együttes alkalmazása tette lehetővé, hogy a Fragstore egy összetett, mégis jól átlátható és bővíthető webalkalmazásként valósuljon meg, amely megfelelő alapot biztosít további funkciók és fejlesztések bevezetéséhez.

1. Felhasználói dokumentáció

1.1. Munka megosztás

1.1.1. A felosztás

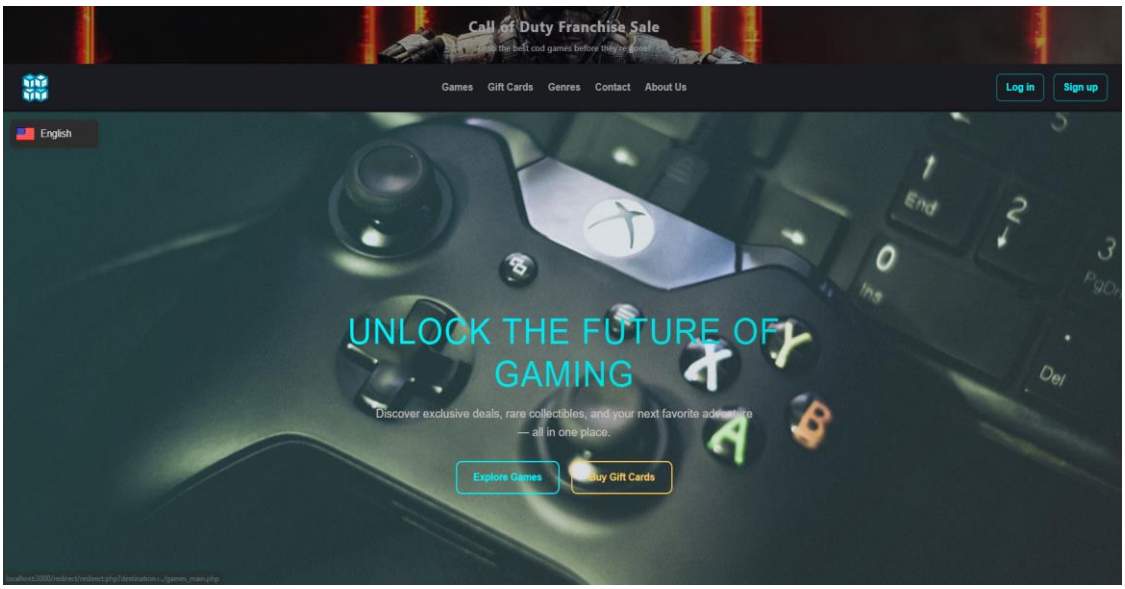
Közöttünk, vagyis Kelemen Gábor és Visegrádi Tamás között a közös munka felosztása nem a megszokott módon történt. Mindketten dolgoztunk dizájn, frontend, backend és adatbázis szinten is, így egyik terület sem volt kizárólagosan egy emberhez kötve. Ennek ellenére létezett egy alapvető felosztás, amely főként Visegrádi Tamás irányításával valósult meg, hiszen ő töltötte be a projektvezetői szerepet.A projekt során azonban folyamatosan segítettük egymást, és többször előfordult az is, hogy egyikünk átvette a másik feladatát, amennyiben úgy láttuk, hogy azt az adott pillanatban hatékonyabban vagy gyorsabban tudjuk megvalósítani. Ez a rugalmas hozzáállás nagyban hozzájárult ahhoz, hogy a fejlesztés folyamatos maradjon, és ne akadjon meg egy-egy részfeladatnál.Ahogy korábban is említettük, a feladatok egyeztetését és az elkészült részek bemutatását főként Discordon keresztül végeztük. A rendszeres megbeszélések, képernyőmegosztások és bemutatók segítettek abban, hogy mindig tisztában legyünk a projekt aktuális állapotával, valamint azzal, hogy ki min dolgozik éppen.Mivel mindketten több területen is aktívan részt vettünk a fejlesztésben, a Trello felületen kizárólag magukat a feladatokat rögzítettük, és nem rendeltük hozzá őket konkrét személyhez. Számunkra ez bizonyult a leghatékonyabb munkamódszernek, mivel nem korlátozott bennünket, és teljes szabadságot adott abban, hogy mindig az végezze el az adott feladatot, aki éppen a legalkalmasabb volt rá. Ennek köszönhetően a munka rugalmas, hatékony és jól szervezett maradt a projekt teljes időtartama alatt.

1.1.2. Skill alapján való felosztás

Skill	Tamás	Gábor
Frontend	X	X
Backend	X	X
API-k	X	-
Dokumentáció	-	X
Tervezés	X	X
Cloud-service	X	-
Bug-fixing	X	X
Tesztelés	-	X

1.2. A home weboldal

1.2.1. A navbar, hero és felső banner



1. ábra - Hero Section

1.2.1.1. A hero

A kezdőlap központi eleme a nagyméretű hero szekció, amely azonnal megragadja a látogatók figyelmét. A háttérben egy Xbox kontroller részlete látható, amely vizuálisan is erősen közvetíti a gaming témát, valamint modern és technológiai hangulatot teremt.

Főbb elemek:

Főcím:

A képernyő közepén elhelyezett *„UNLOCK THE FUTURE OF GAMING”* felirat ciánkék színnel kiemelve jelenik meg. A nagybetűs, látványos szöveg azonnal kommunikálja az oldal célját és a brand identitását, valamint hatékonyan felkelti a felhasználók érdeklődését.

Alcím:

A főcím alatt egy rövid leírás található:

„Discover exclusive deals, rare collectibles, and your next favorite adventure — all in one place.”

Ez a szöveg tömören összefoglalja az oldal értékajánlatát, és

egyértelműen bemutatja, hogy a felhasználók milyen tartalomra számíthatnak.

CTA gombok:

A hero szekcióban két jól elkülöníthető call-to-action gomb kapott helyet:

– **„Explore Games”** – ciánkék kerettel és átlátszó háttérrel

– **„Buy Gift Cards”** – arany/sárga kerettel és átlátszó háttérrel

Ezek a gombok közvetlen navigációs lehetőséget biztosítanak, lehetővé téve a felhasználók számára, hogy azonnal elérjék az őket érdeklő tartalmakat anélkül, hogy további menüpontokat kellene keresniük.

1.2.1.2. Navigációs menü

A hero szekció alatt helyezkedik el a fő navigációs sáv, amely sötét, enyhén áttetsző háttérrel rendelkezik. Ez a megoldás biztosítja az olvashatóságot, miközben a háttérkép vizuálisan továbbra is érvényesül.

- **Bal oldal:**
  Az oldal logója (kék színű doboz ikon), amely egyben a főoldalra visszavezető linkként is funkcionál.
- **Középső rész:**
  A fő menüpontok vízszintesen elrendezve jelennek meg:
  Games, Gift Cards, Genres, Contact, About Us.
- **Jobb oldal:**
  Felhasználói műveletekhez kapcsolódó gombok találhatók itt:
  – **„Log in”** – ciánkék kerettel
  – **„Sign up”** – ciánkék kitöltéssel

Ezek a gombok jól elkülönülnek a navigáció többi elemétől, és egyértelmű belépési, illetve regisztrációs lehetőséget biztosítanak.

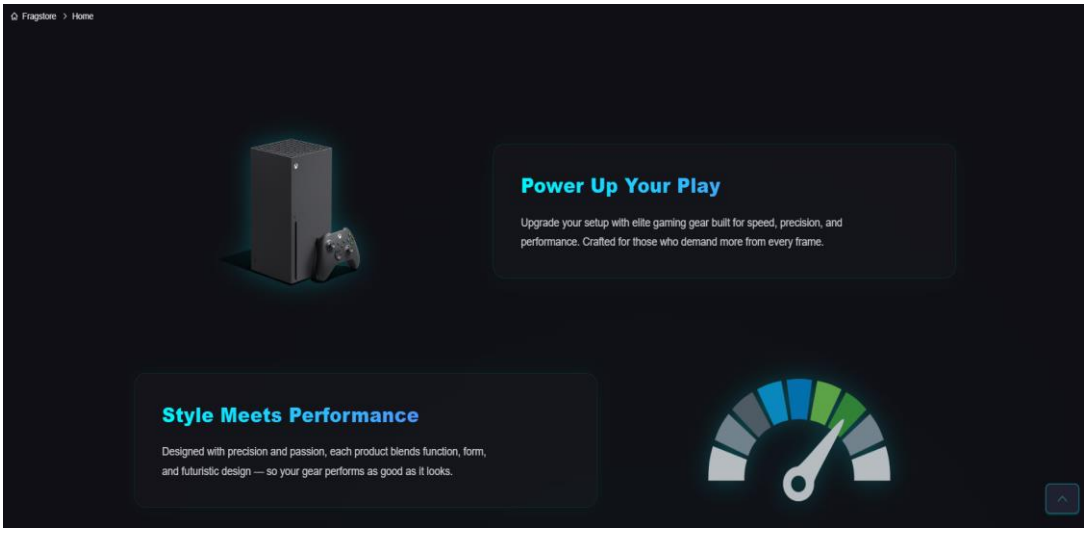
1.2.1.3. Felső banner

Az oldal legtetején egy figyelemfelkeltő banner található sötét háttérrel és piros díszítő elemekkel a széleken. A banner tartalma: "Call of Duty Franchise Sale - Grab the best cod games before they're gone!" Ez az elem időérzékeny akciókra hívja fel a figyelmet, és sürgősséget kelt a látogatókban, ösztönözve őket a gyors vásárlásra.

1.2.1.4. Nyelv választó

A navigációs sáv alatt, a bal felső sarokban található az amerikai zászlós nyelv választó ("English" felirattal), amely lehetővé teszi a felhasználók számára, hogy megváltoztassák az oldal nyelvét. Ez a funkció különösen hasznos nemzetközi közönség esetén, és javítja az oldal hozzáférhetőségét.

1.2.2. About Us (Rólunk oldal)



2. ábra - About Us

1.2.2.1. Navigációs Fejléc (Fixed Position)

Az oldal tetején található navigációs sáv fix pozícióval rendelkezik, ami azt jelenti, hogy görgetés közben is a képernyő tetején marad. Ez a funkció jelentősen javítja a felhasználói élményt, mivel bármikor, az oldal bármely részéről azonnal elérhetők a fő menüpontok anélkül, hogy vissza kellene görgetni a lap tetejére.

A fix navbar elemei:

- **Bal oldal:** Oldal logója (kék doboz ikon)
- **Közép:** Fő menüpontok *(Games, Gift Cards, Genres, Contact, About Us)*
- **Jobb oldal:** "*Log in*" és "*Sign up*" gombok ciánkék színezéssel

A navbar sötét, félig átlátszó háttere biztosítja, hogy görgetés közben is jól látható és olvasható maradjon, miközben nem takarja el teljesen a hátteret.

1.2.2.2. Breadcrumb Navigáció

A fix navbar alatt található a breadcrumb (morzsanavigáció), amely megmutatja a felhasználó aktuális helyét az oldal struktúrájában. A példában látható: "Fragstore > Home", amely segít a felhasználóknak megérteni, hogy hol tartózkodnak az oldalon, és egyszerű visszalépési lehetőséget biztosít a korábbi szintekre. A kis ház ikon a kezdőlapra utal, amely egyértelmű vizuális jelzést ad.

1.2.2.3. Elrendezés és Design Elvek

Szimmetrikus grid rendszer: Az About Us oldal egy kétosztású grid rendszert használ, ahol a tartalom egyenletesen oszlik meg képi és szöveges elemek között. Az első szekcióban a kép bal oldalt, a második szekcióban jobb oldalt található, ami dinamikus, kiegyensúlyozott megjelenést eredményez.

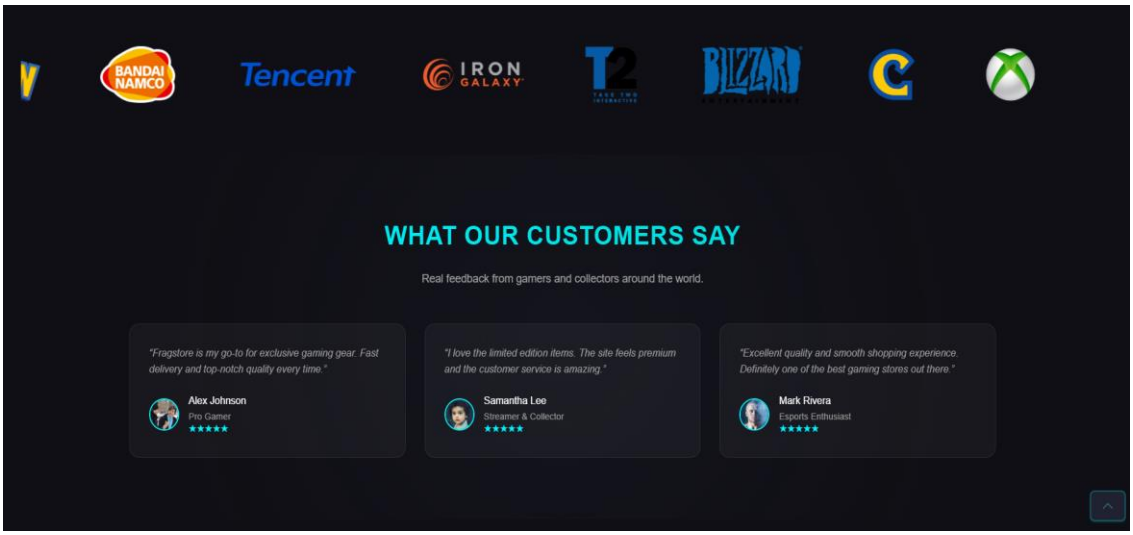
Dark mode dizájn: Az oldal következetesen sötét hátteret használ, amely:

- Kiemeli a színes elemeket (ciánkék címek, világos szöveg)
- Gaming-orientált, modern hangulatot teremt
- Csökkenti a szemek terhelését hosszabb böngészés esetén

Tipográfia: A címek nagybetűsek és ciánkék színűek, míg a törzsszöveg világosszürke, ami kellő kontrasztot biztosít az olvashatósághoz.

Scrolling élmény: A jobb alsó sarokban egy "vissza a tetejére" gomb látható ciánkék színnel, amely megkönnyíti a navigációt hosszabb oldalak esetén.

1.2.3. Logo körhinta és értékelések



3. ábra - Logo Carousel

1.2.3.1. Partnercégek Logó Körhintája

Az About Us oldal felső részén található egy automatikusan pergő logó körhinta (carousel), amely a platform partnercégeinek és együttműködő márkáinak logóit jeleníti meg vízszintes sorban. Ez a szekció vizuálisan erősíti az oldal hitelességét és megbízhatóságát azáltal, hogy bemutatja a nagy nevű gaming iparági szereplőkkel való kapcsolatokat.

Megjelenő logók:

- **Bandai Namco** - narancssárga-piros kerettel
- **Tencent** - kék márkalogó
- **Iron Galaxy** - narancssárga-szürke logó
- **Take-Two Interactive** - kék T2 logó
- **Blizzard** - kék márkanév logó
- **Capcom** - sárga-kék kerek logó
- **Xbox** - zöld-fehér X logó

Működés: A logók automatikusan, folyamatos loop-ban görgetnek végig a képernyőn, ami dinamikus és modern megjelenést biztosít. Ez a megoldás több logó megjelenítését teszi lehetővé korlátozott képernyőhelyen, miközben fenntartja a vizuális egyensúlyt és nem terheli túl az oldalt.

Dizájn elemek:

- Sötét, átmenetes háttér, amely finoman beolvad az oldal főhátterébe
- Egyenletes távolság a logók között
- Minden logó eredeti színében és formájában jelenik meg
- A logók mérete egységes, hogy vizuális harmóniát teremtsen

Célja: Ez a szekció azonnal kommunikálja a látogatók felé, hogy az oldal hivatalos kapcsolatban áll az iparág vezető szereplőivel, ami növeli a bizalmat és a brand presztízsét. A gaming közösség számára ezek az ikonikus logók felismerhetők és jelentéssel bírnak.

1.2.3.2. Vásárlói Értékelések Szekció

A logó körhinta alatt található a "WHAT OUR CUSTOMERS SAY" (Mit mondanak vásárlóink) című rész, amely közösségi bizonyítékokat (social proof) szolgáltat az oldal megbízhatóságának alátámasztására.

Szekció felépítése:

Főcím: "WHAT OUR CUSTOMERS SAY" - nagybetűs, ciánkék színnel kiemelve, amely következetesen illeszkedik az oldal színsémájába.

Alcím: "Real feedback from gamers and collectors around the world." - világosszürke színnel, amely kontextust ad és hangsúlyozza az értékelések hitelességét.

Értékelési Kártyák:

A szekció három értékelési kártyát tartalmaz, amelyek vízszintesen, egyenlő távolságra vannak elrendezve. Minden kártya sötétszürke háttérrel rendelkezik, amely finoman kiemelkedik a fekete oldalhátterből.

Dizájn Elemek:

Kártya struktúra:

- Lekerekített sarkok a modern megjelenésért
- Elegendő padding (belső térköz) az olvashatósághoz
- Kurzív idézetek a személyes jelleg hangsúlyozására

- Profilképek kerek alakban a bal alsó sarokban
- Csillagok ciánkék színben, konzisztens a brand identitással

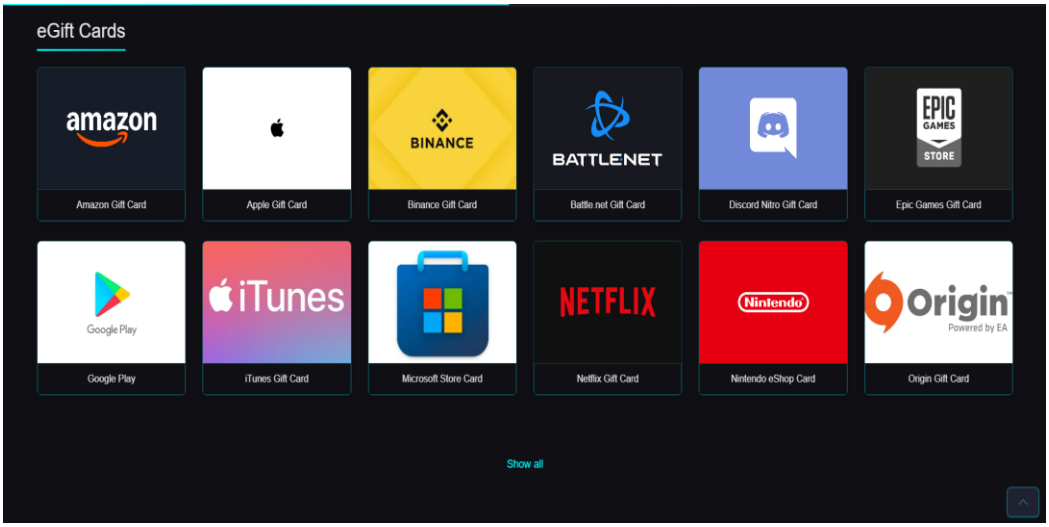
Elrendezés: A három kártya egyenlő szélességű és responsive grid rendszerben van elhelyezve, amely kisebb képernyőkön valószínűleg egymás alá rendeződik a jobb olvashatóság érdekében.

Funkció és Célközönség:

Diverzitás: A három különböző vásárlói profil bemutatása azt közvetíti, hogy az oldal széles közönséget szolgál ki - a professzionális játékosoktól a streameren át az esports rajongókig.

Hitelesség: A konkrét nevek, profilképek és szerepkörök használata hitelesebbé teszi az értékeléseket, mint az általános, névtelen visszajelzések.

1.2.4. eGift Cards



4. ábra - Giftcard Section

1.2.4.1. Oldal Fejléc

Az oldal tetején az "eGift Cards" cím található fehér színnel, amely egyszerűen és világosan kommunikálja az oldal tartalmát. A cím alatt egy vékony ciánkék vonal húzódik, amely vizuális elkülönítést biztosít a fejléc és a tartalom között, miközben következetesen illeszkedik az oldal brand színeihez.

1.2.4.2. Kártyarács (Card Grid) Elrendezés

A digitális ajándékkártyák egyResponzív grid rendszerben jelennek meg, amely 6 oszlopos elrendezést alkalmaz desktop nézetben. Ez az elrendezés lehetővé teszi, hogy egyszerre több kártya is látható legyen anélkül, hogy túlzsúfolttá válna az oldal, miközben könnyű áttekinthetőséget biztosít a felhasználók számára.

Kártya Dizájn Elemek:

Minden ajándékkártya egységes struktúrát követ:

Felső rész (Kártya Megjelenítés):

- Nagy méretű, négyzet alakú kép terület, amely a szolgáltató brandlogóját vagy márkaelemeit jeleníti meg
- Minden kártya egyedi háttérszínnel vagy dizájnnal rendelkezik, amely az adott brand identitásához illeszkedik
- Lekerekített sarkok a modern, barátságos megjelenésért

Alsó rész (Címke):

- Sötét háttér a kártya neve számára
- Fehér szöveg a maximális olvashatóságért
- Egyszerű, világos elnevezés (pl. "Amazon Gift Card", "Apple Gift Card")

1.2.4.3 Megjelenített Ajándékkártyák

Az oldalon látható kártyák két sorban jelennek meg:

Első sor (6 kártya):

Második sor (6 kártya):

1.2.4.4. "Show all" Gomb

A kártyarács alatt, középen elhelyezve található egy "Show all" (Összes megjelenítése) link ciánkék színnel. Ez a gomb egyértelműen jelzi a felhasználók számára, hogy további ajándékkártyák is elérhetők az oldalon.

Funkció: Amikor a felhasználó rákattint a "Show all" gombra, az oldal betölti és megjeleníti az összes elérhető ajándékkártyát. Ez a megoldás több előnnyel jár:

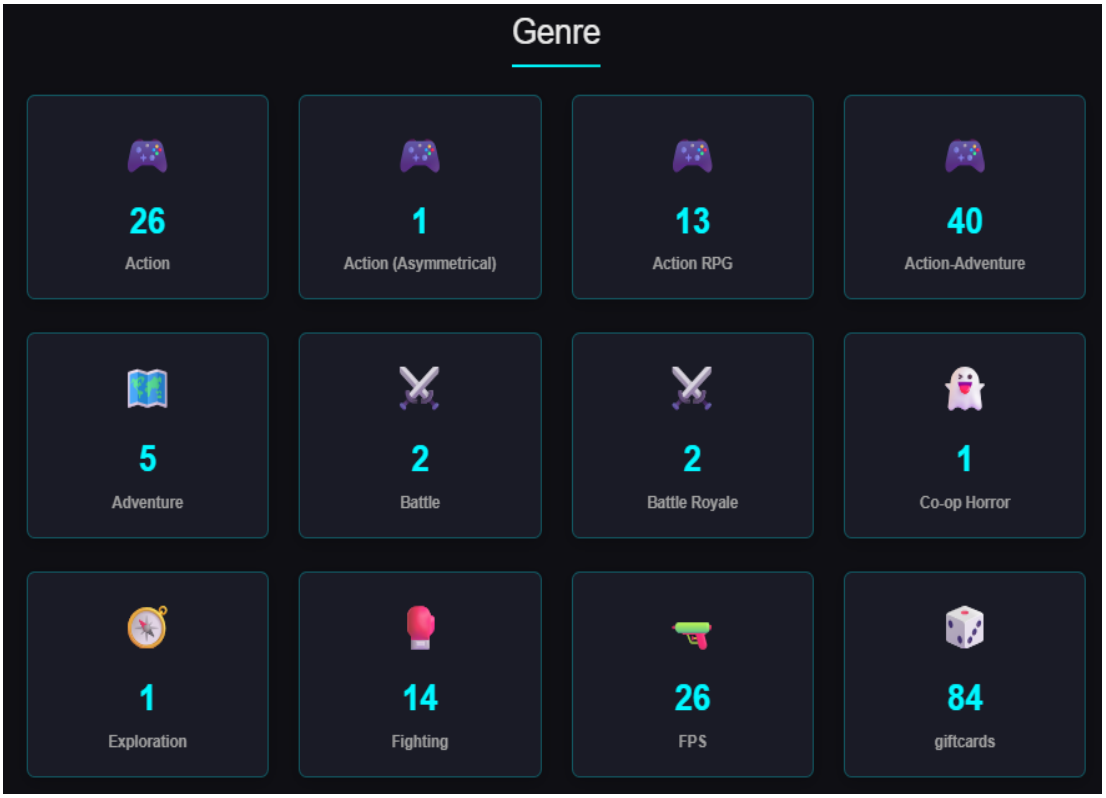
- Gyorsabb kezdeti oldalbetöltés (csak 12 kártya töltődik be először)
- Tisztább, kevésbé túlterhelt kezdeti megjelenés
- Lehetőség a népszerűbb/kiemelt kártyák előre helyezésére
- Jobb felhasználói élmény mobil eszközökön

Navigáció és Interakció

Kártya Hover Effekt: Bár a statikus képen nem látszik, valószínűleg a kártyák hover állapotban (amikor a felhasználó az egérmutatót a kártya fölé viszi) vizuális visszajelzést adnak:

Kártya Kattintás: Amikor a felhasználó rákattint egy ajándékkártyára, valószínűleg egy részletes oldal vagy modal ablak nyílik meg, ahol:

1.2.5. Genre



5. ábra - Genres Section

1.2.5.1. Fejléc

Az oldal tetején a "Genre" cím található fehér színnel, amely alatt egy rövid, vékony ciánkék vonal húzódik. Ez a minimális, de hatásos fejléc következetesen illeszkedik az oldal vizuális nyelvéhez és egyértelműen jelzi az oldal célját - a játékok műfajok szerinti kategorizálását.

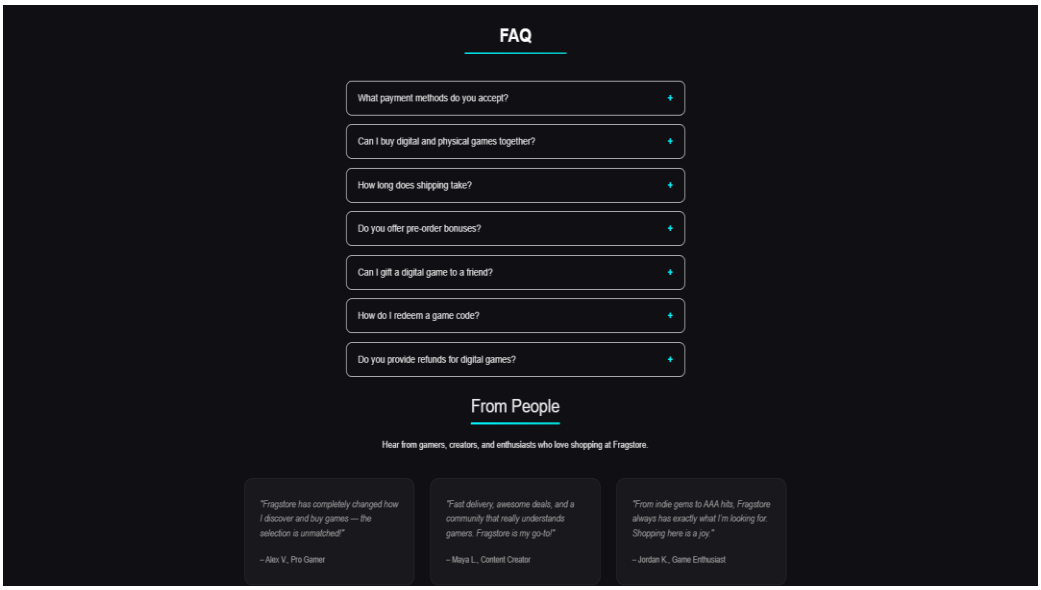
1.2.5.2. Műfaj Kártyák Grid Rendszer

A műfajok 4 oszlopos, responsive grid elrendezésben jelennek meg, amely lehetővé teszi az egyszerű áttekintést és navigációt. A kártyák három sorban helyezkednek el, összesen 12 műfaj kategóriát megjelenítve.

Tartalom felépítés:

1. **Ikon (felül)** - Színes, stilizált emoji vagy ikon, amely vizuálisan reprezentálja a műfajt
2. **Szám (középen)** - Nagy méretű, ciánkék szín, amely a kategóriában elérhető játékok számát mutatja
3. **Címke (alul)** - Világosszürke szöveg, amely a műfaj nevét tartalmazza

1.2.6. FAQ és az emberektől kommentek



6. ábra - FAQ

1.2.6.1. Oldal Fejléc

Az oldal tetején a "FAQ" cím található fehér színnel, nagyméretű, félkövér betűtípussal. A cím alatt egy vékony, ciánkék vonal húzódik, amely vizuális elválasztást biztosít és következetesen illeszkedik az oldal brand identitásához. Ez a minimális, de hatásos fejléc egyértelműen jelzi az oldal funkcióját.

1.2.6.2. Accordion (Harmonika) Kérdés-Válasz Rendszer

A FAQ szekció egy interaktív accordion (harmonika) komponenst használ, amely lehetővé teszi a felhasználók számára, hogy kérdésről kérdésre haladjanak anélkül, hogy az összes válasz egyszerre megjelenne az oldalon. Ez a megoldás tisztább, rendezettebb felületet eredményez és csökkenti az információ túlterhelést.

Accordion Kártya Dizájn:

Minden kérdés egy külön, bezárt accordion elemként jelenik meg:

1.2.6.3. Vizuális elemek:

Sötét háttér (`#1a1d29` körüli árnyalat)

Lekerekített sarkok minden kártyán

Vékony, világos keret (`#3a4152` körüli árnyalat)

Egyenletes térköz (padding) a kártyák között

Padding a szöveg körül az olvashatóság érdekében

Interaktív elemek:

Kérdés szöveg: Fehér színnel, balra igazítva

Plusz ikon (+): Ciánkék színű, a jobb szélen, amely jelzi, hogy a kártya kinyitható

Hover állapot: Valószínűleg a kártya finoman kiemelkedik vagy a keret színe megváltozik

1.2.6.4. "From People" Szekció

A FAQ rész alatt található a "From People" (Emberektől) elnevezésű szekció, amely közösségi visszajelzéseket tartalmaz.

Szekció Fejléc:

Főcím: "From People" fehér színnel, félkövér betűtípussal, ciánkék aláhúzással

Alcím: "Hear from gamers, creators, and enthusiasts who love shopping at Fragstore." - világosszürke színű szöveg, amely kontextust ad és személyesebbé teszi a visszajelzéseket.

Testimonial (Ajánlás) Kártyák:

Három ajánlás kártya jelenik meg vízszintesen, egyenlő távolságra egymástól. Minden kártya sötétszürke háttérrel rendelkezik és lekerekített sarkokkal.

1.2.7. Footer (lábléc)

1.2.7.1 Footer Struktúra és Elrendezés

Az oldal alján található footer szekció sötét háttérrel rendelkezik (`#0f1318` körüli árnyalat), amely vizuálisan elválasztja a fő tartalomtól, miközben konzisztens marad az oldal összképével. A footer három fő részre oszlik: bal oldali branding, középső navigációs oszlopok, és jobb oldali közösségi média ikonok.

1.2.7.2 Bal oldali rész – Branding

Az oldal alján található footer szekció sötét háttérrel rendelkezik, amely vizuálisan elválasztja a fő tartalomtól, miközben konzisztens marad az oldal összképével. A footer három fő részre oszlik: bal oldali branding, középső navigációs oszlopok, és jobb oldali közösségi média ikonok.

1.2.7.3 Középső rész – Navigációs oszlop

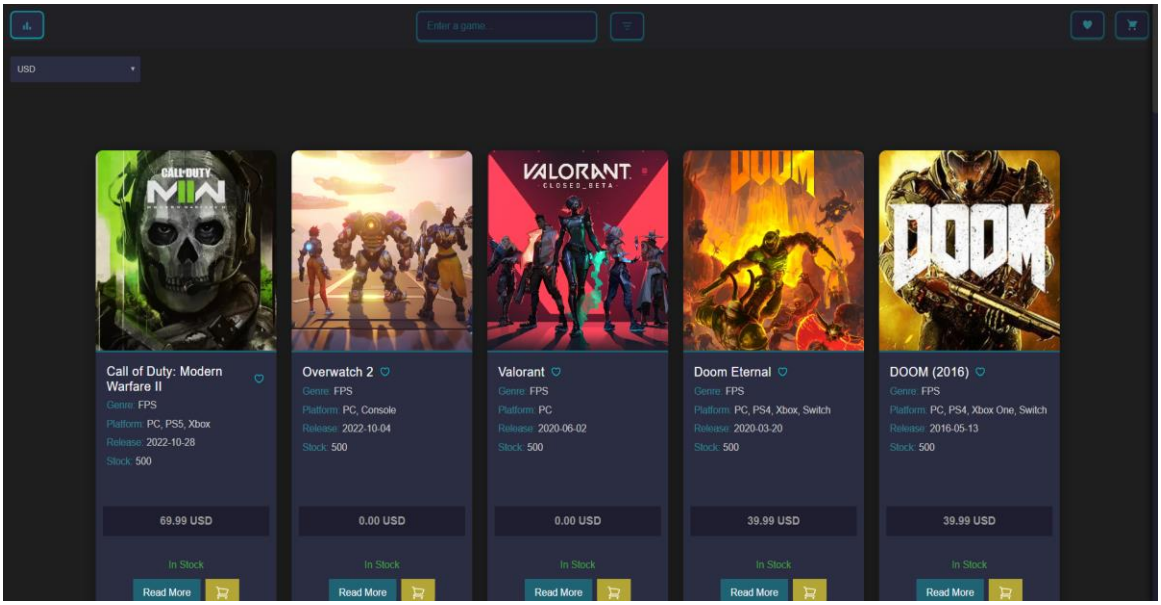
A footer középső részén három oszlopban rendezett linkek találhatók, amelyek kategóriákra vannak bontva. Minden kategória egy felső címkével kezdődik (ABOUT, HELP, LEGAL), amelyek világosszürke színnel, NAGYBETŰS formátumban jelennek meg.

1.2.7.4 Vizuális Elválasztás

A footer tartalom felett egy vékony, világosabb vonal (`#2a3142` körüli árnyalat) húzódik, amely vizuálisan elkülöníti a footer-t a fő tartalomtól. Ez a finom elválasztás professzionális megjelenést kölcsönöz az oldalnak.

1.3. Games main weboldal

1.3.1. A Games main weboldal navbarja, termékek megjelenítése, a szűrők, a wishlist és a kosár



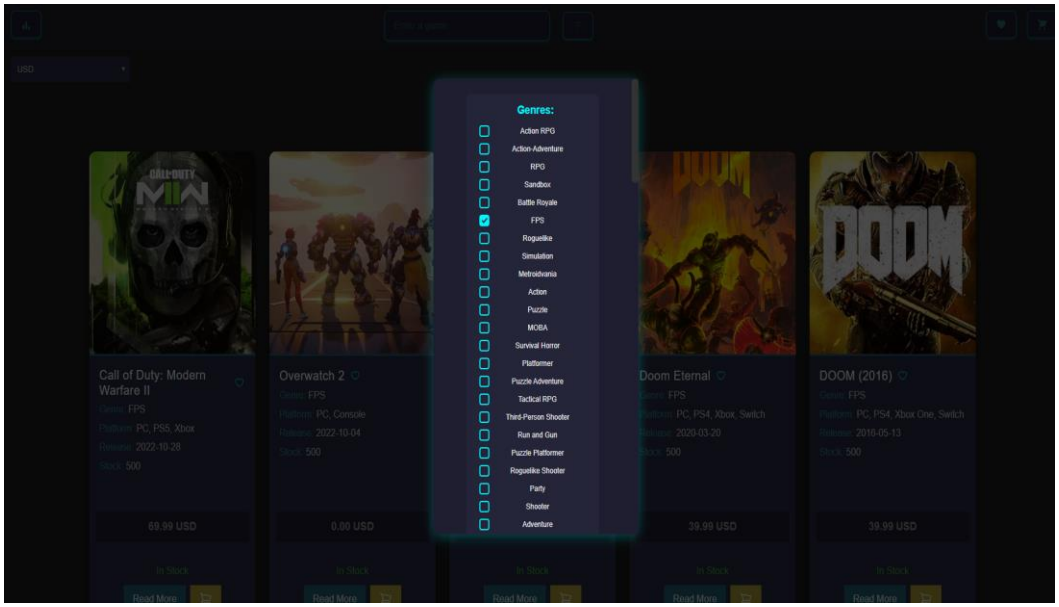
7. ábra - games main

1.3.1.1. Játéklista és böngésző felület

A játékok főoldalán a felhasználók egy áttekinthető, kártyaalapú felületen böngészhetik az elérhető digitális termékeket. Minden egyes játék külön kártyán jelenik meg, amely tartalmazza a legfontosabb információkat, mint például a játék neve, műfaja, platformja, megjelenési dátuma, raktárkészlete és ára. Ez a megoldás lehetővé teszi, hogy a felhasználók gyorsan összehasonlítsák az egyes játékokat, és könnyen megtalálják a számukra releváns tartalmat.

A kártyák vizuálisan egységes megjelenésűek, a játék borítóképével kiegészítve, amely segíti a gyors felismerést és javítja a felhasználói élményt. Az ár és a készletinformáció külön hangsúlyt kap, így a felhasználó azonnal láthatja, hogy az adott termék elérhető-e.

Keresési és szűrési lehetőségek



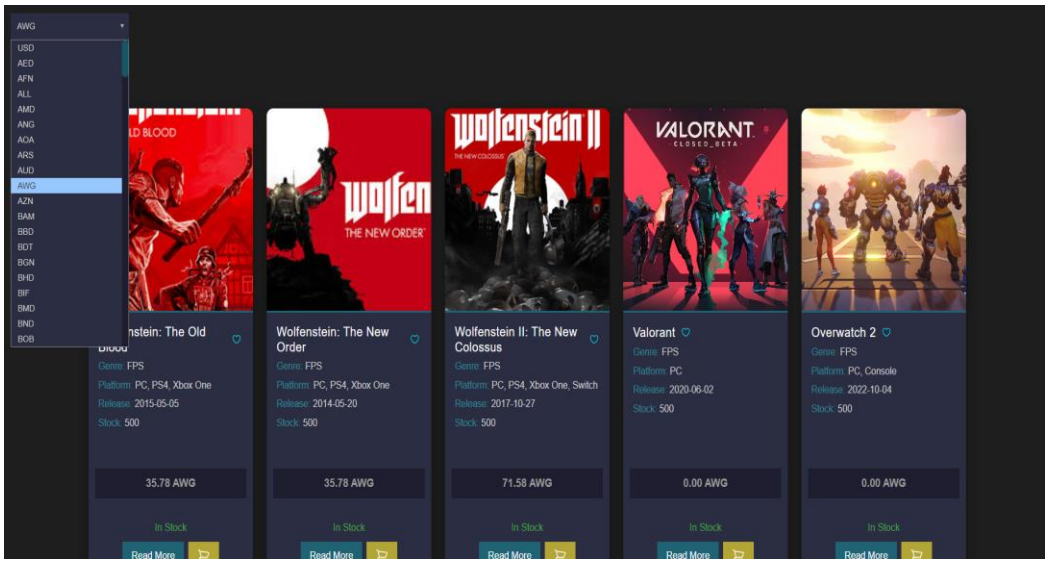
8. ábra - Filtering

Az oldal tetején elhelyezett keresőmező továbbra is lehetőséget biztosít arra, hogy a felhasználók név alapján keressenek játékokra. A keresés valós időben működik, így a begépelt kulcsszavak azonnal szűkítik a megjelenített találatokat, ami gyors és hatékony navigációt tesz lehetővé a játékok között.

A keresőmezőhöz kapcsolódó, kibővített szűrőmenü egy külön felugró felületen jelenik meg, amely letisztult és átlátható módon csoportosítja a különböző szűrési lehetőségeket. A felhasználók többek között műfaj (például Action RPG, FPS, Puzzle), platform (PC, PlayStation, Xbox, Nintendo), ár szerinti rendezés (növekvő, csökkenő, akciós), valamint megjelenési dátum alapján is szűkíthetik a kínálatot. Emellett lehetőség van név szerinti rendezésre (A–Z, Z–A), árintervallum megadására minimum és maximum értékekkel, illetve készletállapot szerinti szűrésre is.

Ez a komplex szűrőrendszer különösen nagyobb játékkínálat esetén hasznos, mivel jelentősen csökkenti a böngészéshez szükséges időt, és lehetővé teszi a pontos, célzott keresést. Az új UI-kialakítás nemcsak vizuálisan illeszkedik az oldal modern, gaming stílusához, hanem felhasználói élmény szempontjából is intuitívabbá és hatékonyabbá teszi a játékok közötti navigációt.

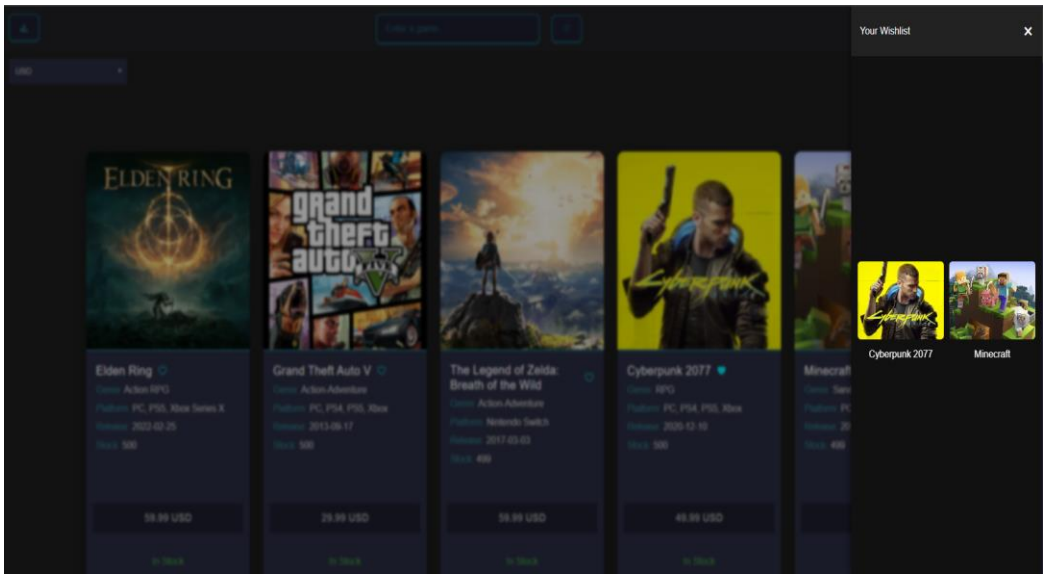
Pénznemválasztás



9. ábra - Currency Conversion

A felhasználói felület bal felső részén található pénznemválasztó segítségével a látogatók kiválaszthatják, hogy milyen valutában szeretnék megtekinteni az árakat. Ez a funkció különösen hasznos nemzetközi felhasználók számára, mivel az árak automatikusan az adott pénznemhez igazodnak, javítva ezzel a felhasználói élményt és az átláthatóságot.

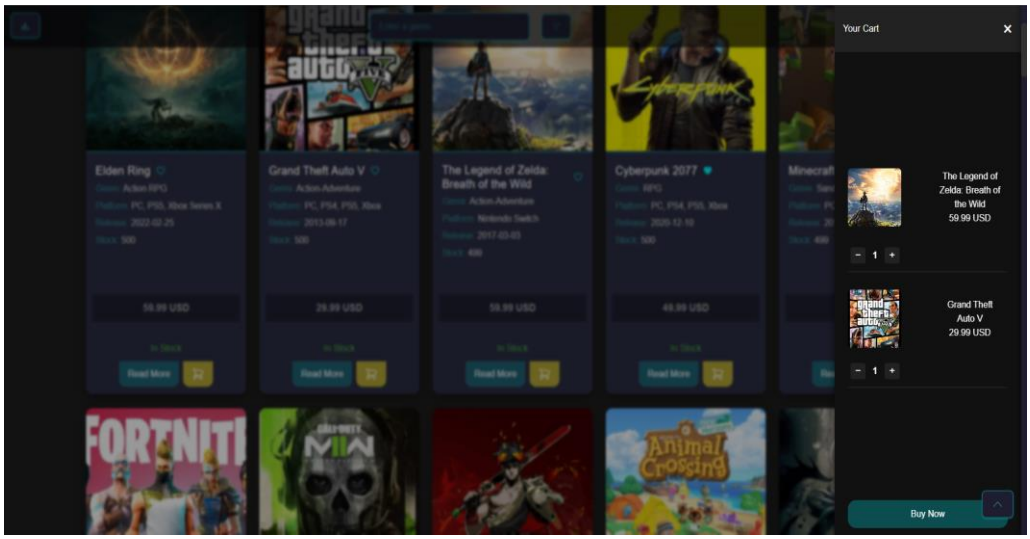
Wishlist funkció



10. ábra - Wishlist

Minden játék kártyáján elérhető egy szív ikon, amely segítségével a felhasználók hozzáadhatják az adott játékot a kívánságlistájukhoz (wishlist). Ez a funkció lehetővé teszi, hogy a felhasználók később könnyen visszatérjenek az őket érdeklő termékekhez anélkül, hogy újra meg kellene keresniük azokat. A wishlist különösen hasznos hosszabb böngészési folyamat során, illetve akkor, ha a felhasználó későbbi vásárlást tervez.

Kosár funkció

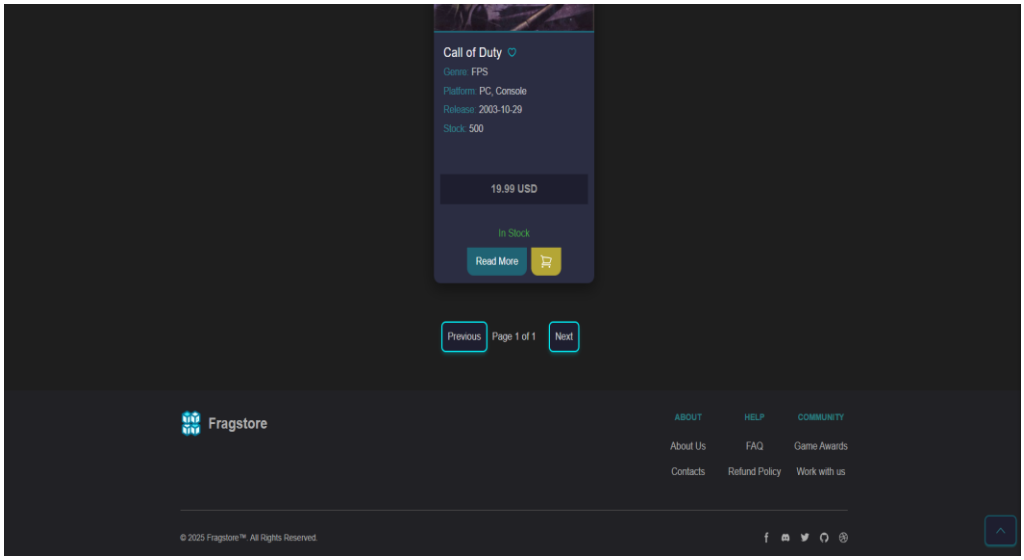


11. ábra - Shopping Cart

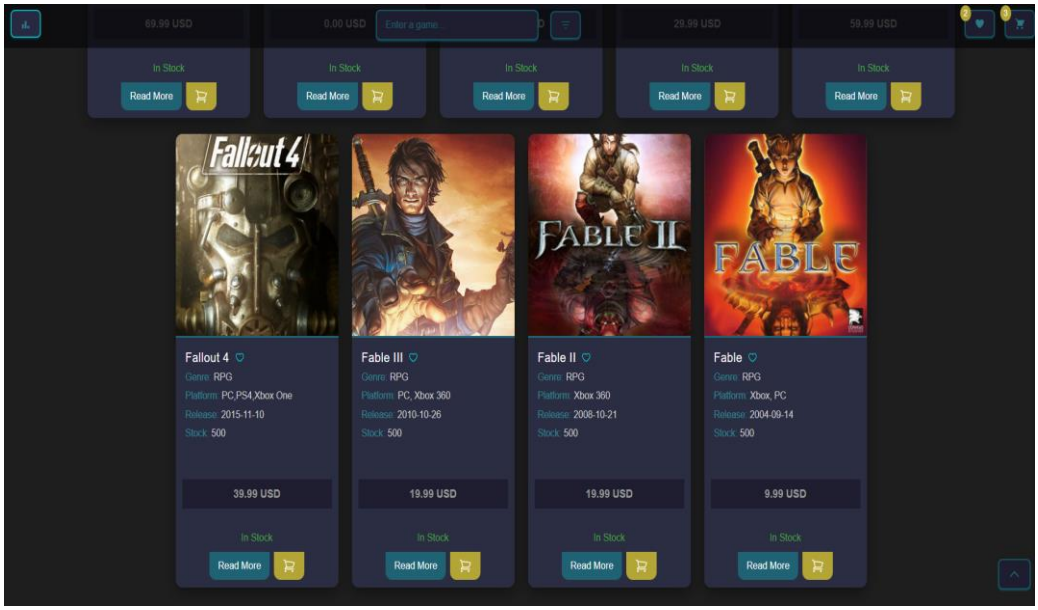
A játék kártyák alsó részén található kosár ikon segítségével a felhasználók a kiválasztott termékeket a kosárba helyezhetik. A kosár funkció lehetővé teszi

több termék egyidejű kezelését, valamint a kiválasztott játékok áttekintését vásárlás előtt. Bár az oldalon jelenleg nem érhető el tényleges fizetési lehetőség, a kosár működése demonstrálja egy webáruház alapvető vásárlási folyamatát.

A grid változása, a lapozó és a footer avagy lábléc



Dinamikus kártyaelrendezés kevés találat esetén

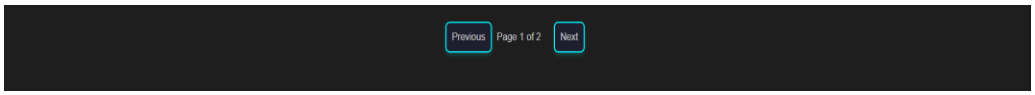


Amennyiben a kiválasztott szűrési feltételek vagy keresési paraméterek eredményeként kevesebb mint öt játék kerül megjelenítésre, az oldal automatikusan módosítja az elrendezést. Ilyen esetekben a rendszer nem hagy

üres helyeket a felületen, hanem a megjelenő kártyákat egyenletesen osztja el az oldalon.

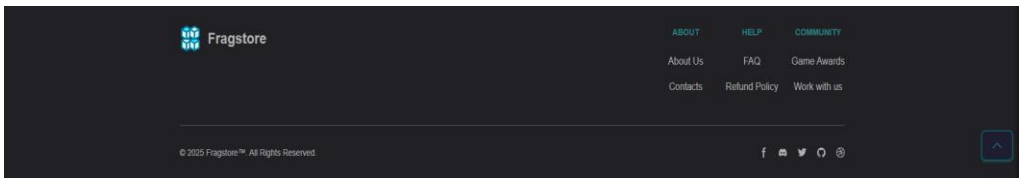
Ez a dinamikus elrendezés biztosítja, hogy a felhasználói felület esztétikus és átlátható maradjon akkor is, ha csak néhány találat érhető el. A megoldás javítja a vizuális megjelenést és a felhasználói élményt, mivel elkerüli az aránytalan üres területek kialakulását, és következetes megjelenést biztosít minden esetben.

Lapozó (Pagination)



Az oldal alsó részén elhelyezett lapozó segítségével a felhasználók könnyedén navigálhatnak az egyes oldalak között. A lapozó „Previous” és „Next” gombokat tartalmaz, valamint egy visszajelzést ad az aktuális oldal számáról. Ez a megoldás lehetővé teszi a tartalom strukturált megjelenítését, és segít elkerülni a túl hosszú, nehezen átlátható oldalak használatát.
A lapozás különösen hasznos nagyobb játékkínálat esetén, mivel egyszerre csak egy meghatározott számú kártya jelenik meg. Ezáltal az oldal gyorsabban betöltődik, és a felhasználók könnyebben áttekinthetik a kínálatot anélkül, hogy folyamatos görgetésre lenne szükség.

Lábléc (Footer)



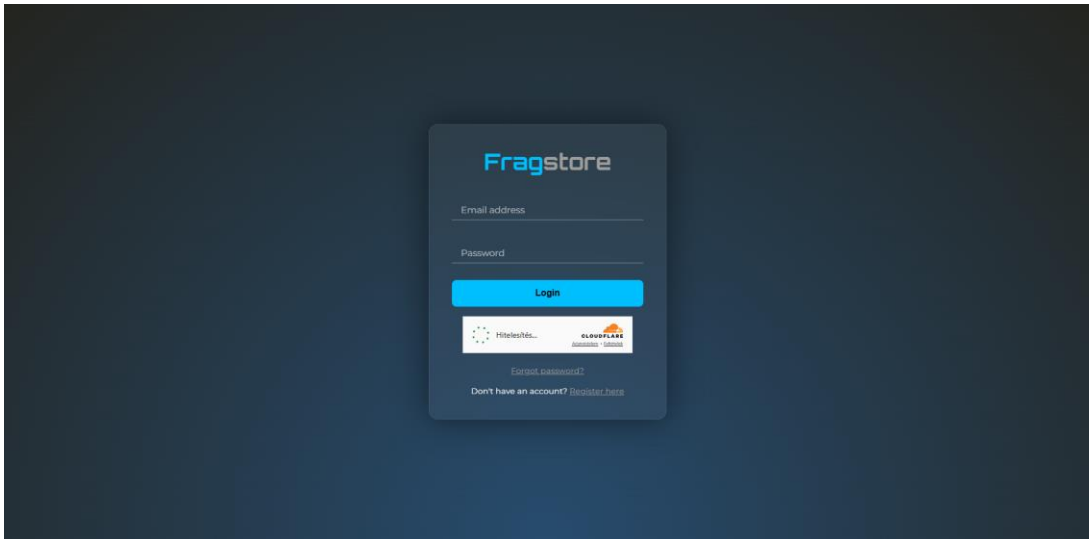
12. ábra - Footer

Az oldal alján elhelyezkedő lábléc kiegészítő információkat és gyors elérési lehetőségeket biztosít a felhasználók számára. A lábléc tartalmazza az oldal nevét és arculati elemét, valamint különböző információs menüpontokat, mint például az „About Us”, „FAQ”, „Refund Policy” és a „Contacts” menüpontok.

Ezen kívül a láblécben közösségi média ikonok is helyet kaptak, amelyek lehetőséget biztosítanak a felhasználóknak arra, hogy kapcsolatba lépjenek az

oldal közösségi felületeivel. A lábléc célja, hogy az oldal alján is biztosítsa a navigációt és a fontos információk gyors elérését, ezáltal növelve a weboldal használhatóságát és professzionális megjelenését.

1.4 Log in weboldal



13. ábra - Log In

1.4.1. Bejelentkezési felület (Login)

A Fragstore webalkalmazás bejelentkezési felülete egy letisztult, középre igazított űrlap formájában jelenik meg, amely biztosítja az egyszerű és gyors belépést a felhasználók számára. A felület minimalista kialakítása segít abban, hogy a felhasználó figyelme kizárólag a bejelentkezéshez szükséges adatok megadására összpontosuljon, elkerülve a felesleges vizuális zavaró elemeket.

A bejelentkezéshez a felhasználónak meg kell adnia az e-mail címét és a jelszavát, majd a „Login” gombra kattintva kezdeményezheti a hitelesítési folyamatot. Az űrlap kialakítása egyértelmű visszajelzést ad a mezők funkciójáról, ezzel csökkentve a hibás adatbevitel esélyét.

1.4.2. Biztonsági ellenőrzés (CAPTCHA)

A bejelentkezési folyamat részeként egy biztonsági ellenőrző mechanizmus is beépítésre került, amelynek célja az automatizált támadások és visszaélések

megelőzése. A CAPTCHA ellenőrzés biztosítja, hogy a bejelentkezési kísérletet valódi felhasználó hajtja végre, ezáltal növelve a rendszer biztonságát.

Ez a megoldás különösen fontos olyan funkciók esetében, ahol felhasználói fiókokhoz és személyes adatokhoz történik hozzáférés, mivel jelentősen csökkenti a brute force és bot alapú támadások kockázatát.

1.4.3. Elfelejtett jelszó funkció

A bejelentkezési felület tartalmaz egy „Forgot password?" hivatkozást is, amely lehetőséget biztosít a felhasználók számára jelszavuk visszaállítására. Amennyiben a felhasználó elfelejti a jelszavát, ezen funkció segítségével új hitelesítő adatokat igényelhet, így elkerülhető a fiók végleges elvesztése.

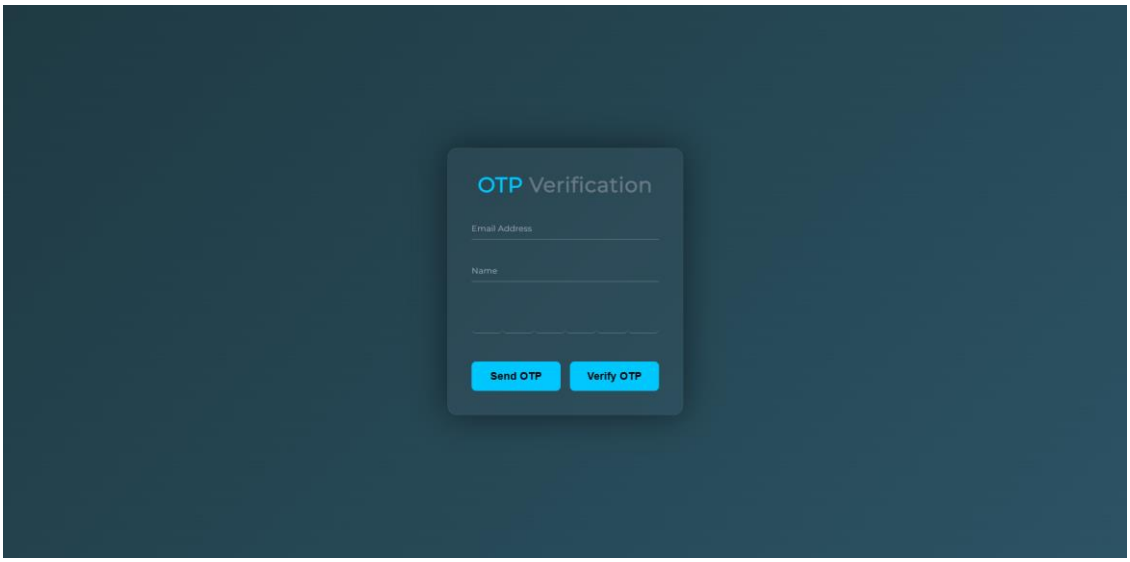
Ez a funkció növeli az oldal felhasználóbarát jellegét, és biztosítja, hogy a felhasználók probléma esetén is könnyedén hozzáférjenek fiókjukhoz.

1.4.4. Regisztrációs lehetőség

A bejelentkezési oldal alsó részén elhelyezett „Register here" hivatkozás lehetőséget biztosít az új felhasználók számára, hogy regisztrációt kezdeményezzenek. Ez a megoldás egyértelműen elkülöníti a meglévő és az új felhasználók számára elérhető funkciókat, és segíti a gyors navigációt a bejelentkezés és a regisztráció között.

A regisztrációs folyamat célja, hogy a felhasználók személyre szabott funkciókat érhessenek el, mint például a kívánságlista (wishlist) használata vagy a kosár tartalmának mentése.

1.4.5. OTP Verification (OTP ellenőrzés)



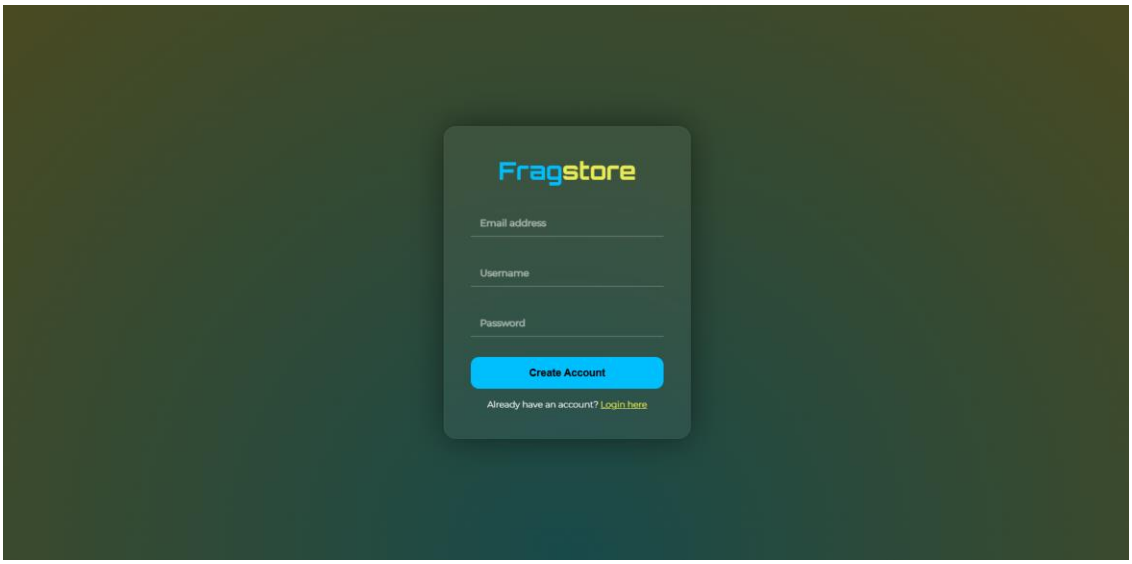
14. ábra - OTP Verification

A bejelentkezési folyamat biztonságának növelése érdekében a Fragstore webalkalmazás kétlépcsős azonosítást alkalmaz. A sikeres e-mail cím és jelszó megadása után a felhasználót egy OTP (One-Time Password) hitelesítő felületre irányítja a rendszer, ahol további megerősítés szükséges a belépés befejezéséhez.
Az OTP hitelesítési oldalon a felhasználónak meg kell adnia az e-mail címét és a nevét, majd a „Send OTP” gombra kattintva egy egyszer használatos kódot igényelhet. A rendszer ezt a kódot a megadott e-mail címre küldi el, biztosítva ezzel, hogy csak a jogosult felhasználó férhessen hozzá a fiókjához.

1.4.6. OTP kód megadása és ellenőrzése a

Az e-mailben kapott egyszer használatos kódot a felhasználónak az OTP mezőkbe kell beírnia, majd a „Verify OTP” gomb megnyomásával véglegesítheti a bejelentkezést. A kód csak korlátozott ideig érvényes, így csökkentve a visszaélések lehetőségét.
A kétlépcsős azonosítás jelentősen növeli a felhasználói fiókok védelmét, mivel egy esetleges jelszókompromittálás esetén is megakadályozza az illetéktelen hozzáférést. Ez a megoldás különösen fontos olyan funkciók esetében, amelyek személyes adatokat vagy felhasználói beállításokat kezelnek.

1.5. Regisztrációs felület (Sign Up)



15. ábra - Sign Up

A Fragstore webalkalmazás regisztrációs felülete lehetőséget biztosít az új felhasználók számára fiók létrehozására. Az oldal kialakítása letisztult és felhasználóbarát, amely segíti a gyors és egyértelmű adatbevitelt. A középre igazított űrlap biztosítja, hogy a felhasználó figyelme kizárólag a regisztrációhoz szükséges mezőkre összpontosuljon.

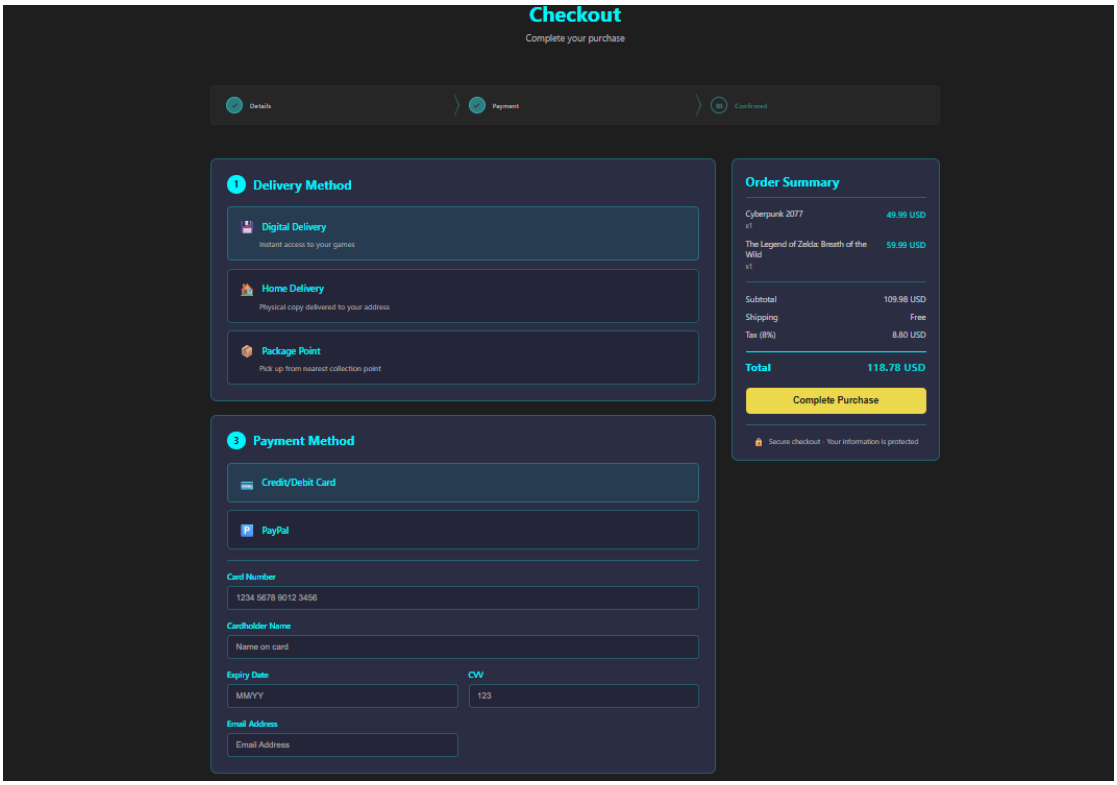
A regisztráció során a felhasználónak meg kell adnia az e-mail címét, egy választott felhasználónevet, valamint egy jelszót. Ezek az adatok képezik a felhasználói fiók alapját, amely később hozzáférést biztosít az oldal személyre szabott funkcióihoz.

1.5.1. Fiók létrehozása és visszajelzés

A „Create Account” gombra kattintva a rendszer ellenőrzi a megadott adatok érvényességét, majd sikeres regisztráció esetén létrehozza a felhasználói fiókot. A felhasználó ezt követően visszajelzést kap a művelet sikerességéről, és lehetősége nyílik a bejelentkezésre a már meglévő hitelesítő adatokkal.

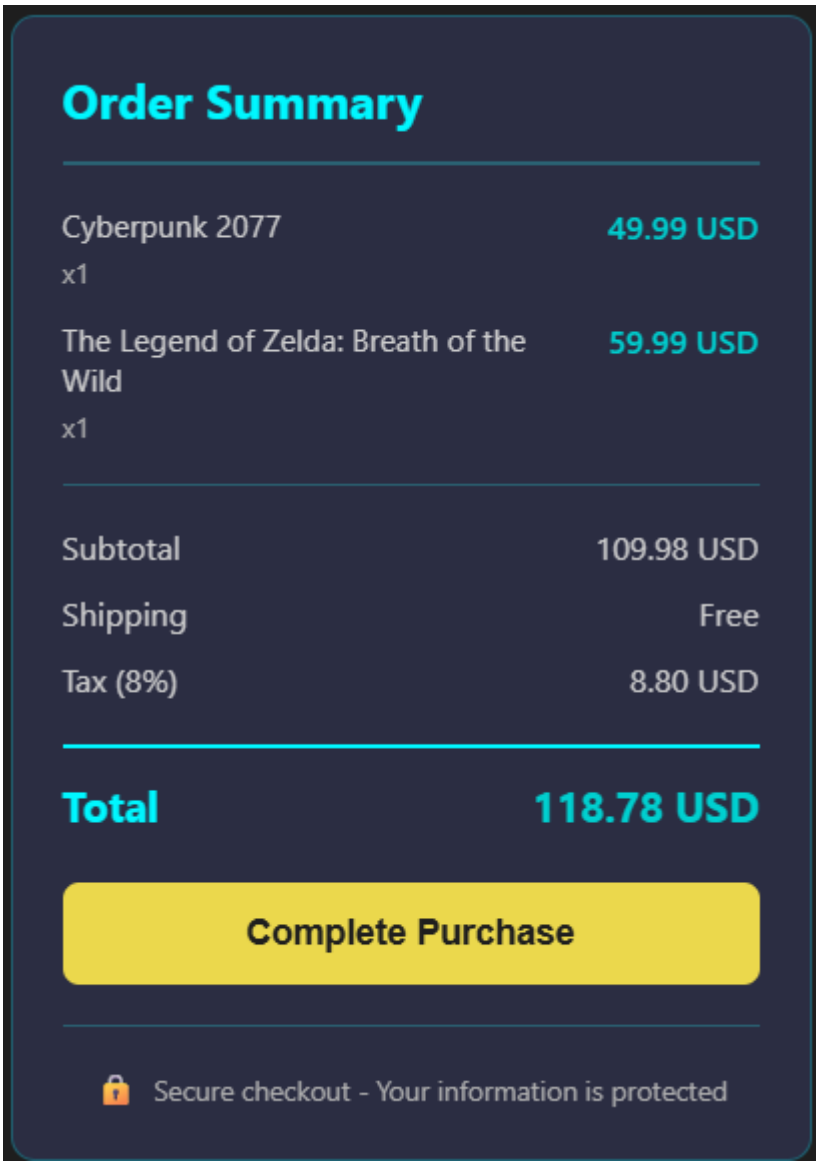
A regisztrációs felület alsó részén elhelyezett „Login here” hivatkozás megkönnyíti a navigációt a bejelentkezési oldalra azok számára, akik már rendelkeznek felhasználói fiókkal, ezáltal biztosítva a zökkenőmentes felhasználói élményt.

1.6. Cart



16. ábra - Transaction

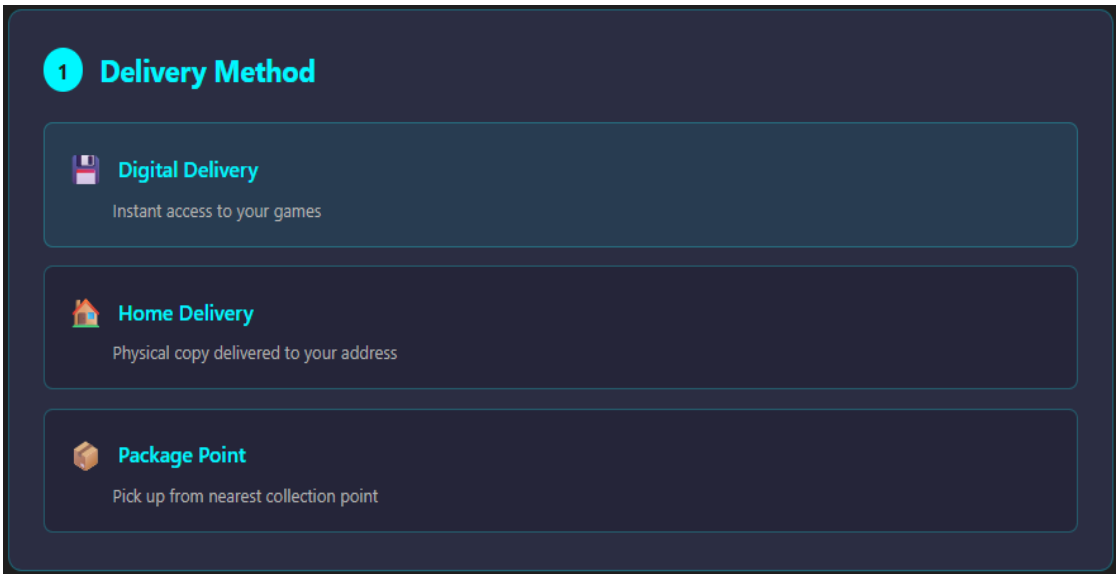
1.6.1 Fizetési folyamat (Checkout)



17. ábra - Order Summary

- A Fragstore webalkalmazás fizetési oldala lehetőséget biztosít a felhasználók számára a kiválasztott termékek vásárlási folyamatának véglegesítésére. Az oldal célja, hogy a vásárlás lépései átláthatóan, logikus sorrendben és biztonságosan történjenek meg. A felület kialakítása segíti a felhasználót abban, hogy egyetlen oldalon áttekintse és befejezze a vásárlást.

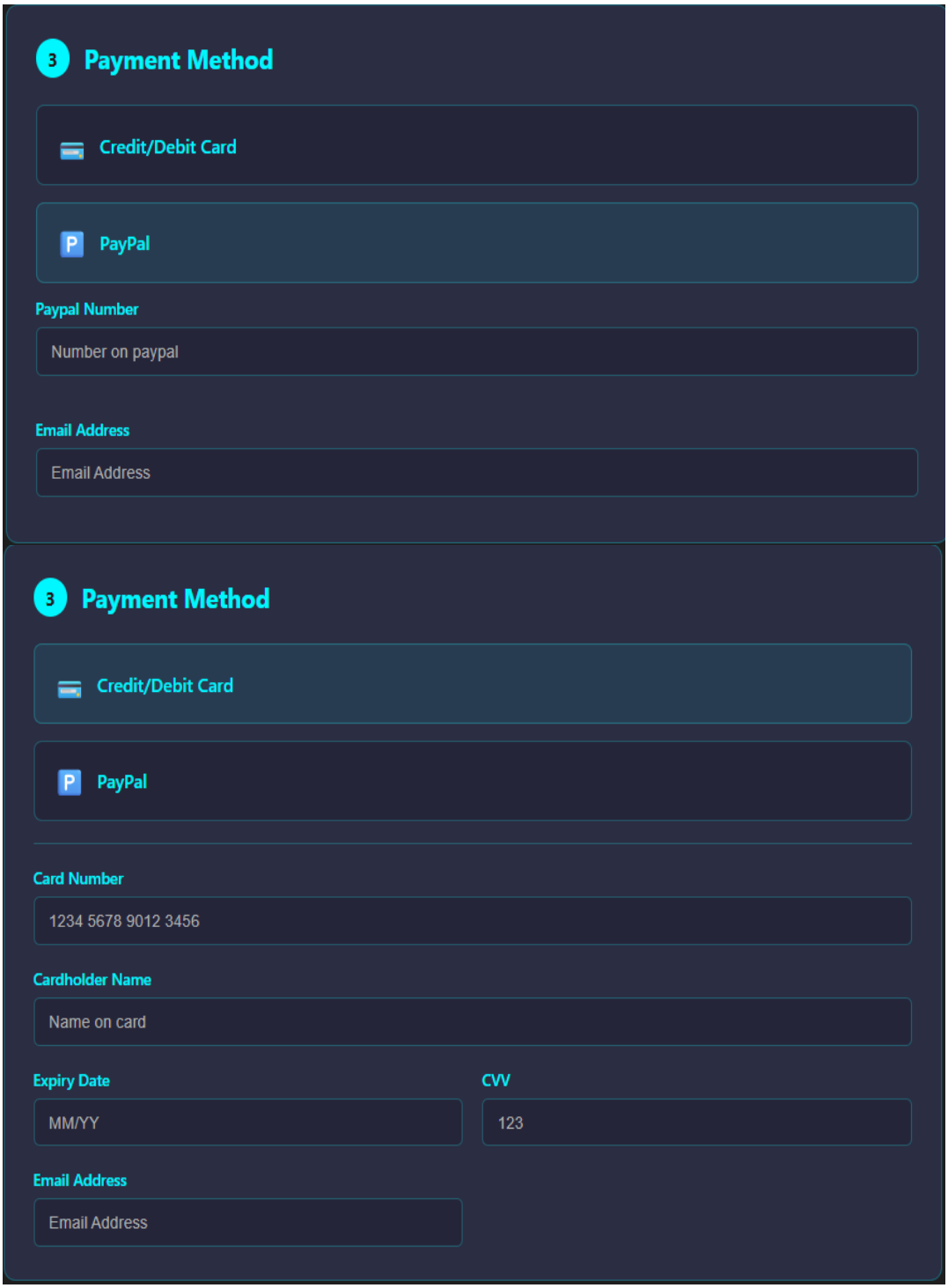
1.6.2 Szállítási mód kiválasztása

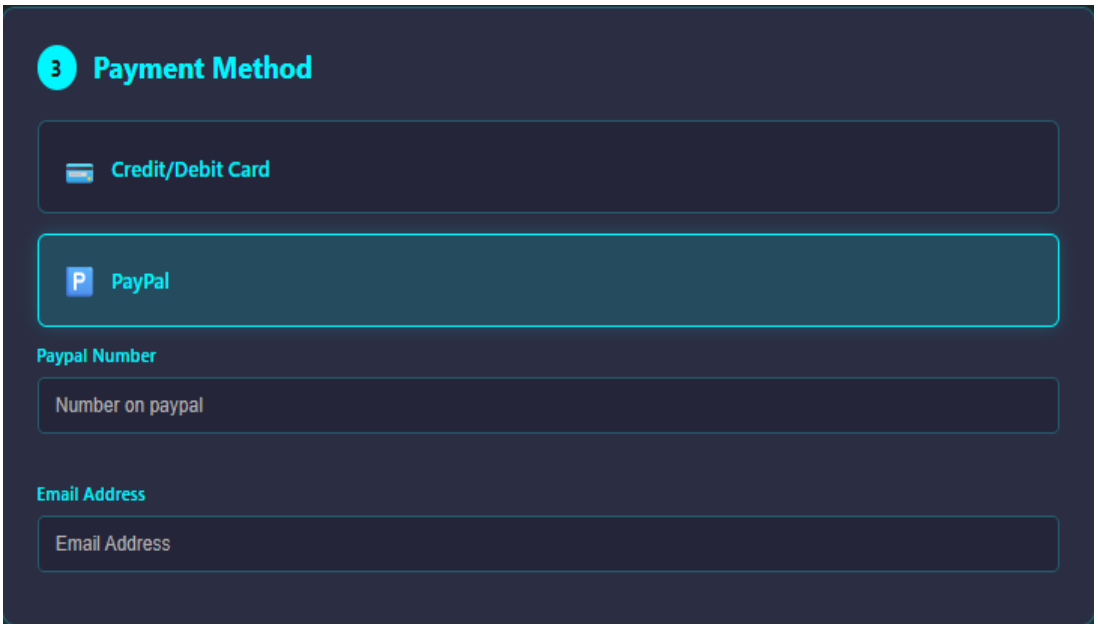


18. ábra - Delivery Method

- A fizetési folyamat első lépéseként a felhasználó kiválaszthatja a kívánt szállítási módot. A rendszer több lehetőséget is kínál, például digitális kézbesítést, házhoz szállítást vagy csomagpontos átvételt. A digitális kézbesítés azonnali hozzáférést biztosít a megvásárolt játékokhoz, míg a fizikai szállítási opciók hagyományos kézbesítési megoldásokat kínálnak.
- A kiválasztott szállítási mód vizuálisan kiemelésre kerül, így a felhasználó egyértelmű visszajelzést kap arról, hogy melyik opció aktív.

1.6.3 Fizetési mód megadása





19. ábra - Payment Method

- A következő lépésben a felhasználó kiválaszthatja a fizetési módot. A felület támogatja a bankkártyás fizetést, valamint az online fizetési szolgáltatások használatát. Bankkártyás fizetés esetén a felhasználónak meg kell adnia a kártya adatait, beleértve a kártyaszámot, a kártyatulajdonos nevét, a lejárati dátumot és a biztonsági kódot.
- A fizetési űrlap logikusan felépített, így a szükséges adatok megadása gyorsan és egyszerűen elvégezhető. Az e-mail cím megadása biztosítja, hogy a felhasználó visszaigazolást kapjon a vásárlásról.

1.6.4 Rendelés összegzése (Order Summary)

- Az oldal jobb oldalán egy rendelési összegzés található, amely részletesen bemutatja a kiválasztott termékeket és a hozzájuk tartozó költségeket. Az összegzés tartalmazza a részösszeget, az esetleges szállítási díjat, az adókat, valamint a végösszeget.
- Ez a megoldás lehetővé teszi, hogy a felhasználó még a vásárlás véglegesítése előtt ellenőrizze a rendelés minden részletét, elkerülve ezzel a félreértéseket vagy hibás tranzakciókat.

1.6.5 Vásárlás véglegesítése

- A „Complete Purchase” gombra kattintva a felhasználó véglegesítheti a vásárlást. A rendszer a gomb alatt egy biztonsági visszajelzést is megjelenít, amely tájékoztatja a felhasználót arról, hogy az adatai védettek, és a tranzakció biztonságos környezetben történik.
- Fontos megjegyezni, hogy a Fragstore projekt esetében a fizetési funkció demonstrációs célt szolgál, így tényleges pénzügyi tranzakció nem történik. A fizetési oldal célja a vásárlási folyamat működésének bemutatása és a felhasználói élmény szemléltetése.

1.7 Pac Man

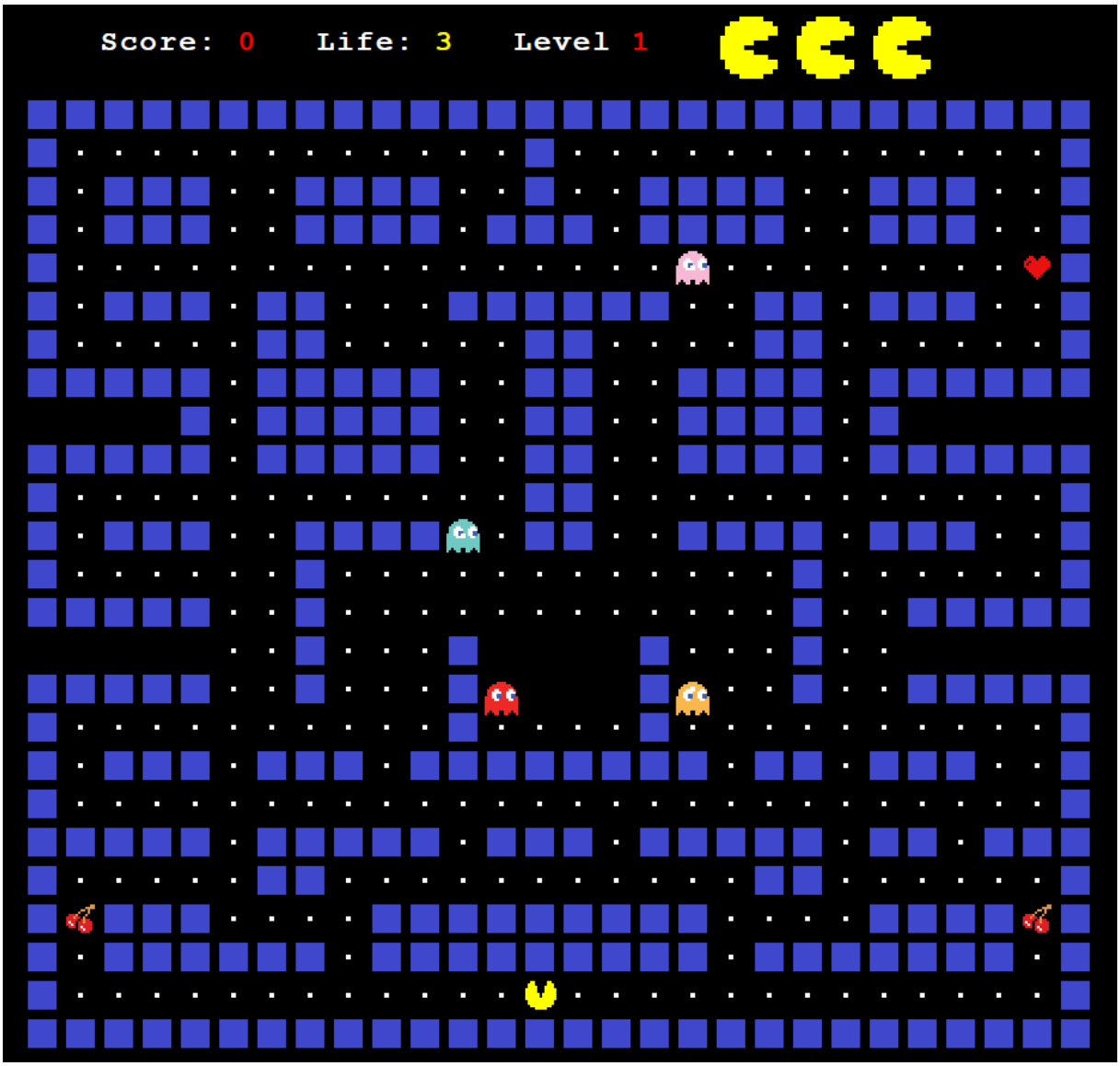
1.7.1 A Pac Man menü

20. ábra - Pac Man Menu

A Pac-Man játék indításakor a felhasználót egy letisztult, klasszikus arcade stílusú kezdőképernyő fogadja. A sötét háttér és az élénk színek kontrasztja azonnal megteremti a retró hangulatot, amely hűen idézi meg az eredeti Pac-Man játék vizuális világát.

A képernyő középpontjában a **„Pac Man”** felirat jelenik meg élénk sárga színnel, amely egyértelműen jelzi a játék típusát és az easter egg jellegét. A nagy méretű, egyszerű tipográfia könnyen felismerhető, és azonnal a játékra irányítja a felhasználó figyelmét.

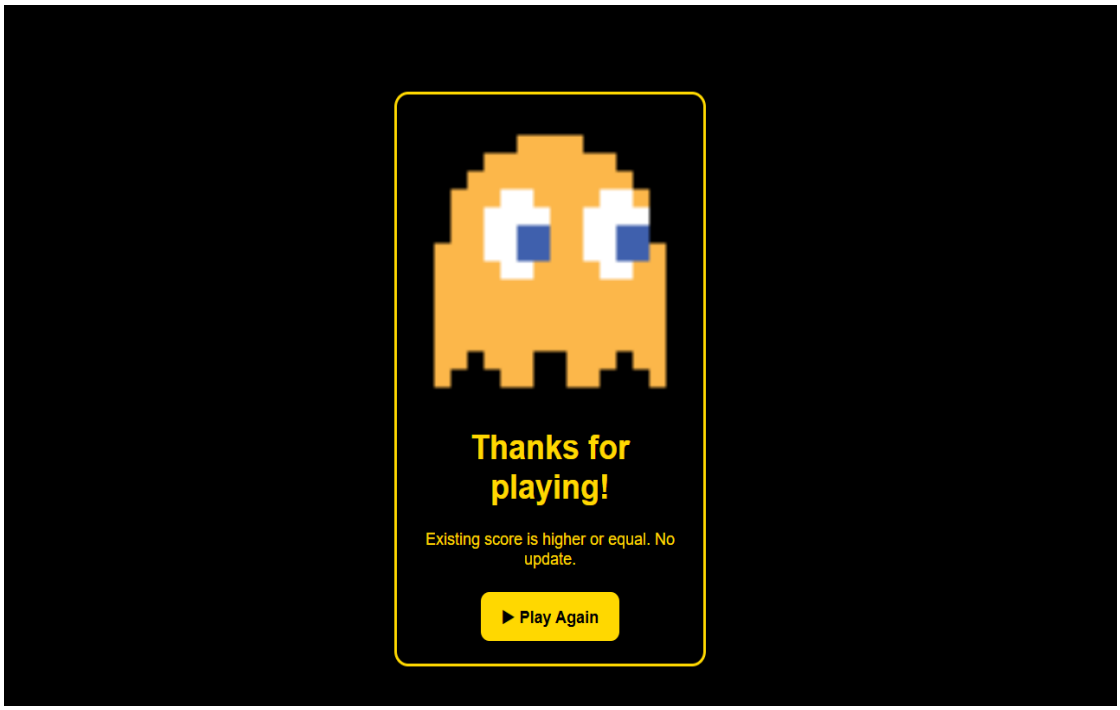
1.7.2. Pac Man játék



21. ábra - Pac Man field

A Pac-Man pálya klasszikus rácsos elrendezésre épül, kék falakkal határolt útvonalakkal és jól elkülönülő folyosókkal, amelyek irányítják a játékmenetet és stratégiai mozgásra kényszerítik a játékost. A pályán elhelyezett pontok jelzik a begyűjthető elemeket, míg a különleges tárgyak (például gyümölcsök és power-upok) extra pontokat vagy ideiglenes előnyöket biztosítanak. A szellemek a központi zónából indulnak, eltérő viselkedési mintákkal, ami folyamatos kihívást jelent a játékos számára. A felső HUD egyértelműen megjeleníti a pontszámot, az életek számát és az aktuális szintet, így a játékos mindig tisztában van az állapotával, miközben a letisztult, retró vizuális stílus hűen idézi meg az eredeti arcade Pac-Man élményt.

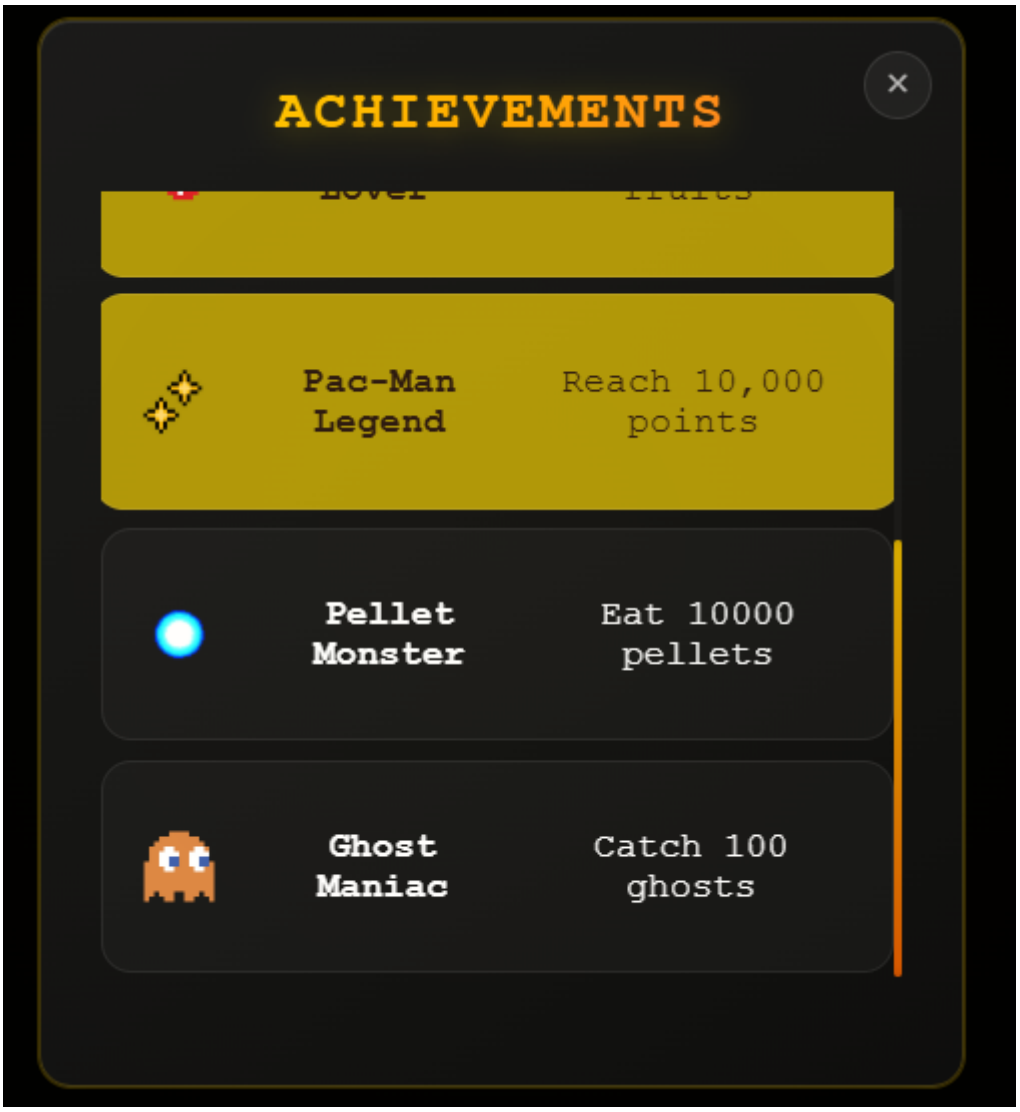
1.7.3 Halál utáni menü



22. ábra - Pac Man Game Over

A játékos elvesztése után megjelenő „Game Over” képernyő egy letisztult, központosan elhelyezett panelen jelenik meg, amely egy Pac-Man stílusú szellem grafikával és jól látható „Thanks for playing!” üzenettel zárja le a játékmenetet. A képernyő visszajelzést ad a pontszám állapotáról, jelezve, hogy az aktuális eredmény nem haladta meg a korábbi legjobb pontszámot, így nem történt frissítés. Az alsó részen elhelyezett „Play Again” gomb egyértelmű és azonnali lehetőséget biztosít az újrajátszásra, ezzel fenntartva a játék folytonosságát és motiválva a felhasználót egy újabb próbálkozásra.

1.7.4 Achievement lista



23. ábra - Pac Man Achievements

Ez a Pac-Man achievement (teljesítmény) rendszerének felülete, amely vizuálisan és egyértelműen mutatja a játékos előrehaladását. A megoldott achievementek arany színnel jelennek meg, ezzel jelezve, hogy a játékos már teljesítette az adott kihívást (például Pac-Man Legend – 10 000 pont elérése). A még nem teljesített achievementek szürke színűek, így azonnal elkülönülnek a megszerzett jutalmaktól, és motivációt adnak a további játékhoz. Minden achievementhez tartozik egy ikon, egy elnevezés és egy rövid leírás, amely pontosan meghatározza a teljesítés feltételét (például pelletek vagy szellemek számának elérése). A felület letisztult, retro stílusú megjelenése illeszkedik a Pac-Man vizuális világához, miközben gyors áttekintést biztosít a játékos számára.

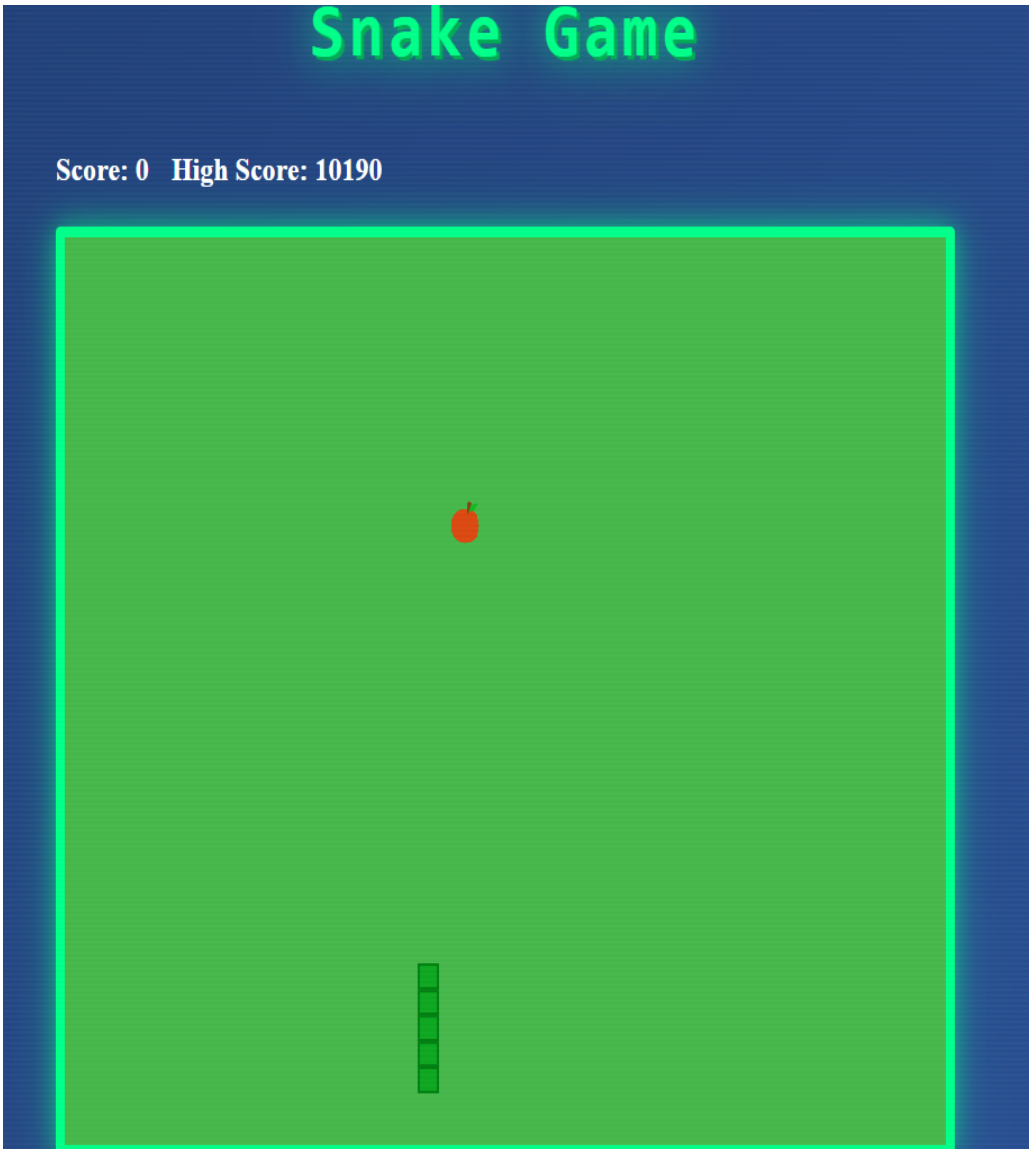
1.8 Snake

1.8.1 A menü

24. ábra - Snake Menu

A Snake Game kezdőmenüje egy letisztult, jól strukturált felületen jelenik meg, amely azonnal áttekinthetővé teszi a játék állapotát és indítási lehetőségeit. A felső részen a játék címe („Snake Game”), valamint az aktuális pontszám és a legjobb eredmény (High Score) látható, ami motiválja a játékost a fejlődésre. A középen elhelyezett indító panel lehetőséget biztosít a nehézségi szint kiválasztására (Easy, Medium, Hard), így a játékos saját képességeihez igazíthatja a játékmenetet. Az egyértelműen kiemelt „Play” gombbal a játék gyorsan elindítható, a vizuális hangsúlyok pedig segítik az intuitív használatot és a felhasználói élményt.

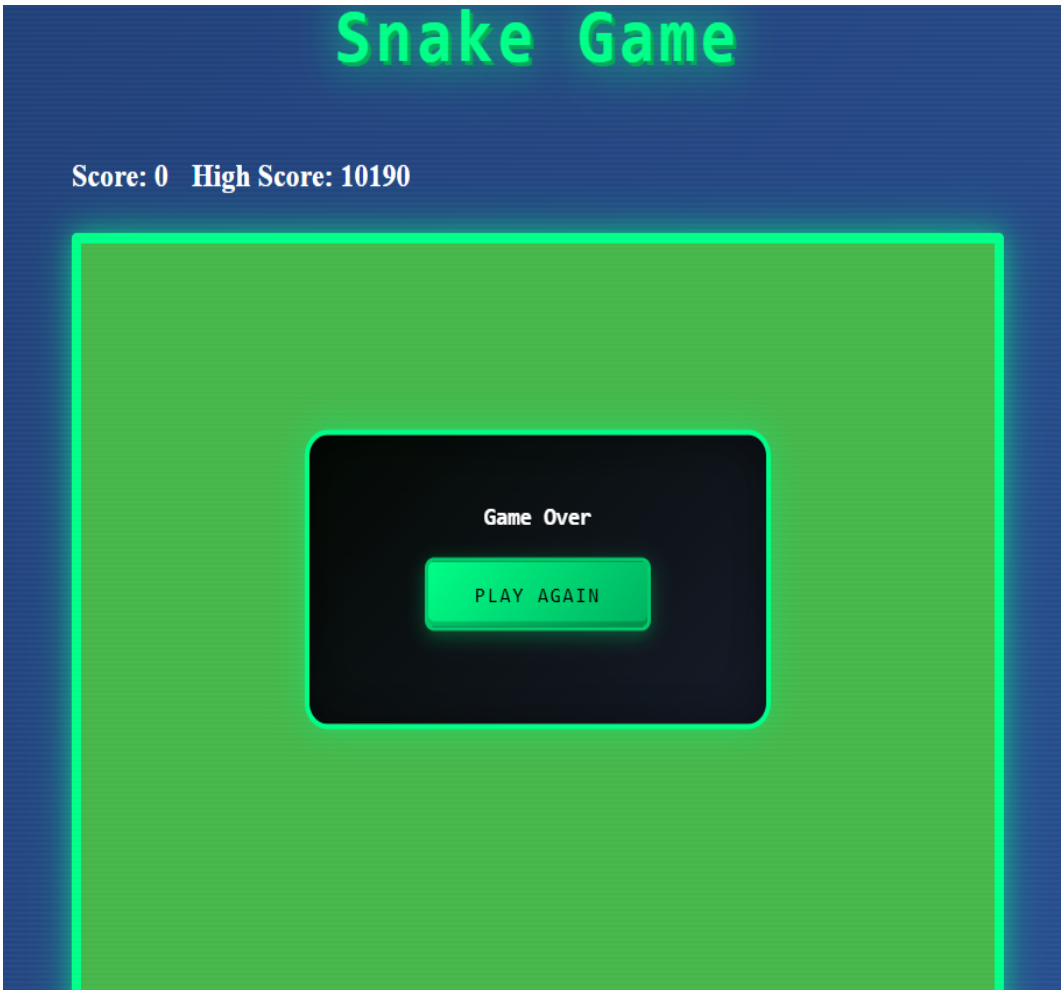
1.8.2 A snake játék maga



25. ábra - Snake game

A Snake Game játékmenete egy klasszikus, felülnézetes arénában zajlik, ahol a játékos egy folyamatosan növekvő kígyót irányít. A cél az, hogy a pályán megjelenő ételt (alma) begyűjtse, amellyel a kígyó hossza és a pontszám növekszik. A játéktér jól elkülönülő, világos kerettel van határolva, amely egyértelműen jelzi a mozgási határokat, míg a felső sávban az aktuális pontszám és a legjobb eredmény (High Score) folyamatosan nyomon követhető. A játék nehézségétől függően a kígyó mozgási sebessége változik, így a játék fokozatosan válik kihívássá, miközben megőrzi az egyszerű, könnyen érthető irányítást és a klasszikus játékélményt.

1.8.3. Game over menü

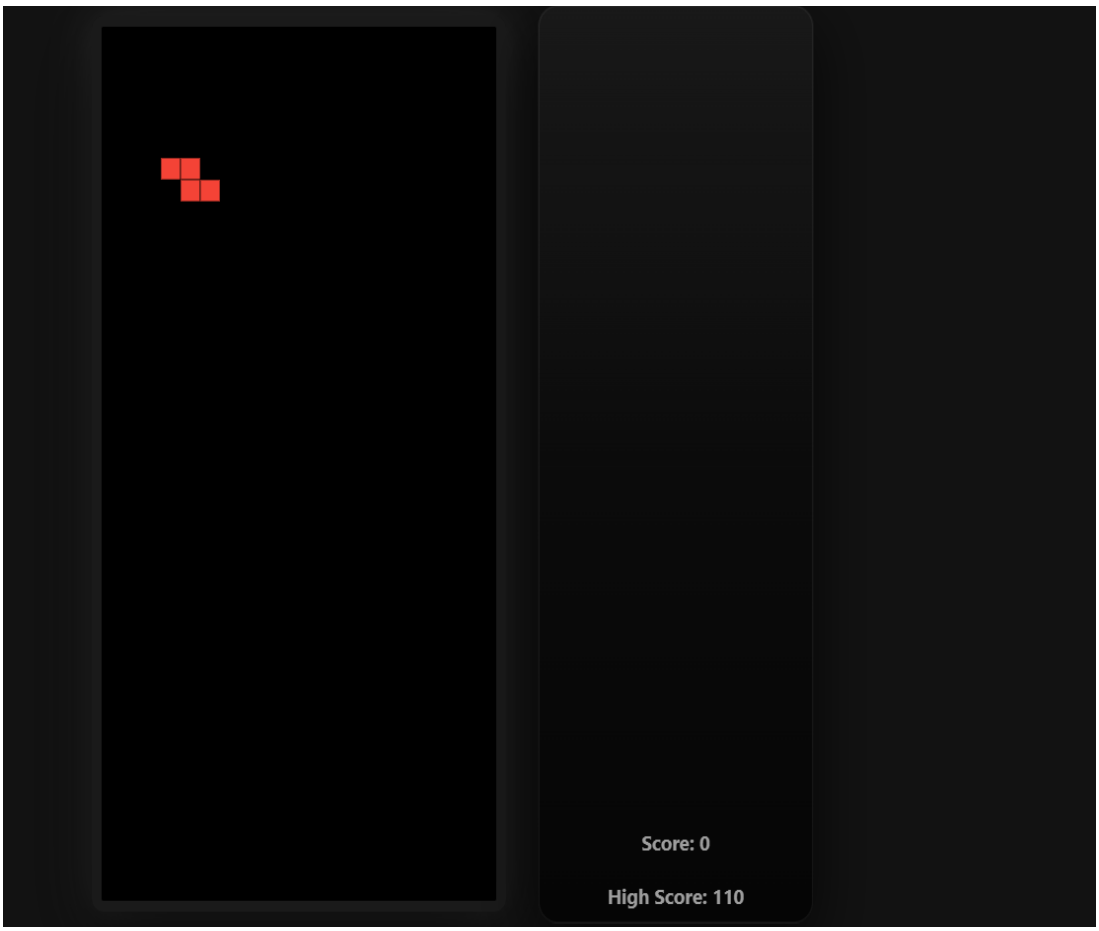


26. ábra - Snake Game Over

A Snake Game *Game Over* képernyője egyértelmű visszajelzést ad a játék befejezéséről, miközben megőrzi az egységes vizuális stílust. A játéktér fölött továbbra is látható az aktuális pontszám és a legjobb eredmény (High Score), így a játékos azonnal össze tudja hasonlítani teljesítményét a korábbi próbálkozásokkal. A középen megjelenő, kontrasztos panelen a „Game Over" felirat jelzi a játék végét, alatta pedig egy hangsúlyos „Play Again" gomb található, amely lehetőséget biztosít az azonnali újrakezdésre. Ez a megoldás minimalizálja a megszakítást, és ösztönzi a játékost a további próbálkozásokra, fenntartva a játék dinamikáját és motiváló jellegét.

1.9 Tetris

1.9.1 A játék



27. ábra - Tetris Game

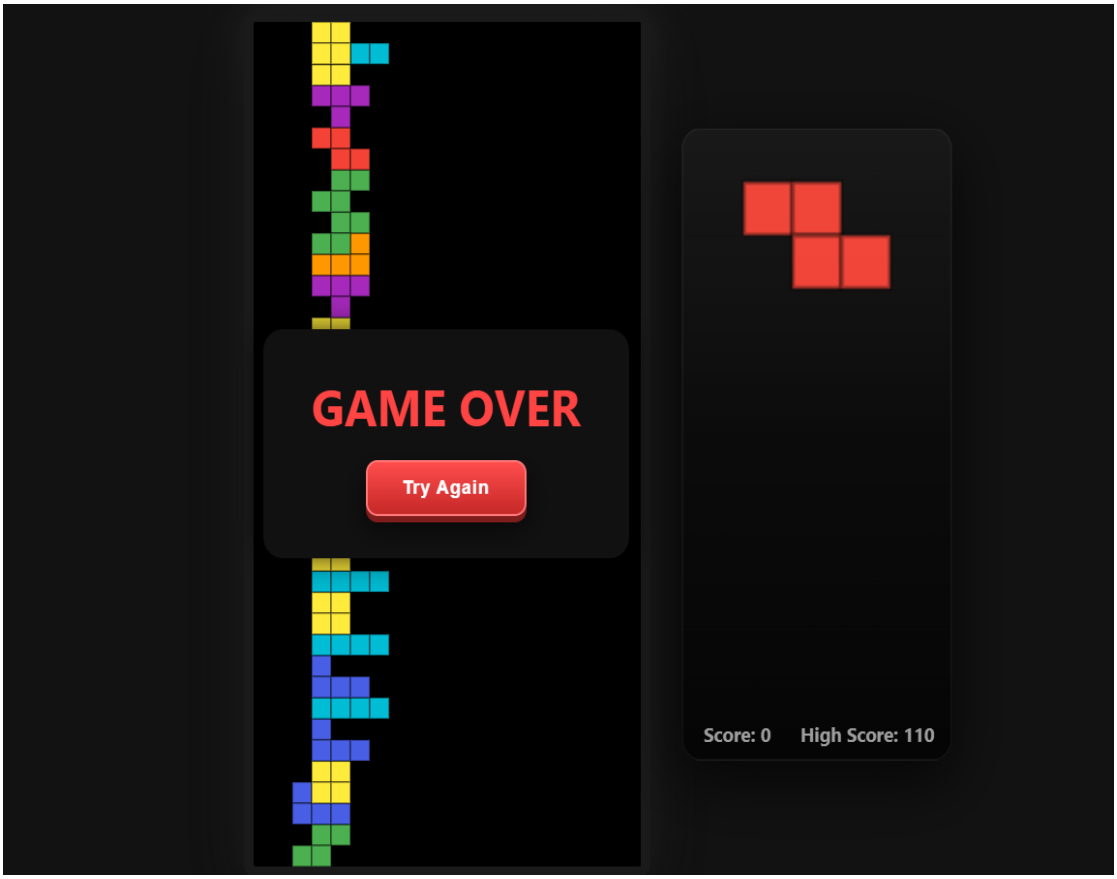
A Tetris játék felülete tudatosan letisztult és minimalista kialakítást kapott, amely a klasszikus Tetris élményt helyezi előtérbe, felesleges vizuális zavaró elemek nélkül. A képernyő bal oldalán található a fő játéktér, ahol a tetrominók megjelennek és folyamatosan lefelé haladnak. A fekete háttér és az élénk színű blokkok erős kontrasztot biztosítanak, így a játékos könnyen felismeri az aktuális alakzatot és annak pozícióját. Ez a megoldás nemcsak esztétikai szempontból előnyös, hanem funkcionálisan is támogatja a gyors döntéshozatalt és a pontos irányítást.

A jobb oldalon egy elkülönített információs panel kapott helyet, amely a játékhoz tartozó statisztikai adatokat jeleníti meg. Itt látható az aktuális pontszám (Score), valamint a legmagasabb elért eredmény (High Score), amely motivációs elemként is szolgál a játékos számára. Az elkülönített panel segít

abban, hogy a játékos figyelme elsősorban a játéktéren maradjon, miközben folyamatos visszajelzést kap a teljesítményéről. Az elrendezés átlátható, logikus és jól strukturált, amely hozzájárul a játék élvezhetőségéhez és hosszabb távon is fenntartja az érdeklődést.

Összességében a Tetris játék felülete a klasszikus játékmenet modern újraértelmezése: egyszerű, fókuszált és felhasználóbarát, miközben technikailag is jól elkülöníti a játékmenetet és az információs elemeket. Ez a megoldás ideális alapot biztosít a további bővítésekhez, például következő tetrominó előnézet, nehézségi szintek vagy statisztikák megjelenítéséhez.

1.9.2 Game over menü



28. ábra - Tetris Game Over

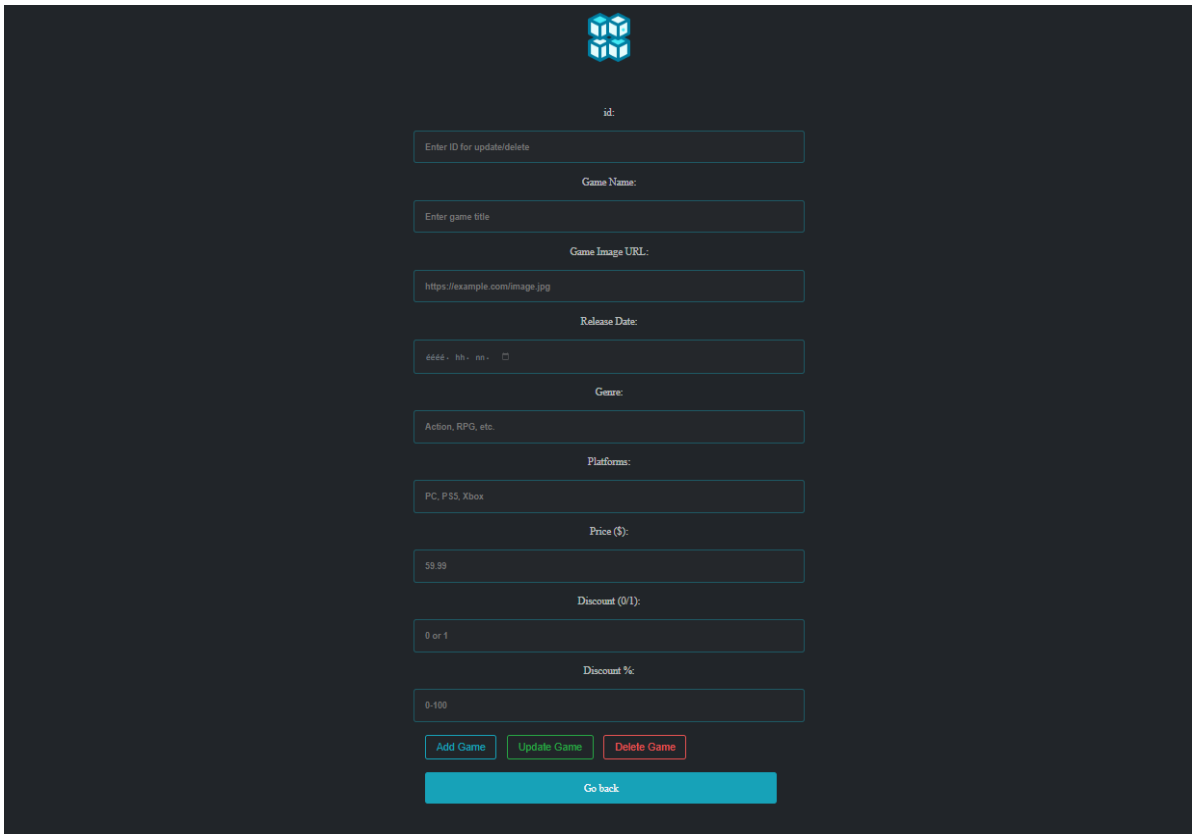
A Tetris játék Game Over képernyője egyértelmű és vizuálisan hangsúlyos visszajelzést ad a játék befejezéséről. A játéktér közepén megjelenő, sötét hátterű felugró ablakban a **„GAME OVER”** felirat piros színnel jelenik meg,

amely azonnal felhívja a játékos figyelmét a játék végére. A kontrasztos színválasztás és a központi elhelyezés biztosítja, hogy a visszajelzés ne maradjon észrevétlen, még intenzív játékmenet után sem.

A Game Over ablak alatt egy jól látható **„Try Again”** gomb található, amely lehetőséget biztosít a játék azonnali újraindítására. Ez a megoldás fenntartja a játék folytonosságát, és ösztönzi a játékost arra, hogy újra próbálkozzon, javítva korábbi teljesítményén. A háttérben halványan továbbra is látható a játéktér és az egymásra torlódott tetrominók, ami vizuálisan is indokolja a játék végét, és segít megérteni a Game Over állapot kialakulását.

A képernyő jobb oldalán elhelyezett információs panel továbbra is megjeleníti az aktuális pontszámot (**Score**) és a legjobb eredményt (**High Score**), ezzel lehetőséget adva a játékosnak a teljesítményének azonnali értékelésére. Ez a megoldás motivációs szerepet is betölt, hiszen egyértelműen látható, hogy sikerült-e új rekordot elérni. Összességében a Game Over menü letisztult, funkcionális és jól illeszkedik a játék vizuális stílusához, miközben hatékony átmenetet biztosít a játék vége és az újrakezdés között.

1.10. Az admin felület



29. ábra - Admin Panel

Az admin felület a webáruház háttérkezelő rendszere, amely lehetővé teszi a játékok adatainak létrehozását, módosítását és törlését. A felület letisztult, sötét témájú dizájnt kapott, amely vizuálisan illeszkedik az oldal gaming stílusához, ugyanakkor jól elkülönül a publikus felhasználói nézettől.

Az admin űrlap mezői a következő adatokat tartalmazzák:

- **ID** – Az adott játék egyedi azonosítója (update vagy delete művelethez szükséges).
- **Game Name** – A játék címe.
- **Game Image URL** – A játék borítóképének elérési útvonala.
- **Release Date** – A megjelenési dátum.
- **Genre** – A játék műfaja (pl. Action, RPG, FPS).
- **Platforms** – Az elérhető platformok felsorolása (pl. PC, PS5, Xbox).
- **Price ($)** – A játék alapára.
- **Discount (0/1)** – Meghatározza, hogy a játék akciós-e (0 = nem, 1 = igen).
- **Discount %** – Az akció mértéke százalékban (0–100).

Az űrlap alján három különálló műveleti gomb található:

- **Add Game** – Új játék hozzáadása az adatbázishoz.
- **Update Game** – Meglévő játék adatainak módosítása ID alapján.
- **Delete Game** – Játék törlése az adatbázisból.

- **Go back** – Viszadob az admin felületről a webshopba

A rendszer közvetlen kapcsolatban áll az adatbázissal, így a módosítások azonnal érvényesülnek a felhasználói oldalon is. Ez az adminisztrációs felület biztosítja a dinamikus tartalomkezelést, lehetővé téve az árak, akciók és készletinformációk gyors frissítését anélkül, hogy a forráskódot manuálisan kellene módosítani

2. Fejlesztői dokumentáció – első szintű címsor

2.1. Felhasznált technológiák

2.1.1. PHP

A PHP (Hypertext Preprocessor) egy szerveroldali szkriptnyelv, amelyet elsősorban dinamikus weboldalak és webalkalmazások fejlesztésére használnak. A PHP lehetővé teszi a szerver és a kliens közötti kommunikáció kezelését, valamint az adatbázisokkal való hatékony kapcsolattartást. A nyelv segítségével a fejlesztők dinamikusan generált tartalmakat hozhatnak létre, feldolgozhatják a felhasználói adatokat, valamint megvalósíthatják az üzleti logikát. A Fragstore projekt esetében a PHP kulcsszerepet játszott a backend működésében, például a bejelentkezési és regisztrációs folyamatok, a kosár és a wishlist funkciók, valamint az adatbázis-kezelés megvalósításában.

2.1.2. HTML

A HTML (Hypertext Markup Language) a weboldalak szerkezetének kialakítására szolgáló jelölőnyelv. Segítségével határozható meg egy weboldal tartalma és annak hierarchikus felépítése, például szövegek, képek, űrlapok és egyéb elemek elhelyezése. A HTML a webfejlesztés alapját képezi, mivel minden weboldal erre épül. A Fragstore fejlesztése során a HTML biztosította az oldalak logikus és átlátható struktúráját, amelyre a további technológiák – például a CSS és a JavaScript – hatékonyan tudtak épülni.

2.1.3. CSS

A CSS (Cascading Style Sheets) egy stíluslapnyelv, amely a weboldalak megjelenésének és vizuális stílusának meghatározására szolgál. A CSS lehetővé teszi a tartalom és a megjelenés elkülönítését, így a HTML kizárólag a szerkezetért, míg a CSS a formázásért felel. A CSS segítségével

szabályozható az oldalak színvilága, betűtípusai, elrendezése és reszponzív viselkedése. A Fragstore esetében a CSS nagyban hozzájárult az esztétikus megjelenéshez és a felhasználói élmény javításához, különösen a modern, letisztult dizájn kialakításában.

2.1.4. JavaScript

A JavaScript egy széles körben elterjedt programozási nyelv, amelyet elsősorban weboldalak és webalkalmazások interaktív funkcióinak megvalósítására használnak. A JavaScript jellemzően kliensoldalon fut, vagyis a felhasználó böngészőjében kerül feldolgozásra. Segítségével dinamikus tartalmak, eseménykezelések és felhasználói interakciók hozhatók létre. A Fragstore projektben a JavaScript biztosította az oldal élő működését, például a kosár frissítését, a wishlist kezelését, valamint az interaktív felhasználói elemek megvalósítását.

2.1.5. TypeScript

A TypeScript a JavaScript egy kibővített változata, amely statikus típusosságot vezet be a nyelvbe. Ez lehetővé teszi a fejlesztők számára, hogy már a fejlesztési folyamat során felismerjék az esetleges hibákat, ezáltal növelve a kód megbízhatóságát és karbantarthatóságát. A TypeScript különösen hasznos nagyobb, összetettebb projektek esetén, ahol a kód átláthatósága kiemelt szempont. A Fragstore fejlesztése során a TypeScript hozzájárult a frontend kód minőségének javításához és a hosszú távon fenntartható kódbázis kialakításához.

2.1.6. JSON

A JSON (JavaScript Object Notation) egy könnyen olvasható és strukturált adatformátum, amelyet leggyakrabban adatcserére használnak különböző rendszerek között. A JSON formátum egyszerűsége és rugalmassága miatt ideális a frontend és a backend közötti kommunikációhoz. A Fragstore alkalmazásban a JSON biztosította az adatok hatékony továbbítását, például a termékadatok, felhasználói információk és kosár tartalmának kezelésénél.

2.1.7. SQL

Az SQL (Structured Query Language) egy adatbázis-kezelő nyelv, amely relációs adatbázisok kezelésére szolgál. Segítségével létrehozhatók, módosíthatók és lekérdezhetők az adatbázisban tárolt adatok. A Fragstore projektben az SQL kulcsszerepet játszott az adatbázis struktúrájának kialakításában, valamint a felhasználók, termékek, rendelések és wishlist adatok biztonságos és hatékony kezelésében.

2.1.8. Bash

A Bash egy parancssori értelmező és szkriptnyelv, amelyet elsősorban fejlesztési és automatizálási feladatok során használnak. Lehetővé teszi különböző parancsok és folyamatok gyors végrehajtását, például buildelési, fájlkezelési vagy verziókezelési műveletek esetén. A Fragstore fejlesztése során a Bash hozzájárult a fejlesztési folyamatok egyszerűsítéséhez és hatékonyabbá tételéhez.

2.1.9. Twig

A Twig egy modern sablonkezelő (template engine), amelyet elsősorban PHP-alapú webalkalmazásokban használnak a megjelenítési réteg elkülönítésére az üzleti logikától. A Twig célja, hogy a HTML-oldalak dinamikus tartalommal való feltöltése átláthatóbbá, biztonságosabbá és könnyebben karbantarthatóvá váljon. A sablonrendszer lehetővé teszi változók, ciklusok és feltételek használatát a HTML-kódban anélkül, hogy a teljes PHP-logikát közvetlenül a megjelenítési rétegbe kellene ágyazni.

2.2. Keretrendszerek

2.2.1. React

A React egy JavaScript alapú felhasználói felület fejlesztésére szolgáló könyvtár, amelyet elsősorban dinamikus és komponensalapú webalkalmazások készítéséhez használnak. A React lehetővé teszi a felhasználói felület kisebb, újrahasznosítható komponensekre bontását, ami átláthatóbbá és könnyebben karbantarthatóvá teszi a kódot. A Fragstore projekt során a React alkalmazása

hozzájárult a gyorsabb felületfrissítésekhez és az interaktív elemek hatékony kezeléséhez, ezáltal javítva a felhasználói élményt.

2.2.2. Angular

Az Angular egy teljes körű, TypeScript alapú frontend keretrendszer, amelyet komplex webalkalmazások fejlesztésére használnak. Strukturált felépítése és beépített megoldásai – például adatkezelés és komponenskommunikáció – megkönnyítik a nagyobb alkalmazások fejlesztését. A Fragstore esetében az Angular használata segítette az alkalmazás logikus felépítését és a kód egységes kezelését, különösen az összetettebb felhasználói felületi funkciók esetén.

2.2.3. Alpine.js

Az Alpine.js egy könnyű JavaScript keretrendszer, amelyet egyszerűbb interaktív funkciók megvalósítására terveztek. Elsősorban kisebb dinamikus elemek kezelésére alkalmas, ahol egy teljes keretrendszer használata túlzott lenne. A Fragstore projektben az Alpine.js hozzájárult az oldal gyorsabb betöltéséhez és az egyszerűbb felhasználói interakciók hatékony megvalósításához.

2.2.4. p5.js

A p5.js egy JavaScript alapú grafikus könyvtár, amelyet kreatív kódoláshoz és vizuális elemek megvalósításához használnak. Különösen alkalmas animációk, interaktív grafikai megoldások és kisebb játékok fejlesztésére. A Fragstore alkalmazásban a p5.js az easter egg játékok és vizuálisan különleges elemek megvalósításában kapott szerepet, növelve az oldal egyediségét és élményszerűségét.

2.2.5. Sketch

A Sketch egy felhasználói felületek tervezésére szolgáló grafikus eszköz, amelyet elsősorban webes és mobilalkalmazások dizájnjának megalkotására használnak. A Sketch segítségével pontosan megtervezhetők az oldalak elrendezései, komponensei és vizuális elemei. A Fragstore tervezési folyamatában a Sketch hozzájárult az egységes megjelenés kialakításához és a fejlesztési munka előkészítéséhez.

2.2.6. GSAP

A GSAP (GreenSock Animation Platform) egy nagy teljesítményű JavaScript animációs könyvtár, amelyet komplex és látványos animációk létrehozására használnak. A GSAP pontos időzítést és zökkenőmentes animációkat biztosít, amelyek jelentősen javítják a felhasználói élményt. A Fragstore projekt során a GSAP segítségével valósultak meg az oldal vizuális átmenetei és animált elemei.

2.2.7. Bootstrap

A Bootstrap egy széles körben elterjedt CSS keretrendszer, amely előre definiált komponenseket és rácsrendszert biztosít a reszponzív weboldalak gyors elkészítéséhez. A Bootstrap használata megkönnyíti a különböző képernyőméretekhez való alkalmazkodást. A Fragstore fejlesztése során a Bootstrap hozzájárult a gyorsabb frontend fejlesztéshez és az oldal reszponzív működéséhez.

2.2.8. Tailwind CSS

A Tailwind CSS egy modern, utility-first szemléletű CSS keretrendszer, amely lehetővé teszi az egyedi dizájnok gyors és rugalmas kialakítását. A Tailwind segítségével a stílusok közvetlenül a HTML elemekhez rendelhetők, ami nagyfokú testreszabhatóságot biztosít. A Fragstore projektben a Tailwind CSS alkalmazása elősegítette a letisztult, modern megjelenés kialakítását.

2.2.9. Bulma

A Bulma egy modern, nyílt forráskódú CSS keretrendszer, amely a reszponzív weboldalak gyors és hatékony kialakítását segíti elő. A keretrendszer alapja a Flexbox elrendezési modell, amely lehetővé teszi a rugalmas, jól skálázható és különböző képernyőméretekhez alkalmazkodó felületek létrehozását. A Bulma használata nem igényel JavaScriptet, így kizárólag a megjelenésért és az elrendezésért felelős.

2.2.10. Butterfly

A Butterfly egy saját fejlesztésű Bulma és Tailwind szerű frontend keretrendszer. A célja, hogy a projekt admin oldalán ezt felhasználva

egyszerűsített classokkal tudjunk dizájnolni és ezzel kevesebb CSS-t és javascriptet kell használni egy hasonló vagy jobb dizájn megoldáshoz, mint nélküle. Manipulátorok és Makrók (cikklusok, elágazások és hasonlókat) tartalmaz.

2.3. API-k

2.3.1. Chart.js

A Chart.js egy JavaScript könyvtár, amely egyszerű és testreszabható grafikonok készítését teszi lehetővé weboldalakon és webalkalmazásokban. Segítségével vizuálisan átlátható módon lehet megjeleníteni adatokat, például statisztikákat vagy elemzéseket. A Fragstore projektben a Chart.js főként adminisztrációs nézetekben és statisztikai elemek megjelenítésében játszott szerepet, lehetővé téve a termékek és felhasználói aktivitás vizuális értékelését.

2.3.2. Flags API

A Flags API egy külső szolgáltatás, amely országok zászlóinak megjelenítésére alkalmas. Segítségével egyszerűen lehet vizuális információt nyújtani a felhasználóknak, például nemzetközi termékekhez vagy felhasználói beállításokhoz kapcsolódóan. A Fragstore alkalmazásban a Flags API hozzájárult a felhasználói élmény gazdagításához és a vizuális tájékozódás megkönnyítéséhez.

2.3.3. Currency Exchange API

A Currency Exchange API lehetővé teszi az árak különböző pénznemek közötti automatikus átváltását. Ez különösen hasznos nemzetközi felhasználók vagy több pénznemű árlisták esetén, mivel valós idejű és pontos átváltást biztosít. A Fragstore projektben a Currency Exchange API hozzájárult a nemzetközi felhasználók kiszolgálásához, valamint a termékek árainak rugalmas és naprakész megjelenítéséhez.

2.3.4. EmailJS

Az EmailJS egy külső szolgáltatás, amely lehetővé teszi e-mailek küldését közvetlenül a frontendből, anélkül, hogy a fejlesztőnek külön szerveroldali e-mail kezelést kellene beállítania. A Fragstore projektben az EmailJS

segítségével valósult meg a kapcsolatfelvételi és értesítési funkciók, például a felhasználói kérdések vagy visszajelzések egyszerű kezelése, anélkül hogy külön backend e-mail szervert kellett volna konfigurálni.

2.3.5. RAWG API

A RAWG API egy nyilvános videojáték-adatbázisra épülő programozói felület, amely részletes információkat biztosít különböző videojátékokról. Az API segítségével lekérdezhetők többek között játéknevek, megjelenési dátumok, platformok, műfajok és egyéb, a játékokhoz kapcsolódó adatok. A RAWG API célja, hogy strukturált és könnyen feldolgozható formában biztosítson naprakész adatokat a videojátékok világából.

2.3.6. Google Translate API

A Google Translate API egy felhőalapú fordítási szolgáltatás, amely lehetővé teszi szövegek automatikus lefordítását különböző nyelvek között programozott módon. Az API célja, hogy a webalkalmazások egyszerűen és gyorsan tudjanak többnyelvű tartalmat biztosítani, anélkül hogy minden fordítást manuálisan kellene elkészíteni és karbantartani. A szolgáltatás képes rövid feliratok, menüpontok, üzenetek vagy akár hosszabb szövegek fordítására is, így rugalmasan alkalmazható különböző felhasználói felületeken.

2.4. Cloud service

2.4.1. Firebase

A Firebase egy Google által fejlesztett felhőalapú platform, amely különböző backend szolgáltatásokat biztosít webes és mobilalkalmazások számára. A Firebase célja, hogy leegyszerűsítse az alkalmazások fejlesztését és üzemeltetését azáltal, hogy kész megoldásokat kínál többek között adatkezelésre, hitelesítésre és valós idejű kommunikációra. A szolgáltatás felhőalapú működése lehetővé teszi az adatok biztonságos tárolását és elérését anélkül, hogy a fejlesztőknek saját szerverinfrastruktúrát kellene fenntartaniuk.

2.5. Fejlesztési eszközök

2.5.1. Visual Studio Code

A Visual Studio Code egy könnyű, mégis nagy teljesítményű forráskódszerkesztő, amelyet széles körben használnak webes és szoftverfejlesztési projektek során. A program számos beépített és bővíthető funkciót kínál, mint például szintaxiskiemelés, automatikus kódkiegészítés, hibajelzések és integrált terminál. A Visual Studio Code lehetővé teszi a különböző programozási nyelvek – például PHP, JavaScript, TypeScript, HTML és CSS – hatékony kezelését egyetlen fejlesztői környezeten belül. A Fragstore projekt fejlesztése során ez az eszköz biztosította az egységes munkakörnyezetet, amely jelentősen hozzájárult a hatékonyabb kódoláshoz és a hibák gyorsabb felismeréséhez.

2.5.2. Figma

A Figma egy modern, felhőalapú tervezőeszköz, amelyet elsősorban felhasználói felületek és weboldalak vizuális megtervezésére használjuk. A Figma lehetővé teszi a dizájnok valós idejű és közös szerkesztését, így különösen alkalmas csapatmunkára. Az eszköz segítségével már a fejlesztés korai szakaszában elkészíthetők a drótvázlatok (wireframe-ek), prototípusok és végleges látványtervek. A Fragstore esetében a Figma hozzájárult ahhoz, hogy az oldal egységes megjelenéssel, átgondolt, struktúrával és felhasználóbarát dizájnnal valósuljon meg.

2.5.3. XAMPP

A XAMPP egy ingyenes, nyílt forráskódú szervercsomag, amely helyi fejlesztési környezetet biztosít webalkalmazások készítéséhez és teszteléséhez. A csomag tartalmazza az Apache webszervert, a MySQL/MariaDB adatbáziskezelőt, valamint a PHP futtatókörnyezetet. A XAMPP használata lehetővé teszi, hogy a fejlesztők internetkapcsolat nélkül, lokális környezetben teszteljék az alkalmazás backend működését. A Fragstore projektben a XAMPP kulcsfontosságú szerepet játszott a PHP-alapú backend és az adatbázis funkciók fejlesztésében és ellenőrzésében.

2.5.4. GitHub

A GitHub egy verziókezelő és együttműködési platform, amely a Git rendszerre épül. Lehetővé teszi a forráskód változásainak nyomon követését, a fejlesztési folyamat dokumentálását, valamint a csapattagok közötti hatékony együttműködést. A GitHub használatával minden módosítás visszakövethetővé válik, így könnyen kezelhetők a hibák és a fejlesztési irányok. A Fragstore projekt teljes fejlesztési folyamata GitHub segítségével zajlott, ahol a publikus repository biztosítja az átláthatóságot és a projekt hosszú távú fenntarthatóságát.

2.5.5. Vite

A Vite egy modern frontend build eszköz és fejlesztői szerver, amely a gyors betöltési időre és hatékony fejlesztési élményre helyezi a hangsúlyt. A hagyományos build eszközökkel szemben a Vite azonnali fejlesztői környezetet biztosít, így a változtatások szinte azonnal megjelennek a böngészőben. A Vite különösen jól együttműködik modern JavaScript és TypeScript alapú projektekkel, valamint keretrendszerekkel, mint például a React. A Fragstore fejlesztése során a Vite jelentősen felgyorsította a frontend fejlesztési folyamatot és hozzájárult a hatékonyabb munkavégzéshez.

2.5.6. Discord

A Discord egy valós idejű kommunikációs platform, amelyet széles körben használnak fejlesztői csapatok együttműködésre és projektkommunikációra. A szolgáltatás lehetőséget biztosít szöveges és hangalapú kommunikációra, valamint fájlok és kódrészletek megosztására, ami különösen hasznos a közös fejlesztési munka során. A Discord felülete lehetővé teszi különböző csatornák létrehozását, amelyek segítik a feladatok, hibák és ötletek elkülönített kezelését.

A Fragstore projekt fejlesztése során a Discord szolgált a csapaton belüli kommunikáció elsődleges eszközeként. Segítségével a csapattagok folyamatos kapcsolatban tudtak maradni, gyorsan egyeztetni a fejlesztési irányokról, valamint hatékonyan megoldani a felmerülő problémákat. A Discord használata jelentősen hozzájárult a gördülékeny csapatmunkához és a fejlesztési folyamat szervezett lebonyolításához.

2.6. Adatbázis

2.6.1. Adatbázis tervezése

2.6.1.1.Adatbázis célja

Az adatbázis célja egy játék webshop, azaz a Fragstore játékadatainak és az ezekhez kapcsolódó kiegészítő adatok tárolása, valamint egy működő kosárfunkció és felhasználói felület megvalósítása. A rendszer kialakításához egy hibrid adatkezelési megoldást terveztünk alkalmazni, amely különböző adattárolási technológiákat használ a feladat jellegétől függően. A játékok alapadatai és a díjazó oldal feldolgozása hagyományos adatbázisban történik, míg a hibrid megoldás másik része JSON formátum alkalmazásával valósul meg, amelyre a legjobb példa a kosár funkció, mivel annak adatait Firebase adatbázisban tároljuk. Ennek több oka is van, például az adatbázis globalizálhatósága saját szerver üzemeltetése nélkül, az egyszerűbb adatkezelés a JSON struktúra miatt, valamint az, hogy a Firebase adattárolási megoldása átláthatóbb és rugalmasabb, mint egy hagyományos SQL-alapú rendszer. Ezeket figyelembe véve számunkra a hibrid adatkezelési megoldás vonzóbb és hatékonyabb választásnak bizonyult, mint egy kizárólag hagyományos adatbázisra épülő megközelítés.Adatbázisok.

2.6.1.2. Tervezési lépések

A tervezési folyamat során az egyik legfontosabb döntés a megfelelő adattárolási megoldás kiválasztása volt. Kezdetben egy hagyományos, relációs adatbázisban gondolkodtunk, azonban a projekt előrehaladtával egyértelművé vált, hogy bizonyos funkciók – például a kosár kezelése vagy a valós idejű adatmódosítások – rugalmasabban és hatékonyabban kezelhetők egy NoSQL alapú rendszerben. Ennek hatására egy hibrid megoldás mellett döntöttünk, amely ötvözi az SQL adatbázis stabilitását és strukturáltságát a Firebase által nyújtott gyors és globálisan elérhető adattárolással.

A relációs adatbázist elsősorban a strukturált, jól definiálható adatok tárolására használtuk, mint például a játékok, fejlesztők, kiadók, műfajok és platformok. Ezeknél az adatoknál fontos szempont volt a normalizálás, az adatok közötti

kapcsolatok pontos definiálása, valamint az adatkonzisztencia fenntartása. Ezzel szemben a Firebase alapú megoldást olyan dinamikusabb adatok kezelésére alkalmaztuk, mint például a felhasználói kosár, ahol fontos a gyors elérés, az egyszerű adatstruktúra és a valós idejű frissíthetőség.

A két adatbázis együttes használata lehetővé tette, hogy a rendszer mindkét megközelítés előnyeit kihasználja. Az SQL biztosítja a megbízható és jól strukturált adattárolást, míg a Firebase rugalmasságot és skálázhatóságot nyújt a felhasználói interakciók kezelésében. Ez a megoldás összességében hatékonyabbá és jobban bővíthetővé tette a rendszert, mintha kizárólag egyetlen adatbázis típust alkalmaztunk volna. Ezért választottuk a második adatbázis típusnak a Firebase-t hiszen rengeteg nagy cég használta ezt a technológiát. Erre itt néhány példa.

- Hotstar: A Disney+ Hotstar a Firebase segítségével skálázza szolgáltatását, és 38%-kal növelte a felhasználói elköteleződést.

- Google alkalmazások: Számos Google-alkalmazás, például a Google Pay, a Google Ads és a Google Cloud, használja a Firebase-t.

- Duolingo (saját oktatási alkalmazás): A Duolingo maga is használhat Firebase szolgáltatásokat (például analitika, értesítések) a több millió aktív felhasználó valós idejű kezelésére.

2.6.1.3. Egyedek meghatározása (terv)

2.6.1.3.1. Videogames

datas

A **datas** tábla a játékok alapvető adatait tárolja, amelyek a webshopban való megjelenítéshez és értékesítéshez szükségesek. Ide tartoznak az olyan információk, mint a játék neve, ára, megjelenési dátuma, valamint az elérhetőségi és akciós adatok.

- name – A játék neve
- game_pic – A játék borítóképének elérési útja

- release_date – A játék megjelenési dátuma
- prize – A játék alap ára
- discountPerc – Az aktuális kedvezmény mértéke százalékban
- available – A játék elérhetőségi állapota

awards

- award_name – A díj neve
- award_year – A díj elnyerésének éve

developers

- person_name – A fejlesztő neve

publishers

- company_name – A kiadó neve

roles

- role_name – A fejlesztő szerepköre (pl. designer, programmer)

genres

- genre_name – A műfaj neve (pl. FPS, RPG)

platforms

- platform_name – A platform neve (pl. PC, PlayStation, Xbox)

Kapcsolatok:

videogames.awards – videogames.publishers (N:M)

Indoklás:

Egy kiadó több díjat is elnyerhet különböző játékai vagy üzleti sikerei miatt, és egy díj (például „Best Publisher”) több kiadóhoz is tartozhat különböző években.

videogames.awards – videogames.developers (N:M)

Indoklás:

Egy fejlesztő pályafutása során több díjat is kaphat, illetve egy díj több fejlesztőhöz is tartozhat, mivel egy játék elkészítésében több fejlesztő is részt vehet.

videogames.publishers – videogames.developers (N:M)

Indoklás:

Egy kiadónál egyszerre több fejlesztő is dolgozhat, és egy fejlesztő a pályafutása során több kiadónál is megfordulhat különböző időszakokban.

videogames.developers – videogames.roles (N:M)

Indoklás:

Egy fejlesztő több szerepkört is betölthet (pl. programmer és designer),illetve egy szerepkör több fejlesztőhöz is tartozhat.

videogames.datas – videogames.platforms (N:M)

Indoklás:

Egy játék több platformon is megjelenhet, és egy platformon több játék is elérhető.

videogames.datas – videogames.genres (N:1)

A kapcsolat **N : 1 típusú**, mivel több játék is tartozhat ugyanahhoz a műfajhoz, azonban egy adott játék egyszerre csak egy elsődleges műfajhoz van rendelve. A kapcsolat megvalósítása idegen kulcs segítségével történik, ahol a **datas** tábla tartalmaz egy olyan mezőt, amely a **genres** tábla elsődleges kulcsára hivatkozik.

videogames.datas – videogames.awards(N:1)

A kapcsolat **1 : N típusú**, mivel egy játék több különböző díjat is elnyerhet, azonban egy adott díj rekord egy konkrét játékhoz tartozik. A kapcsolat idegen kulcs segítségével valósul meg, ahol az **awards** tábla tartalmazza annak a játéknak az azonosítóját, amelyhez a díj tartozik.

2.6.1.3.2. Users

A **Users** tábla a felhasználók alapvető adatait tárolja, amelyek a bejelentkezéshez és azonosításhoz szükségesek. Ezek az adatok biztosítják a felhasználói fiókok kezelését és a rendszerhez való hozzáférést.

- email – A felhasználó email címe
- username – A felhasználó neve
- password_hash – A felhasználó titkosított jelszava
- created_at – A fiók létrehozásának időpontja

2.6.1.3.3. Giftcard

A **Giftcard** tábla a különböző ajándékkártyák adatait tartalmazza, amelyek a webshopban megvásárolhatók. Ide tartoznak az árra, megjelenésre és régióra vonatkozó információk.

- name – A giftcard neve
- img – A giftcard képének elérési útja
- price – A giftcard ára
- region – A giftcardhoz tartozó régió

2.6.1.3.4. Leaderboard

A **Leaderboard** tábla a játékosok pontszámait tárolja, amelyeket a különböző játékok során értek el. Ez lehetővé teszi ranglisták létrehozását és a felhasználók teljesítményének összehasonlítását.

- user_id – A felhasználó azonosítója
- score – A játékos által elért pontszám

2.6.1.4.Az adatbázis fejlesztés alatt feljött problémák és azok megoldása

- **többértékű mezők meghatározása:** A többértékű mezők felbontása lényeges, hiszen ezzel gyorsabb és hatékonyabb lekérdezéseket lehet írni az adatbázis rekordjai között. Sajnos a tervezés korai fázisában nagyon siettünk, és nem vettünk sok ilyet figyelembe. Erre jó példa a játékokat és giftcardokat

tároló datas tábla, ahol a platform mező varchar típusként volt megadva. Ez azonban többértékű mező, ráadásul N:M kapcsolatot is jelentett, ezért két új táblára volt szükség a megfelelő kezeléséhez. Illetve ilyen volt még a developer role-ja is, amely szintén több értéket vehet fel, ezért ahhoz is szükség volt egy külön táblára.

- **Nem megfelelő kapcsolat típusok:** A kapcsolat típusok megfelelő meghatározása kötelező egy jól működő adatbázis elkészítéséhez. Ezzel függőségeket és anomáliákat lehet elkerülni. Sajnos ebből is volt több probléma. Erre jó példa a datas és az award N:M kapcsolata, amelyet kezdetben nem tudtunk megfelelően megvalósítani. Emellett az award**,** developer és publisher kapcsolatok logikailag is rosszul voltak meghatározva, amelyeket később átdolgoztunk.

- **Nem létező táblák későbbi létrehozása:** Egy adatbázis működhet kevesebb táblával is, azonban így nem lesz hatékony, és nem biztos, hogy megfelel a 3NF követelményeinek. Ezeket a táblákat legfőképpen az N:M kapcsolatok kialakításánál, illetve a többértékű mezők felbontására használtuk. Erre jó példa a datas és award tábla közötti kapcsolótábla, vagy a developerhez tartozó role tábla, amely segítségével egy többértékű mezőt tudtunk felbontani. Ezek és az ehhez hasonló hibák javításával az adatbázis már megfelelt a 3NF-nek.

- **Idegen kulcsok fordított bekötése:** Az adatbázisban fontos a kapcsolatok megfelelő bekötése, mivel ezek nélkül a táblák közötti kapcsolatok nem működnek, és egy nem megfelelően működő adatbázis jön létre. A kezdeti átgondolatlanság miatt több ilyen hiba is volt. Például egyes N:M kapcsolatoknál fordítva kötöttük be az idegen kulcsokat, amelyeket utólag javítani kellett, de sajnos ez 1:N kapcsolatoknál is előfordult.

- **Duplikáns adatok (tranzitív függések kezelése):** A duplikált adatok kezelése fontos egy jól működő adatbázisban. Bár nem mindig kritikus hiba, nagyobb adatbázisoknál a nem megfelelő optimalizáltság miatt lassulhat a rendszer, és a szerver is könnyebben leterhelhetővé válhat. A kezdeti állapotban nálunk is voltak ilyen hibák, azonban ezeket később javítottuk. Erre jó példa a kezdési és befejezési dátum mező, amely a developer és publisher

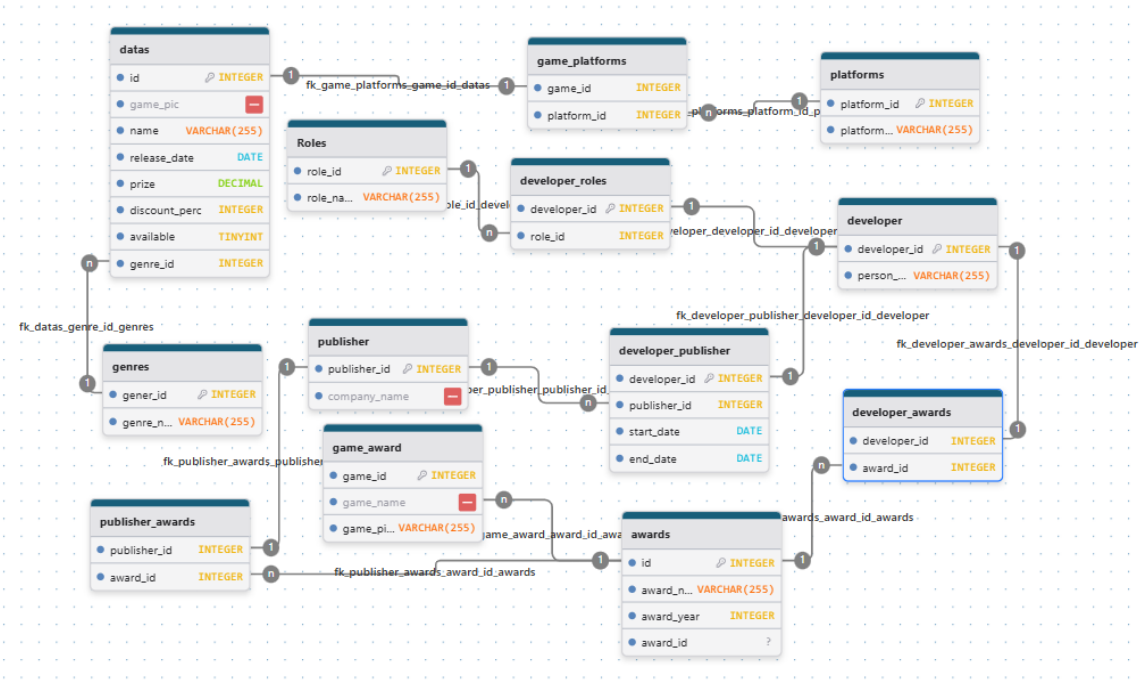
táblákban szerepelt. Ezt végül a kapcsoló táblában oldottuk meg, így kevesebb mező maradt az eredeti táblákban.

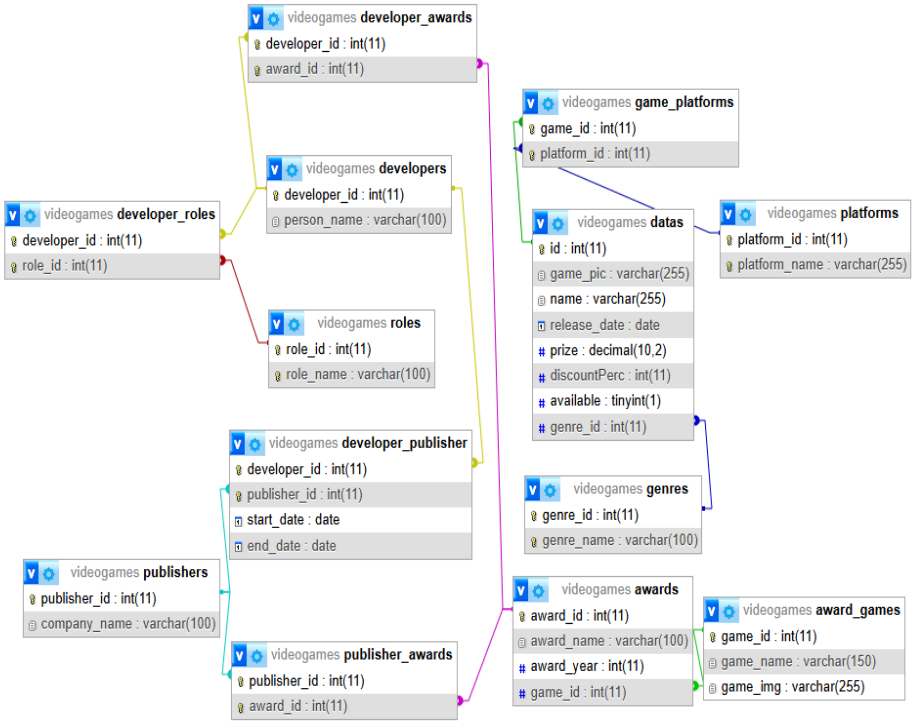
- **3NF nem létezése:** Ez lényegében az előbb felsorolt hibák összessége. A 3NF (harmadik normál forma) célja, hogy az adatbázisban ne legyenek felesleges ismétlődések és logikai hibák, valamint hogy könnyebb legyen az adatok frissítése. Egy adatbázis akkor felel meg a 3NF-nek, ha nincs benne tranzitív függés, és már megfelel a 2NF követelményeinek is.

- **Logikai hibák:** Egy adatbázisban a logikai hibák javítása fontos, mert így hatékonyabb lesz a működés, és kevesebb adatot kell feldolgozni a lekérdezések során. Az adatbázisunkban erre egyetlen példa volt a datas táblában, amelyet viszont utólag könnyű volt javítani. Ez az isDiscount mező volt, amely egy későbbi átnézés során fölöslegesnek bizonyult, és rontotta az adatbázis logikai működését.

2.6.2. Adatbázis megvalósítása

2.6.2.1.videogames





30. ábra - Database Plan

2.6.2.1.1. datas

Mező neve	Típus	Leírás
id	INT (PK)	Játék egyedi azonosító
name	VARCHAR(255)	Játék neve
game_pic	VARCHAR(255)	Játék borítókép
release_date	DATE	Megjelenés dátuma
prize	DECIMAL(10,2)	Ár
discountPerc	INT	Kedvezmény százalék
available	TINYINT	Elérhetőség
genre_id	INT (FK)	Műfaj

2.6.2.1.2. award_games

Mező neve	Típus	Leírás
developer_id	INT (PK)	Egyedi azonosító minden fejlesztőhöz
role_id	int(11)	Egyedi azonosító minden szerepnek

2.6.2.1.3. awards

Mező neve	Típus	Leírás

award_id	INT (PK)	Díj azonosító
award_name	VARCHAR(100)	Díj neve
award_year	INT	Díj éve
game_id	INT (FK)	Díjazott játék

2.6.2.1.4. developers

Mező neve	Típus	Leírás
developer_id	INT (PK)	Egyedi azonosító minden fejlesztőhöz
person_name	VARCHAR(100)	A fejlesztő neve

2.6.2.1.5. publishers

Mező neve	Típus	Leírás
publisher_id	INT (PK)	Egyedi kiadó azonosító
company_name	VARCHAR(100)	Kiadó neve

2.6.2.1.6. publisher_awards (kapcsoló tábla)

Mező neve	Típus	Leírás
publisher_id	INT (FK)	Díjazott kiadó
award_id	INT (FK)	Díj

- Ezzel fel tudjuk bontani a kiadó és a díj közötti N:M-es kapcsolatot

2.6.2.1.7. developer_awards (kapcsoló tábla)

Mező neve	Típus	Leírás
developer_id	INT (FK)	Díjazott fejlesztő
award_id	INT (FK)	Díj

- Ezzel tudjuk fel bontani a fejlesztő és a díjak közötti N:M-es kapcsolatot

2.6.2.1.8. roles

Mező neve	Típus	Leírás
role_id	INT (PK)	Szerep azonosító
role_name	VARCHAR(100)	Szerep neve (pl. Programmer, Designer)

Kapcsolatok:

2.6.2.1.9. developer_roles

Mező neve	Típus	Leírás
developer_id	INT (FK)	Fejlesztő
role_id	INT (FK)	Szerep

Összetett kulcs:

- (developer_id, role_id)

2.6.2.1.10. developer_publisher

Mező neve	Típus	Leírás
developer_id	INT (FK)	Fejlesztő
publisher_id	INT (FK)	Kiadó
start_date	DATE	Együttműködés kezdete
end_date	DATE	Együttműködés vége

Lehetővé teszi időben változó kapcsolat kezelését

2.6.2.1.11. genres

Mező neve	Típus	Leírás
genre_id	INT (PK)	Műfaj azonosító
genre_name	VARCHAR(100)	Műfaj neve

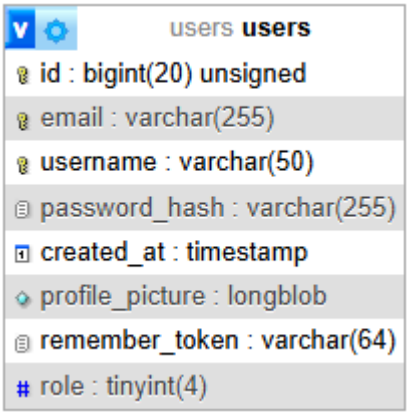
2.6.2.1.12. platforms

Mező neve	Típus	Leírás
platform_id	INT (PK)	Platform azonosító
platform_name	VARCHAR(255)	Platform neve

2.6.2.1.13. game_platforms

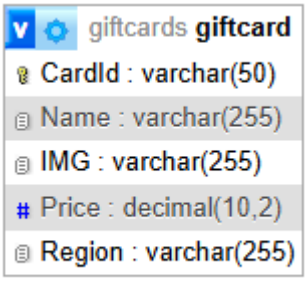
Mező neve	Típus	Leírás
game_id	INT (FK)	Játék
platform_id	INT (FK)	Platform

2.6.2.2.Users



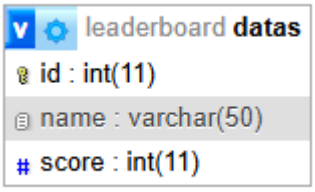
Mező neve	Típus	Kulcs	Leírás
id	bigint	PK	A felhasználó egyedi azonosítója
email	varchar		A felhasználó email címe
username	varchar		Felhasználónév
password_hash	varchar		A felhasználó jelszavának hash-e
created_at	timestamp		A felhasználói fiók létrehozásának ideje

2.6.2.3. Giftcard



Mező neve	Típus	Kulcs	Leírás
card_id	varchar	PK	Az ajándékkártya egyedi azonosítója
name	varchar		Az ajándékkártyán szereplő cég neve (pl. Netflix)
img	varchar		A cég logóját tartalmazó kép elérési útja
price	decimal		Az ajándékkártya értéke
region	varchar		Az ajándékkártya régiója (pl. EU, NA)

2.6.2.4. Leaderboard



Mező neve	Típus	Kulcs	Leírás
id	int	PK	A leaderboard rekord egyedi azonosítója
user_id	bigint	FK	A felhasználó azonosítója
score	int		A leaderboardon elért pontszám

2.6.3. Teszt adatok kóddal

2.6.3.1.Akciós Játékok lekérdezése

SQL Lekérdezés:

Ez a lekérdezés azokat a játékokat jeleníti meg, amelyek rendelkeznek kedvezménnyel. A discountPerc mező azt mutatja meg, hogy hány százalékos akció van a játékra. A lekérdezés csak azokat a rekordokat adja vissza, ahol a kedvezmény nagyobb mint 0.

```
SELECT name, prize, discountPerc
FROM datas
WHERE discountPerc > 0;
```

Eredmény:

←T→			name	prize	discountPerc
Módosítás	Másolás	Törlés	Call of Duty: Black Ops Cold War	59.99	42
Módosítás	Másolás	Törlés	Call of Duty: Advanced Warfare	59.99	60
Módosítás	Másolás	Törlés	Call of Duty: Black Ops III	59.99	55
Módosítás	Másolás	Törlés	Call of Duty: Infinite Warfare	59.99	70
Módosítás	Másolás	Törlés	Call of Duty: WWII	59.99	70
Módosítás	Másolás	Törlés	Call of Duty: Black Ops 4	59.99	60
Módosítás	Másolás	Törlés	Call of Duty: Modern Warfare	59.99	75
Módosítás	Másolás	Törlés	Call of Duty: Vanguard	59.99	50

2.6.3.2. Elérhető játékok listázása

SQL Lekérdezés:

Ez a lekérdezés az adatbázisból azokat a játékokat jeleníti meg, amelyek jelenleg elérhetőek. Az available mező logikai értéket tartalmaz, ahol az 1 jelenti az elérhető állapotot.

```
SELECT name, release_date

FROM datas

WHERE available = 1;
```

Eredmény:

				name	release_date
Módosítás	Másolás	Törlés		Elden Ring	2022-02-25
Módosítás	Másolás	Törlés		Grand Theft Auto V	2013-09-17
Módosítás	Másolás	Törlés		The Legend of Zelda: Breath of the Wild	2017-03-03
Módosítás	Másolás	Törlés		Cyberpunk 2077	2020-12-10
Módosítás	Másolás	Törlés		Minecraft	2011-11-18
Módosítás	Másolás	Törlés		Fortnite	2017-07-25
Módosítás	Másolás	Törlés		Call of Duty: Modern Warfare II	2022-10-28
Módosítás	Másolás	Törlés		Hades	2020-09-17
Módosítás	Másolás	Törlés		Animal Crossing: New Horizons	2020-03-20
Módosítás	Másolás	Törlés		Hollow Knight	2017-02-24
Módosítás	Másolás	Törlés		Red dead Redemption 2	2018-10-26
Módosítás	Másolás	Törlés		God of War	2018-04-20
Módosítás	Másolás	Törlés		Overwatch 2	2022-10-04
Módosítás	Másolás	Törlés		Valorant	2020-06-02
Módosítás	Másolás	Törlés		Tetris Effect	2018-11-09
Módosítás	Másolás	Törlés		Doom Eternal	2020-03-20
Módosítás	Másolás	Törlés		Stardew Valley	2016-02-26
Módosítás	Másolás	Törlés		League of Legends	2009-10-27
Módosítás	Másolás	Törlés		Apex Legends	2019-02-04
Módosítás	Másolás	Törlés		Resident Evil 4 Remake	2023-03-24
Módosítás	Másolás	Törlés		Horizon Forbidden West	2022-02-18
Módosítás	Másolás	Törlés		Sekiro: Shadows Die Twice	2019-03-22
Módosítás	Másolás	Törlés		Death Stranding	2019-11-08
Módosítás	Másolás	Törlés		Celeste	2018-01-25
Módosítás	Másolás	Törlés		Bloodborne	2015-03-24

2.6.3.3. Legdrágább játék meghatározása

SQL lekérdezés:

Ez a lekérdezés megkeresi az adatbázisban található legdrágább játékot. Az adatokat ár szerint csökkenő sorrendbe rendezi, majd az első rekordot adja vissza.

```
SELECT name, prize

FROM datas

ORDER BY prize DESC

LIMIT 1;
```

Eredmény:

			name	prize
☐	Módosítás	Másolás	Törlés	Dragon Quest IV: Chapters of the Chosen	90.00

2.6.3.4. Játékok adott platformon

SQL lekérdezés:

Ez a lekérdezés azokat a játékokat jeleníti meg, amelyek egy adott platformon (például PC-n) elérhetőek. A lekérdezés több táblát kapcsol össze (datas, game_platforms, platforms) a megfelelő adatok kinyeréséhez.

```
SELECT d.name, p.platform_name

FROM datas d

JOIN game_platforms gp ON d.id = gp.game_id

JOIN platforms p ON gp.platform_id = p.platform_id

WHERE p.platform_name = 'PC';
```

Eredmény:

name	platform_name
Elden Ring	PC
Grand Theft Auto V	PC
Cyberpunk 2077	PC
Minecraft	PC
Fortnite	PC
Call of Duty: Modern Warfare II	PC
Hades	PC
Hollow Knight	PC
Red dead Redemption 2	PC
God of War	PC
Overwatch 2	PC
Valorant	PC
Tetris Effect	PC
Doom Eternal	PC
Stardew Valley	PC
League of Legends	PC
Apex Legends	PC
Resident Evil 4 Remake	PC
Sekiro: Shadows Die Twice	PC
Death Stranding	PC
Celeste	PC
NieR: Automata	PC
It Takes Two	PC
A Plague Tale: Requiem	PC
Sifu	PC

2.6.3.5. Játékok műfaj szerint

SQL lekérdezés:

Ez a lekérdezés a játékok adatait jeleníti meg a hozzájuk tartozó műfajjal együtt. A datas és a genres táblák összekapcsolásával jeleníti meg az információkat.

```
SELECT d.name AS game_name, g.genre_name, d.release_date, d.prize

FROM datas d

JOIN genres g ON d.genre_id = g.genre_id;
```

Eredmény:

game_name	genre_name	release_date	prize
God of War	Action	2018-04-20	49.99
Death Stranding	Action	2019-11-08	49.99
Bayonetta 3	Action	2022-10-28	59.99
Sifu	Action	2022-02-08	39.99
Assassin's Creed Valhalla	Action	2020-11-10	59.99
Watch Dogs: Legion	Action	2020-10-29	59.99
Ghostrunner	Action	2020-10-27	29.99
Shadow of the Tomb Raider	Action	2018-09-14	39.99
Dead Rising 4	Action	2016-12-06	29.99
Mad Max	Action	2015-09-01	29.99
Devil May Cry 5	Action	2019-03-08	39.99
Just Cause 4	Action	2018-12-04	39.99
Uncharted 4: A Thief's End	Action	2016-05-10	39.99
Watch Dogs 2	Action	2016-11-15	39.99
Tomb Raider (2013)	Action	2013-03-05	29.99
Watch Dogs	Action	2014-05-27	39.99
Alan Wake	Action	2010-05-14	19.99
Grand Theft Auto IV	Action	2008-04-29	29.99
Marvel's Spider-Man	Action	2018-09-07	59.99
Yakuza 0	Action	2017-03-12	39.99
Bayonetta 2	Action	2014-10-24	39.99
Mafia III	Action	2016-10-07	29.99
L.A. Noire	Action	2011-05-17	29.99
Saints Row IV	Action	2013-08-20	29.99
Max Payne 3	Action	2012-05-15	19.99

2.6.3.6. 2020 után megjelent játékok

SQL lekérdezés:

Ez a lekérdezés azokat a játékokat listázza ki, amelyek 2020 után jelentek meg. A szűrés a release_date mező alapján történik.

```
SELECT name, release_date
FROM datas
WHERE release_date > '2020-01-01';
```

Eredmény:

←T→				name	release_date
☐	Módosítás	Másolás	Törlés	Elden Ring	2022-02-25
☐	Módosítás	Másolás	Törlés	Cyberpunk 2077	2020-12-10
☐	Módosítás	Másolás	Törlés	Call of Duty: Modern Warfare II	2022-10-28
☐	Módosítás	Másolás	Törlés	Hades	2020-09-17
☐	Módosítás	Másolás	Törlés	Animal Crossing: New Horizons	2020-03-20
☐	Módosítás	Másolás	Törlés	Overwatch 2	2022-10-04
☐	Módosítás	Másolás	Törlés	Valorant	2020-06-02
☐	Módosítás	Másolás	Törlés	Doom Eternal	2020-03-20
☐	Módosítás	Másolás	Törlés	Resident Evil 4 Remake	2023-03-24
☐	Módosítás	Másolás	Törlés	Horizon Forbidden West	2022-02-18
☐	Módosítás	Másolás	Törlés	It Takes Two	2021-03-26
☐	Módosítás	Másolás	Törlés	Splatoon 3	2022-09-09
☐	Módosítás	Másolás	Törlés	Bayonetta 3	2022-10-28
☐	Módosítás	Másolás	Törlés	A Plague Tale: Requiem	2022-10-18
☐	Módosítás	Másolás	Törlés	Sifu	2022-02-08
☐	Módosítás	Másolás	Törlés	Ghost of Tsushima	2020-07-17
☐	Módosítás	Másolás	Törlés	Tunic	2022-03-16
☐	Módosítás	Másolás	Törlés	Returnal	2021-04-30
☐	Módosítás	Másolás	Törlés	Dead Space Remake	2023-01-27
☐	Módosítás	Másolás	Törlés	Halo Infinite	2021-12-08
☐	Módosítás	Másolás	Törlés	Assassin's Creed Valhalla	2020-11-10
☐	Módosítás	Másolás	Törlés	Fall Guys	2020-08-04
☐	Módosítás	Másolás	Törlés	Minecraft Dungeons	2020-05-26
☐	Módosítás	Másolás	Törlés	Genshin Impact	2020-09-28
☐	Módosítás	Másolás	Törlés	Ori and the Will of the Wisps	2020-03-11

2.6.3.7. Játékok átlagos ára

SQL lekérdezés:

Ez a lekérdezés kiszámolja az adatbázisban szereplő játékok átlagos árát az AVG() aggregációs függvény segítségével.

```
SELECT AVG(prize) AS average_price

FROM datas;
```

Eredmény:

average_price
33.294729

2.6.4. Továbbfejlesztési lehetőség

- A publisher entitás összekapcsolható a videogames (datas) táblával idegen kulcs használatával. Ennek segítségével a játékokhoz tartozó fejlesztő vagy fejlesztők adatai strukturált módon tárolhatók az adatbázisban. Mivel egy játékhoz több fejlesztő vagy kiadó is tartozhat, illetve egy fejlesztő több különböző játék létrehozásában is részt vehet, ezért ez az adatkapcsolat N:M (több a többhöz) kapcsolatként modellezhető.

2.6.5. N:M-es kapcsolatok felbontása

A kapcsolat egy publisher_awards nevű kapcsolótábla bevezetésével került felbontásra, amely tartalmazza a publisher_id és award_id mezőket.
Ennek eredményeként két darab 1:N kapcsolat jön létre:

- publishers → publisher_awards
- awards → publisher_awards

videogames.awards – videogames.developers
Felbontás:
A kapcsolat a developer_awards kapcsolótábla segítségével lett kezelve, amely a developer_id és *award_id* mezőket tartalmazza.
Ezáltal a következő 1:N kapcsolatok jönnek létre:

- developers → developer_awards

- awards → developer_awards

videogames.publishers – videogames.developers
Felbontás:
A kapcsolat egy külön kapcsolótábla bevezetésével valósítható meg (pl. publisher_developers), amely a publisher_id és developer_id mezőket tartalmazza.
Ez két 1:N kapcsolatra bontható:

- publishers → publisher_developers
- developers → publisher_developers

videogames.developers – videogames.roles
Felbontás:
A többértékű szerepkörök kezelése érdekében a developer_roles kapcsolótábla került bevezetésre, amely a developer_id és role_id mezőket tartalmazza.
A felbontás eredményeként:

- developers → developer_roles
- roles → developer_roles

videogames.datas – videogames.platforms
Felbontás:
A játékok és platformok közötti kapcsolat a game_platforms kapcsolótábla segítségével lett megvalósítva, amely a game_id és platform_id mezőket tartalmazza.
Ennek megfelelően:

- datas → game_platforms
- platforms → game_platforms

2.6.6. A többikapcsolat

videogames.datas – videogames.genres

- A kapcsolat **N : 1 típusú**, mivel több játék is tartozhat ugyanahhoz a műfajhoz, azonban egy adott játék egyszerre csak egy elsődleges műfajhoz van rendelve. A kapcsolat megvalósítása idegen kulcs segítségével történik, ahol a **datas** tábla tartalmaz egy olyan mezőt, amely a **genres** tábla elsődleges kulcsára hivatkozik.

videogames.datas – videogames.awards

- A kapcsolat **1 : N típusú**, mivel egy játék több különböző díjat is elnyerhet, azonban egy adott díj rekord egy konkrét játékhoz tartozik. A kapcsolat idegen kulcs segítségével valósul meg, ahol az **awards** tábla tartalmazza annak a játéknak az azonosítóját, amelyhez a díj tartozik.

2.6.7. Firebase database

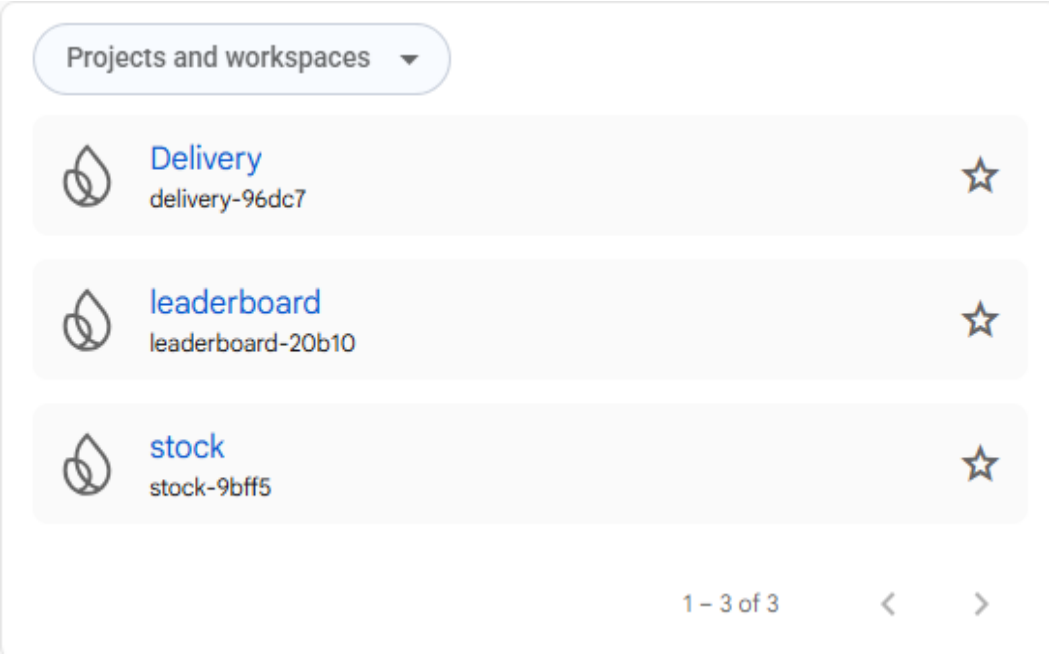
A projekt során a Firebase használata egy hybrid adatkezelési megoldás részeként valósult meg. A Firebase alkalmazása elsősorban a valós idejű adatkezelés és az egyszerű integráció miatt volt előnyös, mivel nem igényel külön szerveroldali infrastruktúrát, valamint JSON alapú adatstruktúrát használ, amely könnyen kezelhető JavaScript környezetben.

A Firebase-ben főként a kosárhoz tartozó adatok kerülnek tárolásra, mivel ezek dinamikusan változnak, és fontos a gyors, valós idejű frissítés. Ezzel szemben a statikusabb adatok, például a játékok adatai, SQL alapú adatbázisban kerülnek tárolásra.

A Firebase-ben tárolt adatok hierarchikus JSON struktúrában jelennek meg, amely lehetővé teszi az egyszerű és átlátható adatkezelést. Például egy felhasználó kosara tartalmazhatja a kiválasztott játékokat, azok mennyiségét és az összesített árat.

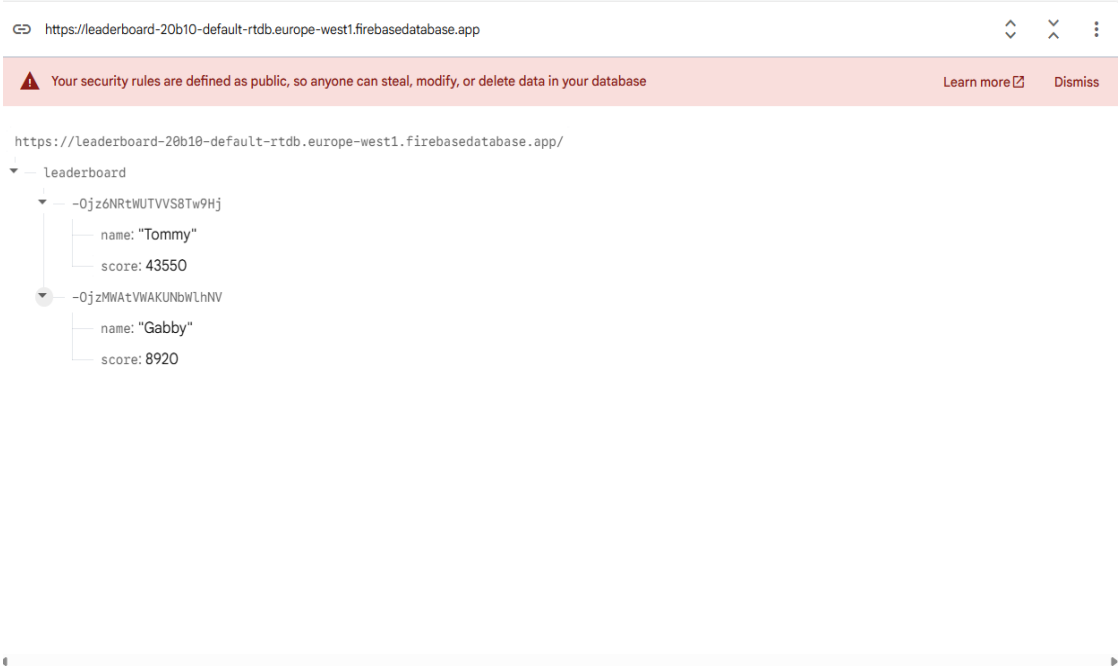
A Firebase és az SQL adatbázis együttes használata lehetővé teszi, hogy a rendszer mind a strukturált, mind a dinamikusan változó adatok kezelésében hatékony legyen.

3 különböző globális adatbázishoz használjuk a képen láthatóak.



31. ábra - Firebase Databases

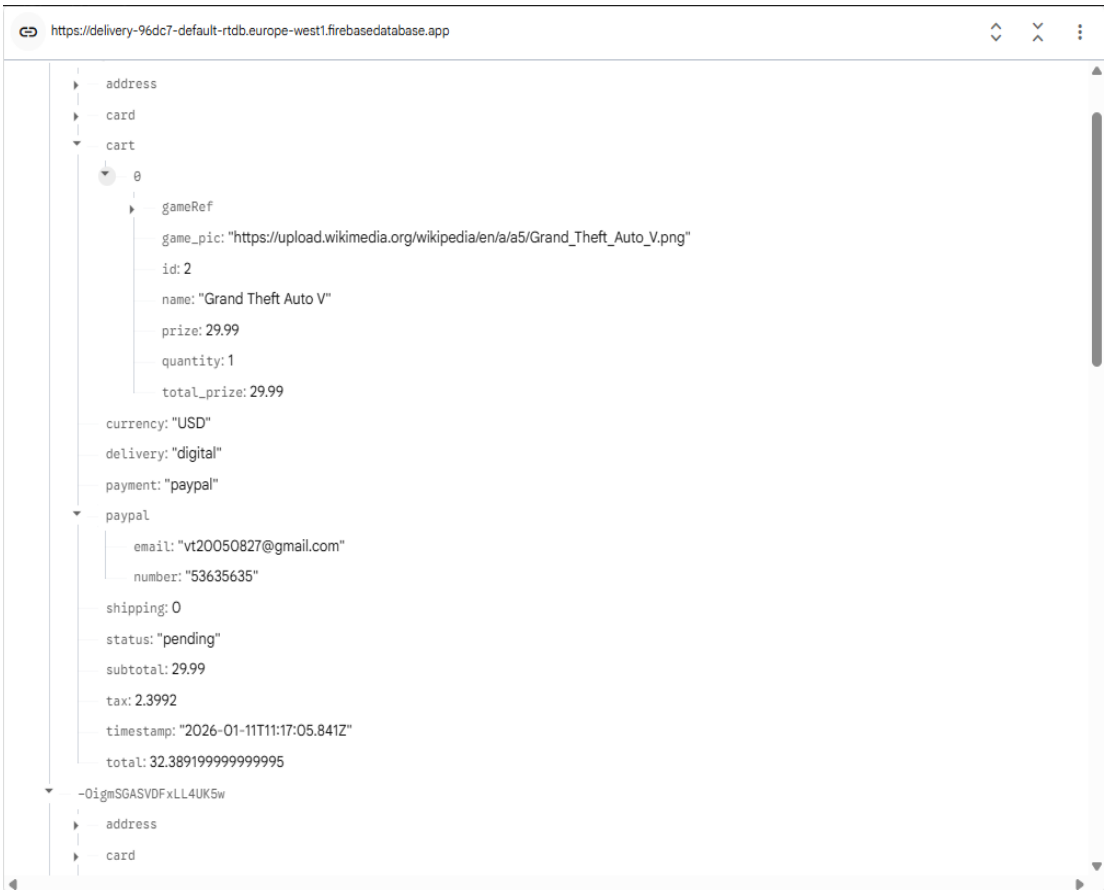
2.6.7.1. Leaderboard



32. ábra - Leaderboard

- **name string** (A játékos neve, amely megjelenik a ranglistán)
- **score number / int** (A játékos által elért pontszám)

2.6.7.2. A rendelés tábla



33. ábra - Delivery

cart

A vásárlói kosár tartalma, amely több terméket is tartalmazhat.

játék adatai

- **id integer** – A termék egyedi azonosítója
- **name string** – A játék neve
- **game_pic string (URL)** – A játék borítóképe
- **price decimal** – Egységár
- **quantity integer** – Vásárolt mennyiség
- **total_prize decimal** – Az adott termék teljes ára
- **delivery string** – A szállítás típusa (pl. digital)
- **currency string** – A fizetés pénzneme (pl. USD)

Fizetési adatok

- **payment string** – A fizetési mód (pl. paypal)

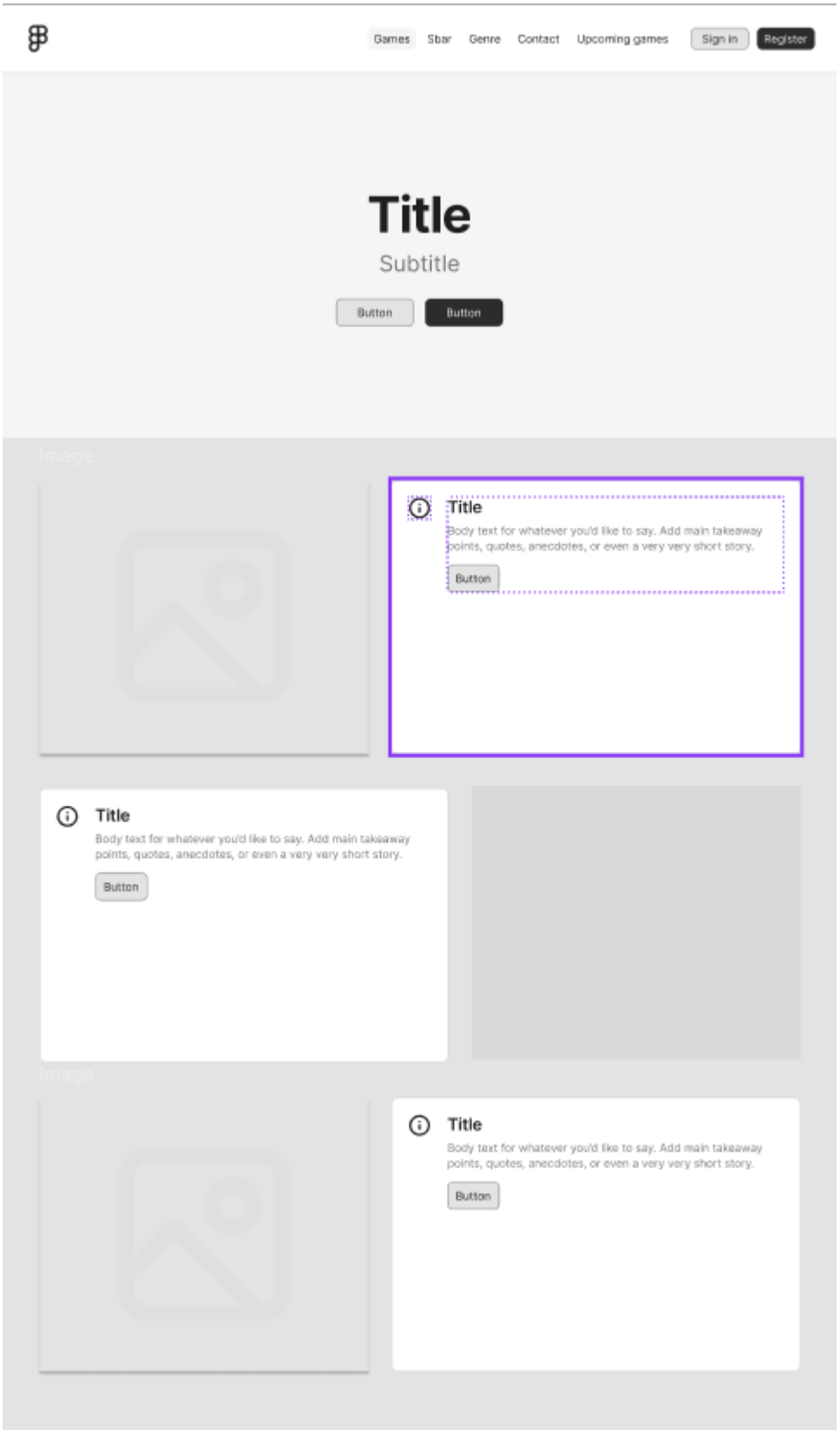
- **email string** – A PayPal fiókhoz tartozó email cím
- **number string** – A fizetéshez kapcsolódó azonosító

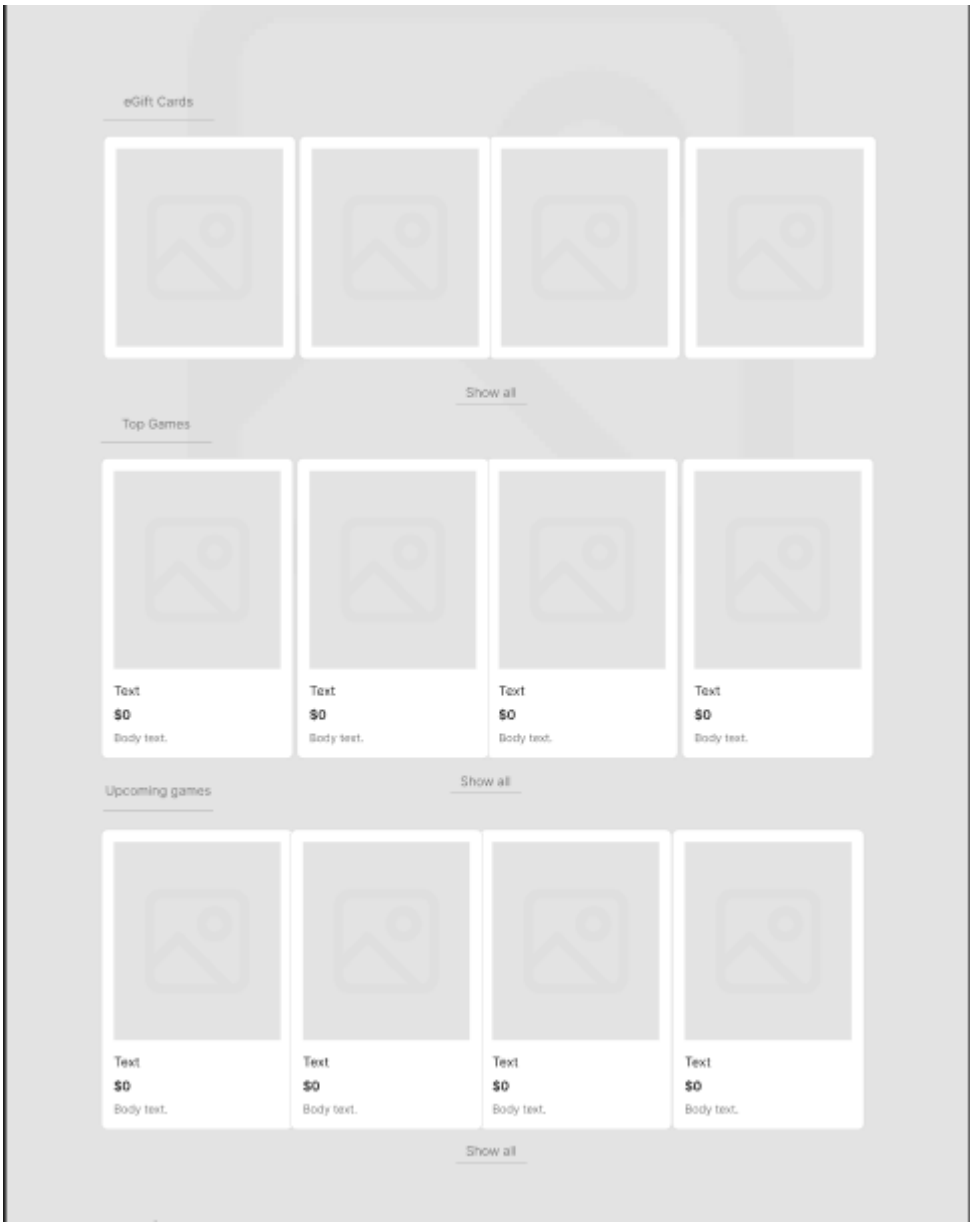
Ár és állapot információk

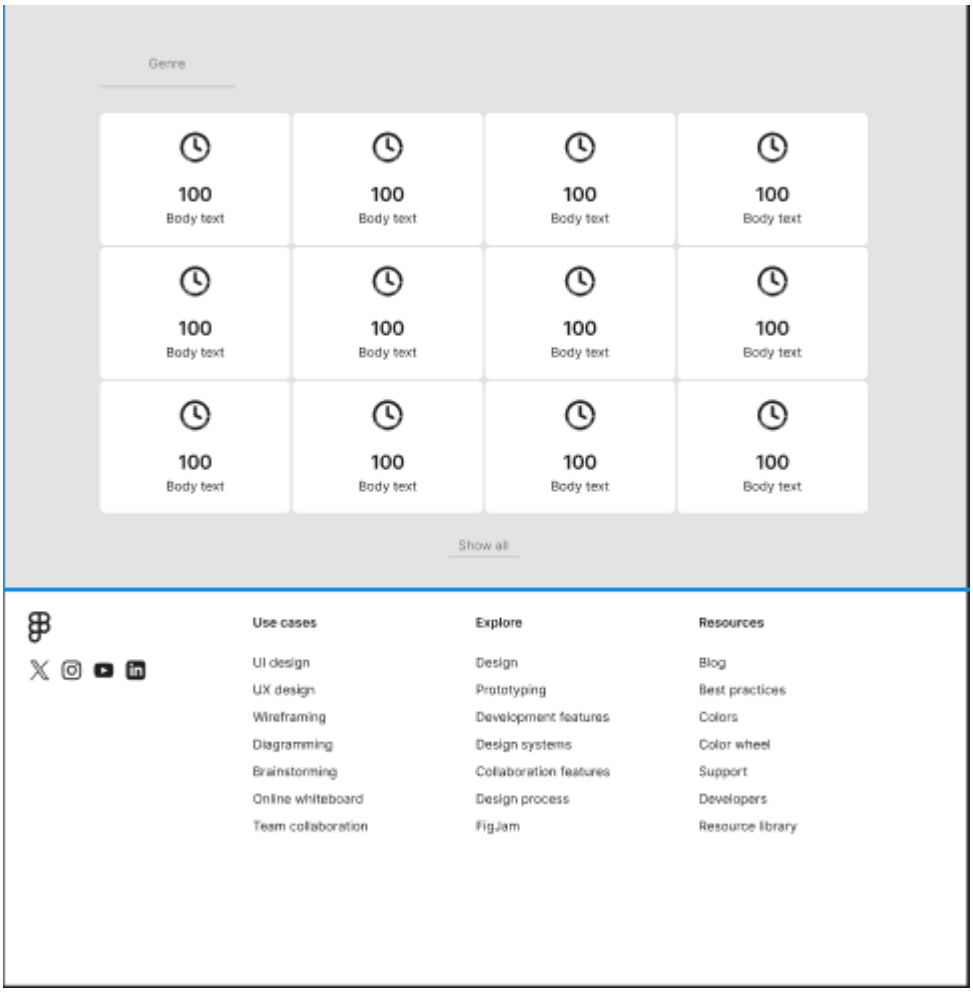
- **subtotal number** – Nettó végösszeg
- **tax number** – ÁFA összege
- **shipping number** – Szállítási költség
- **total number** – Bruttó végösszeg
- **status string** – Rendelés állapota (pl. pending)
- **timestamp string (ISO 8601)** – A rendelés leadásának időpontja

2.7. Figma használata az oldal teszteléséhez

2.7.1. Home weboldal terve

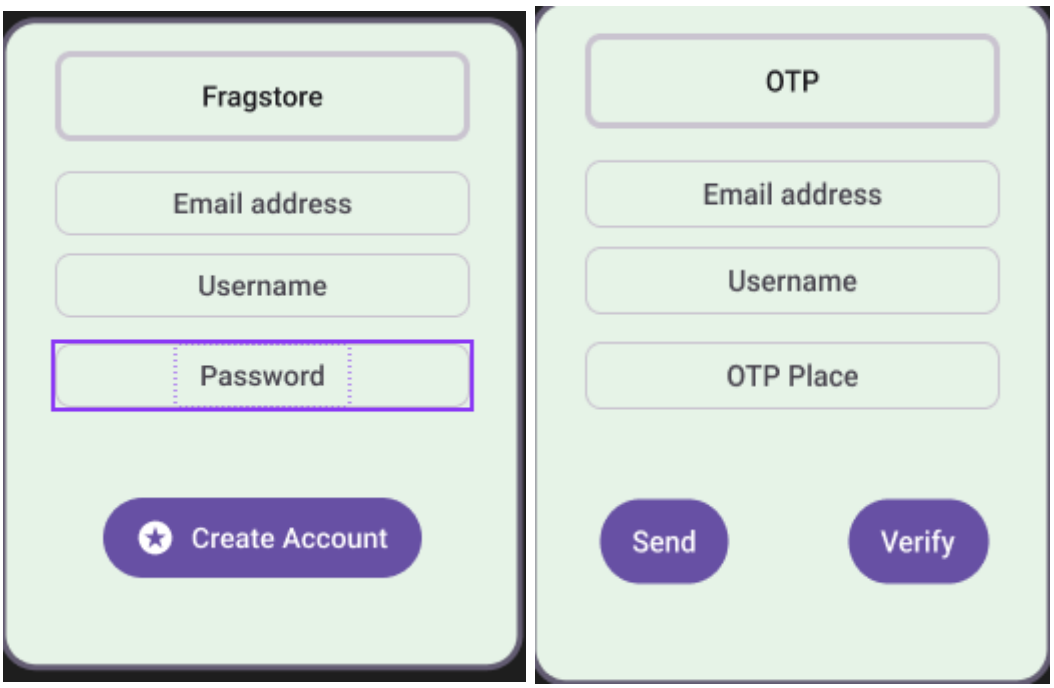



eGift Cards
Show all
Top Games
Text
$0
Body text.
Text
$0
Body text.
Text
$0
Body text.
Text
$0
Body text.
Show all
Upcoming games
Text
$0
Body text.
Text
$0
Body text.
Text
$0
Body text.
Text
$0
Body text.
Show all



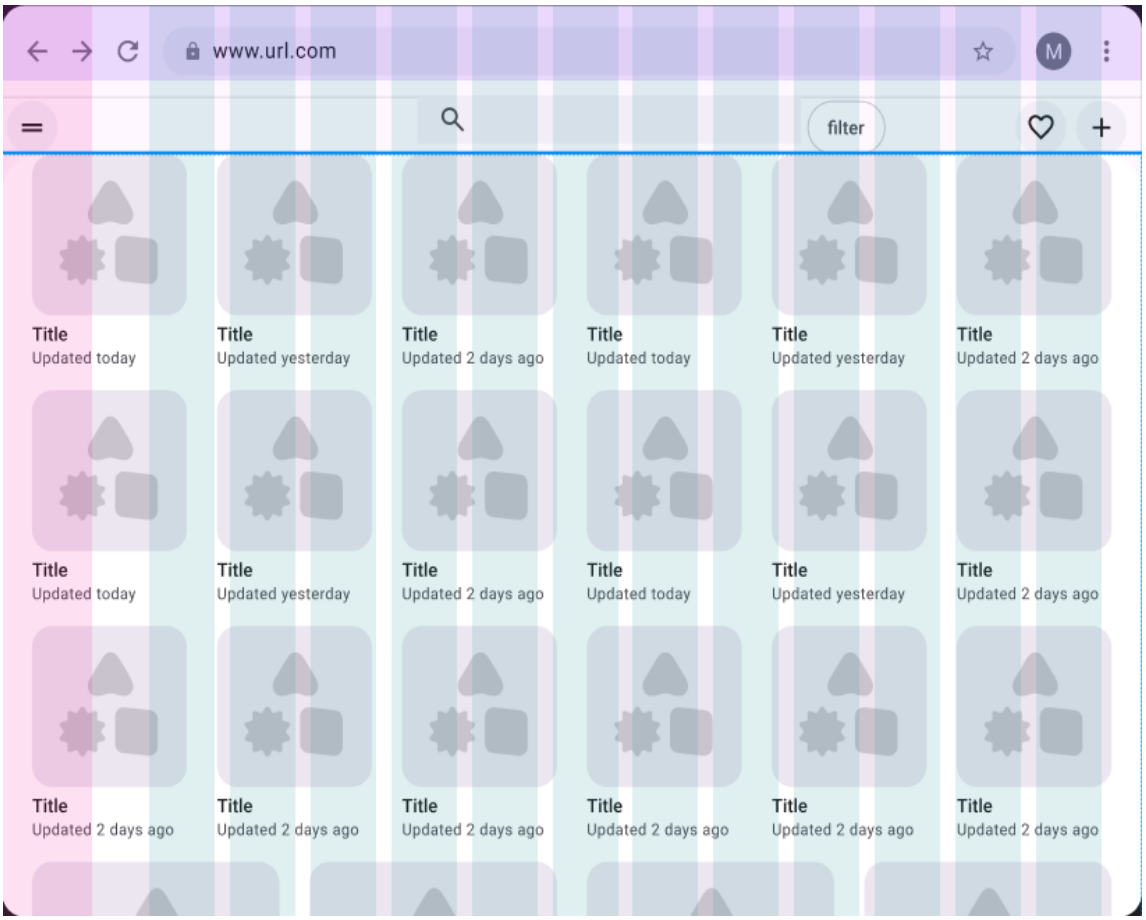
34. ábra - Home Figma

2.7.2. Login/Sign up



35. ábra - Login / Sign up

2.7.3. Main (webshop)



36. ábra - Main webshop Figma

2.8. Kódrészletek

2.8.1. Bevásárlókosár és kívánságlista (Wishlist)

```
<button class="shop-cart" ng-click="cartOpen = true" data-count={{count}}>
```

```
<svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" fill="#37e6ec73" viewBox="0 0 24 24">
<path d="M7 18c-1.1 0-1.99.9-1.99 2S5.9 22 7 22s2-.9 2-2-.9-2-2-2zm10
0c-1.1 0-1.99.9-1.99 2S15.9 22 17 22s2-.9 2-2-.9-2-2-2zM7.16
14h9.58c.75 0 1.41-.41 1.75-1.03l3.58-6.49A1 1 0 0 0
21.25 5H6.21L5.27 2H2v2h2l3.6 7.59-1.35
```

```
                 2.44C5.52  14.37  6.23  16  7.16  16H19v-
2H7.16z" />
            </svg>
        </button>

        <div   class="cart-sidebar"   ng-class="{'active':
cartOpen}">
            <div class="cart-header">
                <h2>Your Cart</h2>
                <button        class="close-btn"        ng-
click="cartOpen = false">×</button>
            </div>

            <div class="cart-content">
                <div  ng-if="cartItems.length  ===  0">Your
cart is empty.</div>

                <div  class="cart-item"  ng-repeat="item  in
cartItems">
                    <img ng-src="{{item.game_pic}}">
                    <div class="cart-data">
                        <h4>{{item.name}}</h4>
                        <p>{{item.total_prize  |  number:2
}} {{select_currency}}</p>
                    </div>

                    <div class="qty-controls">
                        <button                        ng-
click="decreaseQty(item)">-</button>
                        <span>{{item.quantity}}</span>
                        <button                        ng-
click="increaseQty(item)">+</button>
                    </div>
                </div>
```

```
                <button      class="redirect_btn"      ng-
click="checkout()"   ng-show="cartItems.length   >   0">Buy
Now</button>
            </div>
        </div>

        <div      class="overlay"      ng-class="{'active':
cartOpen}" ng-click="cartOpen = false"></div>

        <!-- Wishlist Sidebar -->
        <div          class="wishlist-sidebar"          ng-
class="{'active': wishlistOpen}">
            <div class="wishlist-header">
                <h2>Your Wishlist</h2>
                <button        class="close-btn"        ng-
click="wishlistOpen = false">×</button>
            </div>

            <div class="wishlist-content">
                <div    ng-if="wishlistItems.length    ===
0">Your wishlist is empty.</div>

                <div       class="wishlist-item"       ng-
repeat="item in wishlistItems track by item.id">
                    <img ng-src="{{item.game_pic}}">
                    <div class="wishlist-data">
                        <h4>{{item.name}}</h4>
                    </div>

                </div>
                <button      class="redirect_btn"      ng-
click="wishlistOpen          =          false"          ng-
hide="wishlistItems.length > 0">Add Now</button>
            </div>
```

```
        </div>

        <!-- Wishlist Overlay -->
        <div      class="overlay"      ng-class="{'active':
wishlistOpen}" ng-click="wishlistOpen = false"></div>


        <button class="wish-btn" ng-click="wishlistOpen =
true" data-count2={{count2}}>
            <svg        xmlns="http://www.w3.org/2000/svg"
fill="#37e6ec73"  viewBox="0  0  24  24"  stroke-width="0"
width="20" height="20" stroke="currentColor">
                <path    stroke-linecap="round"    stroke-
linejoin="round"  d="M21  8.25c0-2.485-2.099-4.5-4.688-4.5-
1.935  0-3.597  1.126-4.312  2.733-.715-1.607-2.377-2.733-
4.313-2.733C5.1  3.75  3  5.765  3  8.25c0  7.22  9  12  9  12s9-
4.78 9-12Z" />
            </svg>
        </button>
```

- Ez a kódrészlet a Fragstore webalkalmazás bevásárlókosár- és kívánságlista-funkcióinak megvalósításáért felel. A megoldás AngularJS direktívákra épül, amelyek lehetővé teszik az oldal dinamikus frissítését anélkül, hogy a teljes oldal újratöltésére lenne szükség.

- A kosár és a wishlist egy oldalsó panelként (sidebar) jelenik meg, amelyet a felhasználó egy ikonra kattintva tud megnyitni vagy bezárni. A megjelenítés állapotát a cartOpen és wishlistOpen változók szabályozzák, amelyek segítségével az adott panel aktív vagy inaktív állapotba kerül.

2.8.1.1. Bevásárlókosár működése

- A bevásárlókosár ikon egy gomb formájában jelenik meg, amelyen egy számláló (data-count) jelzi az aktuálisan kosárban lévő termékek számát. A gombra kattintva a kosár oldalsáv megnyílik, ahol a felhasználó áttekintheti a kiválasztott termékeket.

- A kosár tartalma dinamikusan kerül megjelenítésre az ng-repeat direktíva segítségével, amely végigiterál a cartItems listán. Minden elemhez tartozik egy kép, név, ár, valamint mennyiségkezelő vezérlő. A „+” és „–” gombok lehetővé teszik a termékek darabszámának növelését vagy csökkentését, amely azonnal frissíti az összegzett árat.

- Amennyiben a kosár üres, a rendszer egyértelmű visszajelzést ad a felhasználónak („Your cart is empty.”). Ha a kosár tartalmaz elemeket, megjelenik a „Buy Now” gomb, amely a vásárlási folyamat következő lépésére irányítja a felhasználót.

2.8.1.2. Kosár bezárása

- A kosár panel bezárása több módon is lehetséges: a bezáró gomb segítségével, illetve a háttérben megjelenő áttetsző overlay-re kattintva. Ez a megoldás intuitív felhasználói élményt biztosít, mivel megfelel a modern webalkalmazásokban megszokott működésnek.

2.8.1.3. Kívánságlista (Wishlist) működése

- A wishlist funkció hasonló elven működik, mint a bevásárlókosár. A kívánságlista ikonra kattintva egy külön oldalsáv jelenik meg, amely a felhasználó által elmentett játékokat tartalmazza. A wishlist elemei szintén dinamikusan kerülnek megjelenítésre az `ng-repeat` direktíva segítségével.

- Ha a kívánságlista üres, a rendszer ezt egy szöveges üzenettel jelzi, valamint egy „Add Now” gombot jelenít meg, amely visszavezeti a felhasználót a böngészéshez. Ez a megoldás ösztönzi a felhasználót arra, hogy új elemeket adjon hozzá a listához.

2.8.2. Játékok megjelenítése

```
<div class="game-container">
        <div class="card" ng-repeat="game in filteredGames | filter:{name: searchText} | limitTo:itemsPerPage:((currentPage - 1) * itemsPerPage)">
            <img ng-src="{{game.game_pic}}" alt="{{game.name}}" ng-click="easter_egg(game)">
            <p class="discount-badge" ng-if="game.isDiscount == 1">
                {{ (((game.prize - game.discountedPrize) / game.prize * 100)) * (-1) | number:0 }}%
            </p>
            <div class="card-content">
                <h2 class="title">{{game.name}} <button class="wish_btn off" data-game-id="{{game.id}}" ng-class="{'active': isInWishlist(game.id), 'off': !isInWishlist(game.id)}" ng-click="Wishlist(game, $event)">
                        <svg xmlns="http://www.w3.org/2000/svg" fill="transparent" viewBox="0 0 24 24" stroke-width="1.3" stroke="#00f7ff" class="WishBtn">
                            <path stroke-linecap="round" stroke-linejoin="round" d="M21 8.25c0-2.485-2.099-4.5-4.688-4.5-1.935 0-3.597 1.126-4.312 2.733-.715-1.607-2.377-2.733-4.313-2.733C5.1 3.75 3 5.765 3 8.25c0 7.22 9 12 9 12s9-4.78 9-12Z" />
                        </svg>
                    </button></h2>
```

```
                <p><strong>Genre:</strong>
{{game.genre}}</p>
                <p><strong>Platform:</strong>
{{game.platforms}}</p>
                <p><strong>Release:</strong>
{{game.release_date}}</p>
                <p
class="stock"><strong>Stock:</strong>
{{game.stock}}</p>
            </div>
            <div class="price-wrapper">
                <div class="status">
                    <h3 class="status_text"></h3>
                </div>
                <div class="price-box">
                    <p        class="price"        ng-
style="{'text-decoration':  game.isDiscount  ==  1  ?
'line-through' : 'none'}">
                        {{  (game.isDiscount  ==  1  ?
convertPrice({prize:          game.prize})          :
convertPrice({prize:         game.prize}))         }}
{{select_currency}}
                    </p>

                    <p      class="discount"      ng-
if="game.isDiscount == 1">
                        {{       convertPrice({prize:
game.discountedPrize      ||      game.prize})      }}
{{select_currency}}
                    </p>

                </div>
                <br>
```

```
                <p id="stock_text" ng-
style="{'color': game.stock > 0 ? 'var(--stock)' :
'var(--out_stock)'}">
                    {{ game.stock > 0 ? 'In Stock' :
'Out of Stock' }}
                </p>
                <div class="btns">
                    <button class="buy_btn" ng-
click="Content(game.name, game.genre)">Read
More</button>
                    <button class="shop_btn" ng-
click="openCart(game)"><svg
xmlns="http://www.w3.org/2000/svg" fill="none"
viewBox="0 0 24 24" stroke-width="1.5"
stroke="currentColor" class="size-6">
                        <path stroke-
linecap="round" stroke-linejoin="round" d="M2.25
3h1.386c.51 0 .955.343 1.087.835l.383 1.437M7.5
14.25a3 3 0 0 0-3 3h15.75m-12.75-3h11.218c1.121-2.3
2.1-4.684 2.924-7.138a60.114 60.114 0 0 0-16.536-
1.84M7.5 14.25 5.106 5.272M6 20.25a.75.75 0 1 1-1.5
0 .75.75 0 0 1 1.5 0Zm12.75 0a.75.75 0 1 1-1.5 0
.75.75 0 0 1 1.5 0Z" />
                    </svg>
                    </button>
                </div>
            </div>
        </div>
    </div>
```

- A játékok megjelenítése az ng-repeat direktíva segítségével történik, amely végigiterál a filteredGames listán. Ez a lista már előzetesen tartalmazza a szűrési és keresési feltételeknek megfelelő játékokat, így a felület mindig az aktuális felhasználói beállításokhoz igazodik.

2.8.2.1. Keresés és lapozás kezelése

- megjelenítés során a kód egyszerre több AngularJS funkciót kombinál. A filter:{name: searchText} lehetővé teszi, hogy a felhasználó név alapján keressen a játékok között, míg a limitTo direktíva az oldalakra bontott megjelenítést valósítja meg. Ennek köszönhetően egyszerre csak egy meghatározott számú játék jelenik meg, ami javítja az oldal áttekinthetőségét és teljesítményét.

- A lapozás logikája a currentPage és az itemsPerPage változók segítségével valósul meg, így a felhasználó könnyedén navigálhat az egyes oldalak között.

2.8.2.2. Árkezelés és készletinformációk

- Az árak megjelenítése egy külön függvény segítségével történik, amely a kiválasztott pénznemhez igazítja az értékeket. Ez lehetővé teszi a többvalutás működést anélkül, hogy az adatokat külön kellene tárolni minden pénznemhez.

- A készlet állapotát a rendszer vizuálisan is jelzi: amennyiben a termék elérhető, „In Stock” felirat jelenik meg, ellenkező esetben „Out of Stock”. A szöveg színe is változik, amely tovább segíti a gyors értelmezést.

2.8.2.3. Játékadatok és vizuális elemek megjelenítése

- Minden játék egy külön kártyán jelenik meg, amely tartalmazza a játék borítóképét, nevét, műfaját, platformját, megjelenési dátumát és készletinformációit. A borítókép kattintható, amely egy speciális funkciót (easter egg) indít el, ezzel növelve az oldal interaktivitását.

- A kód kezeli az akciós játékok megjelenítését is. Amennyiben egy játék kedvezményes áron érhető el, egy százalékos kedvezményt jelző címke jelenik meg, valamint az eredeti ár áthúzva, míg a kedvezményes ár kiemelten kerül megjelenítésre. Ez a megoldás vizuálisan egyértelművé teszi az akciókat a felhasználók számára.

2.8.2.4. Wishlist

- A játék nevéhez kapcsolódó szív ikon lehetővé teszi a játékok kívánságlistához adását. A gomb állapota dinamikusan változik attól függően, hogy az adott játék szerepel-e már a felhasználó wishlistjében. Az ng-class direktíva segítségével az ikon megjelenése azonnal frissül, így a felhasználó azonnali visszajelzést kap a művelet sikerességéről.

2.8.2.5. Műveleti gombok (Read More, Kosár)

- A kártyák alsó részén található gombok lehetővé teszik a további műveleteket. A „Read More” gomb a játék részletes oldalára navigál, míg a kosár ikon segítségével a felhasználó az adott játékot közvetlenül a bevásárlókosárhoz adhatja. Ez a megoldás gyors és kényelmes vásárlási élményt biztosít.

2.8.3. Fordítás a Home oldalon

```
function translate(text, from = "en", to = "auto") {

  if (!("Translator" in window)) {
    return Promise.resolve(text);
  }

  return Translator.create({ sourceLanguage: from, targetLanguage:
to })
    .then(translator => translator.translate(text))
    .catch(err => {
      console.error("Translation failed:", err);
      return text;
    });
}

function translateContent(lang) {
  var elements = document.querySelectorAll("[data-i18n]");

  elements.forEach(function(el) {
```

```
    if (!el.dataset.original) {
      el.dataset.original = el.textContent;
    }

    translate(el.dataset.original, "en", lang)
      .then(function(translated) {
        el.textContent = translated;
      })
      .catch(function(err) {
        console.error("Translation error:", err);
      });
  });
}

var selected = document.querySelector(".selected-lang");

document.querySelectorAll(".lang-menu a").forEach(function(link) {
  link.addEventListener("click", function(e) {
    e.preventDefault();
    var lang = link.className;

    selected.textContent = link.textContent;
    selected.style.setProperty("--flag", "url(" + link.dataset.flag
+ ")");
    document.querySelector(".lang-menu ul").style.display = "none";

    translateContent(lang);
  });
});

selected.addEventListener("click", function() {
  var menu = document.querySelector(".lang-menu ul");
```

```
  menu.style.display = menu.style.display === "block" ? "none" :
"block";
});
```

2.8.3.1. Translator API-val fordítás

- Ez a kódrészlet a Fragstore Home oldalán található többnyelvű felület dinamikus fordítását valósítja meg. A cél az volt, hogy a felhasználó a nyelvválasztó menü segítségével úgy tudjon nyelvet váltani, hogy az oldal tartalma azonnal frissüljön, és ne legyen szükség újratöltésre. A megoldás az oldalon található, fordítható elemeket `data-i18n` attribútummal azonosítja, majd ezeket automatikusan lefordítja a kiválasztott cél nyelvre.

2.8.3.2. `translate()` – fordítási réteg és hibakezelés

- A `translate(text, from, to)` függvény egy általános fordítófüggvény, amely egy szöveget fogad bemenetként, és Promise-alapú választ ad vissza. A kód először ellenőrzi, hogy a böngésző támogatja-e a `Translator` objektumot. Amennyiben nem érhető el, a függvény visszaadja az eredeti szöveget, így a weboldal fordítás nélkül is működőképes marad.
- Ha a fordító elérhető, a rendszer létrehoz egy fordító példányt (`Translator.create`), majd meghívja a `translate()` metódust. Hibakezelés esetén a rendszer naplózza a hibát, és visszaadja az eredeti szöveget, hogy a felhasználói felület ne „törjön szét” egy fordítási hiba miatt.
- Ez a megközelítés biztosítja a stabil működést: a fordítás extra funkció, de nem akadályozza az oldal használatát.

2.8.3.3. `translateContent()` – fordítható elemek és eredeti szöveg mentése

- A translateContent(lang) függvény felelős az oldal elemeinek tényleges lefordításáért. A kód kiválasztja az összes olyan elemet, amely rendelkezik data-i18n attribútummal, majd végigiterál rajtuk.

- Fontos része a megoldásnak, hogy a rendszer az elem eredeti szövegét elmenti az el.dataset.original mezőbe. Ez azért szükséges, mert a nyelvváltás során mindig ugyanabból az eredeti (alap) szövegből érdemes fordítani. Így elkerülhető az a probléma, hogy „fordításból fordítunk”, ami idővel pontatlanságot és minőségromlást okozhat.
- Miután az eredeti szöveg rendelkezésre áll, a rendszer meghívja a translate() függvényt, majd a visszakapott lefordított szöveget beállítja az elem tartalmaként.

2.8.3.4. Nyelvválasztó felület működése

- A nyelvválasztó menü kattintható elemekből áll (.lang-menu a). Minden nyelvhez tartozik egy osztálynév (például de, fr, jp), amelyet a kód nyelvkódként használ a fordítás során.

  A felhasználó kattintására:

  - a rendszer megakadályozza az alapértelmezett linkműködést (e.preventDefault()),
  - frissíti a kiválasztott nyelv megjelenítését,
  - frissíti a zászló ikont CSS változón keresztül,
  - elrejti a lenyíló listát,
  - meghívja a translateContent(lang) függvényt, amely lefordítja az oldal tartalmát.

- A kiválasztott nyelv megjelenítő elem (.selected-lang) külön eseménykezelőt kap, amely a menü lenyitását és bezárását kezeli, így a kezelőfelület egyszerű és intuitív marad.

2.8.3.5. Miért ezt a megoldást választottuk?

- A dinamikus fordítás egyik legnagyobb előnye, hogy a rendszer nem egy előre megírt szótárból dolgozik, hanem képes tetszőleges szöveget lefordítani. Ez különösen hasznos akkor, ha a weboldal tartalma gyakran változik, vagy ha új elemek kerülnek be a felületre.

A megoldás:

- nem igényel oldalfrissítést,
- stabil hibakezelést alkalmaz,
- menti az eredeti szöveget, így elkerüli a fordítási lánc hibáit,
- rugalmasan bővíthető új nyelvekkel.

2.8.4. Giftcard kiíratás a home weboldalon

```
  <section id="giftcard" class="section gift-cards-section">
      <h2 class="section-title gift_cards" data-
i18n="egiftcards">eGift Cards</h2>
      <div class="card-grid gift-cards-grid">
        <?php if (!empty($gift_img)): ?>
          <?php $giftIndex = 0; ?>
          <?php foreach ($gift_img as $img): ?>
            <a href="../games_main.php">
              <div class="gift-card-wrapper <?= $giftIndex >= $limit
? 'hidden-gift' : '' ?>">
                <div class="card gift-card">
                  <img src="<?= htmlspecialchars($img['IMG']); ?>"
class="h-48 gift_card_img" alt="<?= htmlspecialchars($img['Name']);
?>">
                  <p class="body-text name"><?=
htmlspecialchars($img['Name']); ?></p>
                </div>
              </div>
            </a>
            <?php $giftIndex++; ?>
          <?php endforeach; ?>
        <?php else: ?>
          <p>No gift cards found.</p>
        <?php endif; ?>
```

```
      </div>

      <?php if (count($gift_img) > $limit): ?>
        <button class="show-all-gift centered-show-all" data-
i18n="show_all">Show all</button>
      <?php endif; ?>
    </section>

<?php
include '../test.php';
$host = 'localhost';
$db = 'giftcards';
$user = 'root';
$pass = '';
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";

try {
    $pdo = new PDO($dsn, $user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $stmtImg = $pdo->query("SELECT IMG, Name FROM giftcard");
    $Img = $stmtImg->fetchAll(PDO::FETCH_ASSOC);

} catch (PDOException $e) {
    echo $twig->render('error.twig', [
        'title' => 'Unexpected Error',
        'message' => 'Something went wrong.',
        'details' => $e->getMessage(),
        'redirectUrl' => 'home.php'
    ]);
    $Img = [];
```

```
    exit;
}
```

2.8.4.1. Gift card szekció megjelenítése a Home oldalon

- Ez a kódrészlet a Home oldalon található „eGift Cards” szekció dinamikus felépítéséért felel. A gift cardok adatai adatbázisból kerülnek betöltésre, majd a felület kártyarácsos (grid) elrendezésben jeleníti meg őket. A cél az volt, hogy a felhasználó azonnal lássa a legfontosabb gift card típusokat, miközben az oldal megőrzi az átlátható, letisztult megjelenést.
- A szekció címe data-i18n attribútumot használ, ezért a többnyelvű rendszer automatikusan tudja fordítani (például „eGift Cards”, „Show all”).

2.8.4.2. Adatok lekérése MySQL adatbázisból PDO-val

- A gift cardok képei és nevei egy MySQL adatbázisból kerülnek lekérdezésre. A kapcsolat PDO segítségével valósul meg, amely biztonságos és rugalmas megoldás adatbázis-kezelésre. A kód a PDO hibakezelési módját ERRMODE_EXCEPTION-re állítja, így problémák esetén a rendszer kivételt dob, amit a try-catch blokk kezel.
- A lekérdezés egy egyszerű SELECT IMG, Name FROM giftcard, majd az eredmények fetchAll(PDO::FETCH_ASSOC) segítségével tömbként kerülnek feldolgozásra. Ez a formátum kényelmesen iterálható a megjelenítési logikában.

2.8.4.3. Kártyák megjelenítése és biztonságos kiíratás

- A megjelenítési rész PHP foreach ciklussal járja be a lekért rekordokat, és minden gift cardot egy külön kártyaelemben jelenít meg. A kód htmlspecialchars() függvényt használ mind a kép URL-jénél, mind a névnél, amely segít megelőzni a felhasználói inputból vagy adatbázisból származó nem kívánt HTML/JavaScript beágyazásokat (XSS jellegű problémák).

- A kártyák kattintható elemekként jelennek meg (<a href="../games_main.php">), így a felhasználó a gift card szekcióból egyszerűen tovább tud navigálni a játékok oldalára.

2.8.4.4. Limitálás és „Show all” funkció

- A felület nem jeleníti meg egyszerre az összes gift cardot, hanem egy limit ($limit) alapján csak az első elemeket mutatja. A limit feletti elemek egy feltételes CSS osztályt kapnak (hidden-gift), ami elrejti őket alapállapotban.
- Ha az adatbázisból visszakapott elemek száma nagyobb, mint a limit, a rendszer megjelenít egy „Show all” gombot. Ez a megoldás azért előnyös, mert:

    - az oldal nem lesz túl hosszú,
    - gyorsabb és átláthatóbb marad a Home felület,
    - a felhasználó dönthet, hogy szeretné-e az összes elemet megjeleníteni.

2.8.4.5. Hibakezelés Twig sablonnal

- Amennyiben az adatbázis kapcsolat vagy lekérdezés hibára fut, a rendszer nem „nyers” hibaüzenetet dob a felhasználónak, hanem egy Twig sablon segítségével jelenít meg egy kulturált hibaoldalt (error.twig). A hibasablon paraméterként kap egy címet, rövid üzenetet, részleteket, valamint egy visszairányító URL-t. Ez felhasználói szempontból is előnyös, mert nem omlik össze a teljes oldal, és a felhasználó vissza tud térni a főbb funkciókhoz.

2.8.4.6. Összegzés

- A gift card szekció kialakítása egy olyan megoldást valósít meg, amely adatbázisból tölt be tartalmat, reszponzívan megjeleníti azt, és limitálással biztosítja, hogy a Home oldal áttekinthető maradjon. A PDO-alapú adatkezelés, a htmlspecialchars() használata és a Twig-alapú hibakezelés együtt egy stabil és biztonságos megoldást ad a rendszernek.

2.8.5. Log in backendje

```
//starts a new session, initializes $_SESSION superglobal array ->
used for retrieving data across multiple pages
session_start();

require_once dirname(__DIR__) . '/config.php'; //loads PDO from
config.php
include '../test.php';

//helper function for removing special characters
function e($s)
{
  return htmlspecialchars((string)$s, ENT_QUOTES, 'UTF-8');
}

//base variables

$message = '';
$email = '';

//form uses post method -> waits for the user's submit

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
  $email    = trim($_POST['email'] ?? '');
  $password = $_POST['password'] ?? '';

  //checks if values are empty
  if ($email === '' || $password === '') {
    $message = 'Please fill in both fields.';
  } else {
    try { //retrieves username, e-mail, hashed password
```

```
      $stmt = $pdo->prepare('SELECT id, username, email,
password_hash FROM users WHERE email = ? LIMIT 1');
      $stmt->execute([$email]);
      $user = $stmt->fetch();

      //verifies user written password with the password has from
the database
      if ($user && password_verify($password,
$user['password_hash'])) {
        session_regenerate_id(true);
        $_SESSION['user_id']  = (int)$user['id']; //stores user_id
in the session
        $_SESSION['username'] = $user['username']; //stores username
in the session
        header("Location: OTP.php");
        exit;
      } else {
        $message = 'Invalid email or password.';
      }
    } catch (PDOException $e) {
      echo $twig->render('error.twig', [
        'title' => 'Unexpected Error',
        'message' => 'Something went wrong.',
        'details' => $e->getMessage(),
        'redirectUrl' => '../home/home.php'
      ]);
      exit;
    }
  }
}
```

2.8.5.1. Bejelentkezés (Login) backend folyamata

- Ez a kódrészlet a Fragstore bejelentkezési folyamatának szerveroldali logikáját valósítja meg. A cél, hogy a felhasználó által megadott e-mail cím és jelszó alapján a rendszer biztonságosan ellenőrizze a hitelesítő adatokat, majd sikeres belépés esetén munkamenetet (session) hozzon létre. A belépési folyamat egy OTP hitelesítési lépéssel folytatódik, így a rendszer nem áll meg pusztán jelszóellenőrzésnél.

2.8.5.2. Session kezelés és környezet előkészítése

- A folyamat elején a session_start() elindítja a munkamenetet, amely lehetővé teszi a felhasználói adatok több oldal között történő tárolását és későbbi felhasználását. A config.php betöltése biztosítja az adatbázis-kapcsolatot ($pdo), míg a Twig környezetet betöltő fájl segítségével a rendszer egységes hibaoldalakat tud megjeleníteni.
- A e() segédfüggvény a későbbi megjelenítésekhez nyújt biztonságos szövegkezelést, mivel HTML-escape-elést végez (megelőzve a nem kívánt karakterek vagy beágyazások megjelenését).

2.8.5.3. Adatellenőrzés és felhasználó lekérdezése

- A bejelentkezés POST metódus esetén fut le. A rendszer beolvassa az e-mailt és jelszót, majd ellenőrzi, hogy mindkét mező ki van-e töltve. Hiányos kitöltés esetén a felhasználó visszajelzést kap.
- Sikeres alapellenőrzés után egy előkészített SQL lekérdezés fut le:

    - a rendszer a users táblából lekéri a felhasználó azonosítóját, felhasználónevét, e-mail címét és a jelszó hash-t
    - a prepare() + execute() megoldás védi a rendszert az SQL injection jellegű támadásokkal szemben

2.8.5.4. Jelszó ellenőrzés és sikeres belépés

- A jelszó ellenőrzése a password_verify() függvénnyel történik, amely a felhasználó által megadott jelszót veti össze az adatbázisban tárolt hash-sel. Ha az ellenőrzés sikeres:

- session_regenerate_id(true) fut le, amely csökkenti a session hijacking / session fixation kockázatát
- a rendszer elmenti a user_id és username értékeket a sessionbe
- a felhasználó átirányításra kerül az OTP ellenőrző oldalra (OTP.php)

- Sikertelen ellenőrzés esetén a rendszer általános hibaüzenetet ad („Invalid email or password.”), ami biztonsági szempontból előnyös, mert nem árulja el, hogy az e-mail vagy a jelszó volt-e hibás.

2.8.5.5. Hibakezelés Twiggel

- Adatbázis hiba (PDOException) esetén a rendszer nem nyers hibát jelenít meg, hanem Twig sablonon keresztül egy egységes error oldalt renderel. Ez felhasználói szempontból kulturáltabb megoldás, és egyben segít abban is, hogy a rendszer ne szivárogtasson felesleges technikai részleteket.

2.8.6. Regisztráció (Sign up) backend folyamata

```
//starts a new session, initializes $_SESSION superglobal array ->
used for retrieving data across multiple pages This is the login
session_start();

require_once dirname(__DIR__) . '/config.php'; //loads PDO from
config.php
include '../test.php';

//helper function for removing special characters
function e($s)
{
  return htmlspecialchars((string)$s, ENT_QUOTES, 'UTF-8');
}

//base variables
```

```
$message = '';
$email = '';

//form uses post method -> waits for the user's submit

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
  $email    = trim($_POST['email'] ?? '');
  $password = $_POST['password'] ?? '';

  //checks if values are empty
  if ($email === '' || $password === '') {
    $message = 'Please fill in both fields.';
  } else {
    try { //retrieves username, e-mail, hashed password
      $stmt = $pdo->prepare('SELECT id, username, email,
password_hash FROM users WHERE email = ? LIMIT 1');
      $stmt->execute([$email]);
      $user = $stmt->fetch();

      //verifies user written password with the password has from
the database
      if ($user && password_verify($password,
$user['password_hash'])) {
        session_regenerate_id(true);
        $_SESSION['user_id']  = (int)$user['id']; //stores user_id
in the session
        $_SESSION['username'] = $user['username']; //stores username
in the session
        header("Location: OTP.php");
        exit;
      } else {
        $message = 'Invalid email or password.';
      }
```

```
  } catch (PDOException $e) {
    echo $twig->render('error.twig', [
      'title' => 'Unexpected Error',
      'message' => 'Something went wrong.',
      'details' => $e->getMessage(),
      'redirectUrl' => '../home/home.php'
    ]);
    exit;
  }
}
```

2.8.6.1. Regisztráció (Sign up) backend folyamata

- Ez a kódrészlet az új felhasználói fiókok létrehozását végzi a Fragstore rendszerben. A regisztráció célja, hogy a felhasználó által megadott adatok ellenőrzésre kerüljenek, majd a jelszó biztonságosan hash-elve bekerüljön az adatbázisba. A folyamat úgy van felépítve, hogy alapvető validációt végezzen, és kezelje a gyakori hibákat (például duplikált e-mail).

2.8.6.2. Alapadatok és validáció

- A rendszer POST kérés esetén beolvassa az e-mail címet, felhasználónevet és jelszót, majd több lépésben ellenőrzi az adatokat:

    - kötelező mezők kitöltése
    - e-mail formátum ellenőrzése (filter_var)
    - minimum jelszóhossz (legalább 8 karakter)

- Ez a validáció csökkenti a hibás vagy gyenge hitelesítő adatok kialakulását, és javítja a felhasználói élményt is, mivel a felhasználó azonnal visszajelzést kap.

2.8.6.3. Jelszó biztonságos tárolása és adatbázis mentés

- A jelszó tárolása soha nem „nyersen” történik: a rendszer password_hash() segítségével hash-t generál, amely kerül eltárolásra a users táblában. Az adatbázis művelet előkészített lekérdezéssel történik:

    - INSERT INTO users (email, username, password_hash, profile_picture) VALUES (?, ?, ?, ?)
    - a placeholder alapú megoldás szintén védi a rendszert SQL injection ellen

- A kód emellett egy alapértelmezett profilképet is hozzárendel az új felhasználóhoz (default.png tartalma kerül betöltésre). Ez biztosítja, hogy minden fiók egységesen rendelkezzen képpel még akkor is, ha a felhasználó nem tölt fel sajátot.

2.8.6.4. Duplikáció kezelése (e-mail vagy felhasználónév)

- Az egyik leggyakoribb regisztrációs hiba, amikor a felhasználó már létező e-mail címmel vagy felhasználónévvel próbál regisztrálni. Ezt az adatbázis egyedi kulcs (unique constraint) alapján SQLSTATE 23000 hibakóddal jelzi.
- A rendszer ezt külön kezeli: ilyenkor egy Twig hibaoldal jelenik meg „Email or username already exists” üzenettel, amely egyértelmű visszajelzést ad a felhasználónak.

2.8.6.5. Hibakezelés és felhasználóbarát visszajelzés

- A regisztráció során fellépő egyéb adatbázis-hibákat a rendszer szintén Twig sablonnal kezeli, így a felhasználó nem nyers technikai hibát lát, hanem egységes formátumú hibaoldalt. Ez növeli a rendszer professzionális jellegét és megkönnyíti a hibák későbbi diagnosztizálását.

2.8.7. OTP kétlépcsős azonosító

```
emailjs.init("2w8KrX-es6cBuW9Rt"); //User ID for EmailJS

let currentOTP = null;  //variable that stores OTP code
```

```
const d = new Date(); //calls current date

function generateOTP() {
  return Math.floor(100000 + Math.random() * 899999); //generate 6
digits OTP
}

//sends the email in the backend

function sendMail() {
  const emailInput = document.getElementById("email");
  const email = emailInput.value.trim();

  const nameInput = document.getElementById("name");
  const name = nameInput.value.trim();


  currentOTP = generateOTP();

  const params = {
    name: name,
    time: d.getFullYear(),
    user_email: email,
    message: currentOTP,
  };

  const serviceID = "service_7grce58"; //ID of the gmail service
  const templateID = "template_1j019ea"; //ID of the mail UI
template

  emailjs.send(serviceID, templateID, params)
    .then(res => {
```

```
      alert("OTP sent to your email!");
      emailInput.value = "";
      clearOTPInputs();
      focusFirstOTP();
    })
    .catch(err => {
      console.error("Email sending error:", err);
      alert("Failed to send OTP. Please try again later.");
    });
}

//verifying OTP

function Verify() {
  if (!currentOTP) {
    alert("Please request an OTP first.");
    return;
  }
  const otpInputs = document.querySelectorAll('.otp-input');
  let otpEntered = '';
  otpInputs.forEach(input => {
    otpEntered += input.value.trim(); //puts all 6 numbers to one
single string
  });

  if (otpEntered.length < 6) {
    alert("Please enter the full 6-digit OTP."); //checks if all 6
entered
    otpInputs[0].focus();
    return;
  }

  if (otpEntered === currentOTP.toString()) {
```

```
    alert("OTP Verified successfully!");
    sessionStorage.setItem("login", "true");
    window.location.replace("../home/home.php"); //redirection if
successful verification
    clearOTPInputs();
  } else {
    alert("Invalid OTP. Please try again.");
  }
}

function clearOTPInputs() {
  document.querySelectorAll('.otp-input').forEach(input =>
input.value = ''); //clear OTP code fields
}

function focusFirstOTP() {
  const first = document.querySelector('.otp-input');
  first.focus(); //focusing on first number field
}
```

2.8.7.1. OTP (One-Time Password) hitelesítés

- Ez a kódrészlet a Fragstore kétlépcsős azonosításának (OTP) kliensoldali részét valósítja meg. A cél, hogy a felhasználó a bejelentkezés után egy egyszer használatos kóddal igazolja, hogy valóban hozzáfér az e-mail fiókjához. A rendszer az OTP-t automatikusan generálja, majd EmailJS szolgáltatáson keresztül elküldi a megadott e-mail címre. A felhasználó ezután beírja a kapott 6 számjegyű kódot, amelyet a rendszer ellenőriz.

2.8.7.2. EmailJS inicializálása és a kódgenerálás

- A folyamat az EmailJS kliensoldali inicializálásával indul (emailjs.init()), amely lehetővé teszi az e-mailek küldését közvetlenül a böngészőből. Az OTP kódot a rendszer egy külön függvényben generálja (generateOTP()),

amely egy véletlenszerű, 6 számjegyű értéket állít elő. A generált kód ideiglenesen a currentOTP változóban kerül tárolásra, így később összehasonlítható a felhasználó által beírt kóddal.

2.8.7.3. OTP elküldése e-mailben

- A sendMail() függvény a felhasználó által megadott e-mail címet és nevet olvassa ki az űrlapmezőkből, majd létrehoz egy paramétercsomagot (params). Ebben szerepel a felhasználó neve, az aktuális év (például a sablonban megjeleníthető információként), a címzett e-mail címe és maga az OTP kód.
- Az e-mail küldése az EmailJS send() metódusával történik, amelyhez egy szolgáltatásazonosító (serviceID) és egy e-mail sablonazonosító (templateID) tartozik. Sikeres küldés esetén a felhasználó visszajelzést kap („OTP sent to your email!”), valamint a mezők és az OTP inputok alapállapotba kerülnek, hogy a felhasználó azonnal meg tudja kezdeni a kód beírását. Hibás küldés esetén a rendszer hibaüzenetet jelez, és nem engedi tovább a folyamatot.

2.8.7.4. OTP ellenőrzése és beléptetés

- A Verify() függvény először ellenőrzi, hogy lett-e már igényelve OTP kód (currentOTP értéke létezik-e). Ez megakadályozza, hogy a felhasználó kód nélkül próbáljon hitelesíteni.
- Ezután a rendszer összegyűjti a hat darab OTP mező tartalmát (.otp-input), és egyetlen karakterlánccá fűzi össze. Ha a beírt érték nem teljes hosszúságú, a rendszer figyelmezteti a felhasználót, és visszairányítja a fókuszt az első mezőre.
- Sikeres egyezés esetén a rendszer visszajelzést ad („OTP Verified successfully!”), majd sessionStorage segítségével eltárol egy egyszerű belépési jelzőt, és átirányítja a felhasználót a Home oldalra. Sikertelen egyezés esetén a felhasználó hibaüzenetet kap, és újra megpróbálhatja a kód beírását

2.8.7.5. Felhasználói élményt segítő kiegészítő függvények

- A folyamat részeként két kisegítő függvény is szerepel:
    - clearOTPInputs() kiüríti az OTP beviteli mezőket, így a felhasználó tiszta állapotból próbálkozhat újra.
    - focusFirstOTP() automatikusan az első OTP mezőre helyezi a fókuszt, ami gyorsabbá és kényelmesebbé teszi a kód beírását.
- Ezek a kis részletek javítják a felhasználói élményt, és csökkentik a hibázás esélyét a hitelesítési folyamat során.

2.8.8. Easter egg játék load (Pac Man)

```
document.querySelector(".start_text").addEventListener("click", ()
=> {
  const introAudio = Intro();
  introAudio.addEventListener("ended", () => {
    Load();
  });

  // also remove the keyboard listener so it can't trigger again
  document.body.removeEventListener("keyup", handleStart);
});

//Load function: sets game sizes, resets life stats, defines drawing
context, loads assets and the map, updates game every frame, defines
pac man movement

function Load() {
  document.querySelector(".game_canvas").style.display = "block";
  document.querySelector(".content").style.display = "none";
  hearts.style.display = "grid";
  lifeUI.innerHTML = lives;
```

```
  canva.width = canva_width;
  canva.height = canva_height;
  context = canva.getContext("2d"); //Defines drawing context,
rendering for the canvas through an object

  load_images();
  load_map();
  update();

  //movement of pac man

  document.addEventListener("keyup", (e) => {
    switch (e.code) {
      case "ArrowUp":
      case "KeyW":
        nextDirection = "U";
        break;
      case "ArrowDown":
      case "KeyS":
        nextDirection = "D";
        break;
      case "ArrowLeft":
      case "KeyA":
        nextDirection = "L";
        break;
      case "ArrowRight":
      case "KeyD":
        nextDirection = "R";
        break;
    }
  });
```

```
  //Iterates through all ghosts and give them a new random starting
distance

  for (let ghost of ghosts.values()) {
    const new_direction = directions[Math.floor(Math.random() * 4)];
    ghost.Refresh_Direction(new_direction);
  }
}

function Popup(x, y, points) {
  pop_ups.push({
    x,
    y,
    text: `+${points}`,
    alpha: 1,
    dy: -0.5
  });
}




//image load function

function load_images() {
  image_paths.forEach((path, i) => {
    images[i] = new Image();
    images[i].src = path;
  });

  blue_ghost_img = images[0];
  red_ghost_img = images[1];
  orange_ghost_img = images[2];
```

```
  pink_ghost_img = images[3];
  pacman_down_img = images[4];
  pacman_up_img = images[5];
  pacman_left_img = images[6];
  pacman_right_img = images[7];
  wall_img = images[8];
  cherry_img = images[9];
  frozen_orb_img = images[10];
  strawberry_img = images[11];
  portal_img = images[12];
  heart_img = images[13];

  scared_ghost_img = new Image();
  scared_ghost_img.src = "assets/ghosts/scaredGhost.png";
}
```

2.8.8.1. (Pac-Man) betöltése

- Ez a kódrészlet a Fragstore webalkalmazásban található Pac-Man jellegű easter egg játék indításáért és alapvető működésének előkészítéséért felel. A játék célja nem a fő funkciók kiváltása, hanem egy rejtett, szórakoztató extra tartalom biztosítása a felhasználók számára, amely növeli az oldal egyediségét és élményértékét.
- A játék indítása egy felhasználói interakcióhoz kötött, amely biztosítja, hogy az erőforrásigényes elemek (képek, pálya, animációk) csak akkor töltődjenek be, amikor valóban szükség van rájuk.

2.8.8.2. Játék indítása felhasználói esemény hatására

- A játék a „start” feliratra történő kattintással indul. A kattintás hatására egy bevezető hang (intro audio) kerül lejátszásra, amelynek befejeződését az ended esemény figyeli. Ez biztosítja, hogy a tényleges játéklogika csak a hang lejátszása után induljon el.

- A kód eltávolítja a billentyűzetes indítást kezelő eseményfigyelőt, így a játék nem indítható el többször egymás után. Ez megelőzi az esetleges logikai hibákat vagy duplikált inicializálást.

2.8.8.3. Load függvény – a játék környezetének előkészítése

- A Load() függvény felel a játék teljes környezetének inicializálásáért. Ennek során a rendszer:

    - megjeleníti a játékvásznat (canvas),
    - elrejti az oldal egyéb tartalmi elemeit,
    - inicializálja az életek számát és a hozzá tartozó felhasználói felületet,
    - beállítja a canvas méreteit,
    - létrehozza a 2D rajzolási kontextust.

- A rajzolási kontextus (getContext("2d")) teszi lehetővé a játékelemek grafikus megjelenítését és animálását a canvas felületen.

2.8.8.4. Erőforrások és pálya betöltése

- A játék indulásakor külön függvények felelnek az erőforrások betöltéséért:

    - load_images() betölti a játékhoz szükséges képeket (Pac-Man, szellemek, falak, gyümölcsök, portálok stb.),
    - load_map() betölti a pálya elrendezését,
    - update() elindítja a folyamatos frissítési ciklust, amely a játékmenet alapját képezi.

- Ez a struktúra segít elkülöníteni a játék különböző részeit, így a kód átláthatóbbá és könnyebben karbantarthatóvá válik.

2.8.8.5. Karakterirányítás billentyűzettel

- A játék vezérlése billentyűzet segítségével történik. A rendszer figyeli az iránygombokat, valamint a WASD billentyűket, és ezek alapján frissíti a Pac-Man következő mozgási irányát (nextDirection). Ez a megoldás

lehetővé teszi, hogy a játék többféle vezérlési stílust támogasson, így rugalmasabb felhasználói élményt nyújt.

2.8.8.6. Ellenfelek (szellemek) inicializálása

- A játék indulásakor a rendszer végigiterál az összes szellemen, és mindegyik számára véletlenszerű kezdő irányt határoz meg. Ez biztosítja, hogy minden játékindítás kicsit eltérő legyen, így növeli az újrajátszhatóságot és csökkenti a kiszámíthatóságot.

2.8.8.7. Pontfelugró (Popup) rendszer

- A Popup() függvény felelős a pontszerzést jelző animált szövegek létrehozásáért. Amikor a játékos pontot szerez, egy ideiglenes felirat jelenik meg a képernyőn, amely fokozatosan elhalványul és felfelé mozog. Ez vizuális visszajelzést ad a játékosnak, és növeli a játék dinamizmusát.

2.8.8.8. Képek betöltése és kezelése

- A load_images() függvény előre definiált útvonalak alapján tölti be a játékhoz szükséges képfájlokat. A képek tömbbe kerülnek, majd külön változókhoz rendelődnek a könnyebb felhasználás érdekében. A rendszer külön kezeli a „megijedt” szellem állapotához tartozó képet is, amely a játékmenet során speciális helyzetekben jelenik meg.
- Ez az előtöltési megoldás csökkenti a játék közbeni akadozást, mivel az erőforrások már a játékmenet elején betöltődnek.

2.8.9. Szellemek viselkedése (Pac Man)

```
switch (ghost.img) {
        case red_ghost_img: //Blinky's (red) job to target pac man's
position. By default it breaks out the switch cuz target values are
already defined
          break;

        case pink_ghost_img: //Pinky's (pink) job to get ahead of
pac man based on which direction it moves, offset is 32px
```

```
          switch (pacman.direction) {
            case "U":
              targetY -= offset; //pac man moves on -Y (upwards),
exp pac man (0,64), then ghost (0,32) so gets 32 pixels ahead of pac
man on +Y
              break;
            case "D":
              targetY += offset; //pac man(0,128), ghost(0,160)
              break;
            case "L":
              targetX -= offset; //pac man(128,0), ghost(96,0)
              break;
            case "R":
              targetX += offset; //pac man(20,0), ghost(52,0)
              break;
          }
          break;

        case blue_ghost_img: //Inky's (blue) job is the same as
Blinky's
          targetX = dx + pacman.x;
          targetY = dy + pacman.y;
          break;

        case orange_ghost_img: //Clyde's (orange) job is to chase
pac man if it's far away according to the ghost's radius (64px)
          if (distance <= 64) {
            // but if it gets close to Clyde, it runs away (thats
why targetX is 0, and targetY is set to canvas height -> bottom left
corner )
            targetX = 0;
            targetY = canva_height;
          }
```

```
        break;
    }
```

2.8.9.1. célpontkijelölés a Pac-Man játékban

- Ez a kódrészlet a Pac-Man minijáték ellenfeleinek (szellemek) viselkedését határozza meg. A játékban minden szellem külön logika alapján választ célpontot, amelyhez a mozgását igazítja. A cél az volt, hogy a szellemek ne azonos módon „kövessék” a játékost, hanem eltérő stratégiákkal növeljék a játék kihívását és változatosságát.
- A szellemek viselkedése egy switch szerkezettel kerül szétválasztásra a szellem típusa alapján (a hozzá tartozó sprite / kép alapján), így minden karakter saját szabályrendszert kap.

2.8.9.2. Blinky – közvetlen üldözés (piros szellem)

- A piros szellem (Blinky) alapvető feladata, hogy a játékos aktuális pozícióját célozza meg. Ebben az esetben a kódrészlet „break”-kel kilép, mert a célpont (targetX, targetY) már a logika korábbi részében beállításra került a Pac-Man pillanatnyi helyzete alapján.
  Ez a klasszikus üldöző viselkedés biztosítja, hogy legyen egy ellenfél, aki folyamatos nyomás alatt tartja a játékost.

2.8.9.3. Pinky – előre számoló támadás (rózsaszín szellem)

- A rózsaszín szellem (Pinky) nem a játékos aktuális helyét célozza, hanem a mozgásirányát figyelembe véve a Pac-Man elé próbál kerülni. A célpont a játékos irányától függően egy fix eltolással (offset, 32 pixel) módosul:

    - ha Pac-Man felfelé halad, a célpont a jelenlegi pozíciója felett lesz,
    - lefelé mozgásnál alatta,
    - balra/jobbra mozgásnál a megfelelő oldalirányban.

- Ez a viselkedés olyan hatást kelt, mintha a szellem „előre gondolkodna”, és csapdába próbálná terelni a játékost.

2.8.9.4. Inky – dinamikus követés (kék szellem)

- A kék szellem (Inky) a játékos pozíciójához viszonyított célpontot választ. A kód alapján a célpont a Pac-Man helyzetéhez hozzáadott eltolásokból (dx, dy) számolódik.
  Ez a megoldás változó viselkedést eredményez: nem mindig ugyanoda tart, így kevésbé kiszámítható mozgást ad, ami növeli a játék dinamikáját.

2.8.9.5. Clyde – távolságfüggő stratégia (narancssárga szellem)

- A narancssárga szellem (Clyde) viselkedése távolságtól függ. A rendszer figyeli a Pac-Man és Clyde közötti távolságot, és ha a játékos túl közel kerül (a példában 64 pixel radius), a szellem „menekülő” viselkedésre vált.
- Ilyenkor a célpont egy fix sarokpontra áll át (például a bal alsó sarok: targetX = 0, targetY = canva_height).
  Ennek a logikának köszönhetően Clyde nem folyamatosan üldöz, hanem „cikázó”, kiszámíthatatlan ellenfélként működik: néha üldöz, néha elkerül.

2.8.9.6. Miért előnyös a külön viselkedés?

- A különböző szellemek eltérő célpontkijelölése több szempontból is javítja a játékmenetet:

  - változatosabb kihívást ad, mert a játékos nem egyetlen mintára reagál,
  - csökkenti a monotonitást,
  - magasabb szintű tervezést mutat be (nem csak „random mozgás”),
  - jobban visszaadja a klasszikus Pac-Man élményt.

- Összességében ez a megoldás a minijátékot komplexebbé és élvezetesebbé teszi, valamint bemutatja a fejlesztés során alkalmazott logikai gondolkodást és algoritmikus megközelítést.

2.8.10. Snake easter egg game

```
class Snake {
  constructor(x, y, VelocityX, VelocityY) {
```

```
    this.body = []; //snake's body
    this.x = x;
    this.y = y;
    this.VelocityX = VelocityX;
    this.VelocityY = VelocityY;

    //starting length of the snake (5 => head + 4 body part
blocks)
    for (let i = 4; i >= 0; i--) {
      this.body.push({ x: x - 1 * cell, y: y });
    }

    //generating random X and Y positions for the food
    // (width - hitbox) and (height - hitbox) makes sure the
food or its hitbox doesnt go beyond the game canvas border

    this.foodX = Math.floor(Math.random() * (width - hitbox));
    this.foodY = Math.floor(Math.random() * (height - hitbox));

    //Update calculate the snakes new position based on where
the head moves
    //exp.  60 + 1 * 20 => 60 + 20 = 80 (x), 40 + 0 * 20 = 40
(y)
    this.update = function () {
      let body_part = {
        x: this.body[this.body.length - 1].x + this.VelocityX *
cell,
        y: this.body[this.body.length - 1].y + this.VelocityY *
cell,
      };

      this.body.push(body_part); //new head appears at (80,40)
```

```
      //Constrain Logic
      if (body_part.x >= width || body_part.y >= height ||
body_part.x < 0 || body_part.y < 0) {
        this.x = 200;
        this.y = 200;
        this.body = [{ x: 200, y: 200 }]; //defaults back to
starting position
        this.VelocityX = 1;
        this.VelocityY = 0;
        Freeze(); //Game stops
        return;
      }

      //Self-Collision Check => if snake's head collides into
any part of its own body (in any axis), the game stops & resets
      for (let i = 0; i < this.body.length - 1; i++) {
        if (this.body[i].x == body_part.x && this.body[i].y ==
body_part.y) {
          Freeze();
          return;
        }
      }

      //Food Collision logic => Checks if snake collides into
the food in any axis
      // (hitbox - cell) => (30 - 20) = 1 decreases the hitbox,
otherwise too easy to get food.

      if (body_part.x >= this.foodX - (hitbox - cell) / 2 &&
body_part.x < this.foodX + hitbox - (hitbox - cell) / 2 &&
body_part.y >= this.foodY - (hitbox - cell) / 2 && body_part.y
< this.foodY + hitbox - (hitbox - cell) / 2) {
        //Random Food generation again
```

```
        this.foodX = Math.floor(Math.random() * (width -
hitbox));
        this.foodY = Math.floor(Math.random() * (height -
hitbox));
        score++; //increases current score
        scoreDiv = document.querySelector(".score_heading");
        scoreDiv.innerHTML = `Score: ${score}`;

        //High Score Checking
        if (score > high_score) {
        high_score = score;
        localStorage.setItem("high_score", high_score);
        HighScoreDiv.innerHTML = `High Score: ${high_score}`;
    }

      } else {
        //if snake doesn't collide with any food or any other
entity on the map, it removes it's tail as it moves every frame
(represents snake moving)
        this.body.shift();
      }
    };

    //declaring and setting every sprite in the game
    this.show = function () {
      //createImg() built in p5.js function for image creation
      snakeImg = createImg("assets/body.png");
      snakeImg.hide(); //initial hide
      foodImg = createImg("assets/apple.png");
      foodImg.hide();

      //draws out all foods and snake blocks
      for (let part of this.body) {
```

```
        image(foodImg, snake.foodX, snake.foodY);
        image(snakeImg, part.x, part.y);
      }
    };
  }
}
```

2.8.10.1. A snake játékmenete

- Ez a kódrészlet a Fragstore webalkalmazásban található Snake minijáték alaplogikáját valósítja meg. A játék működése egy Snake osztály köré épül, amely a kígyó mozgását, növekedését, az étel generálását, az ütközések kezelését és a pontozást egyetlen, jól elkülönített egységben kezeli. A megoldás célja az volt, hogy a játékmenet könnyen átlátható és bővíthető legyen, miközben a felhasználó egy egyszerű, de élvezetes extra tartalmat kap.

2.8.10.2. Kezdeti állapot és a snake létrehozása

- A Snake konstruktor a kezdő pozíciót és a kezdő sebességvektorokat (VelocityX, VelocityY) kapja paraméterként. A kígyó teste egy tömbben (body) tárolódik, amely koordinátapárokból áll. Induláskor a játék egy fix kezdő hosszúságú kígyót hoz létre: a fej + 4 testelem (összesen 5 blokk). Ez a felépítés vizuálisan is egyértelműen jelzi a játékmenet indulását, és azonnal játszható állapotot eredményez.

2.8.10.3. Étel generálása és pályahatárok

- A játék az étel pozícióját véletlenszerűen generálja (foodX, foodY). A számítás során figyelembe veszi a pálya méreteit (width, height) és a hitbox értékét, így az étel nem kerül a játéktér határain kívülre. Ez megakadályozza az olyan hibákat, amikor az étel elérhetetlen helyen jelenne meg.

2.8.10.4. `update()` – mozgás és játékmenet frissítése

- Az update() függvény felel a játék egy „lépésének” kiszámításáért. A kígyó új fej pozíciója a jelenlegi irányvektor alapján kerül meghatározásra: a sebesség (VelocityX, VelocityY) cellamérettel (cell) szorozva adja meg, merre mozdul a kígyó a következő frissítéskor. Az új fej koordináta hozzáadódik a body tömbhöz, ami vizuálisan a kígyó előrehaladását eredményezi.

2.8.10.5. Pályahatár (constrain) és reset logika

- A kód külön kezeli, ha a kígyó elhagyja a játéktér határait. Amennyiben a fej koordinátája kívül kerül a pályán (x >= width, y >= height, x < 0, y < 0), a rendszer visszaállítja a kígyót alaphelyzetbe:

    - alap kezdőpozícióra állítja a koordinátákat,
    - a testet minimálisra reseteli,
    - visszaállítja az alap mozgásirányt,
    - a Freeze() függvénnyel leállítja a játékot.

- Ez biztosítja, hogy a játék egyértelműen reagáljon a hibás mozgásra, és ne fusson tovább érvénytelen állapotban.

2.8.10.6. Önálló ütközés (self-collision) kezelése

- A játékmenet része, hogy ha a kígyó feje nekiütközik a saját testéhez, a játék véget ér. A kód a test összes elemét összehasonlítja az új fej pozíciójával. Ha egyezést talál, meghívja a Freeze() függvényt, amely leállítja a játékot. Ez a mechanizmus adja a Snake egyik alap nehézségi elemét.

2.8.10.7. Pontozás

Amikor a kígyó feje eléri az étel hitboxát, a rendszer:

- új ételpozíciót generál,

- növeli a score értékét,
- frissíti a megjelenített pontszámot a felületen.

A hitbox szándékosan módosított (csökkentett) méretű, hogy a játék ne legyen túl könnyű, és a játékosnak pontosabban kelljen rámozognia az ételre.

2.8.10.8. High Score mentése localStorageba

- A rendszer folyamatosan figyeli, hogy az aktuális pontszám meghaladja-e a korábbi legjobb eredményt. Ha igen, frissíti a `high_score` változót, majd elmenti azt `localStorage`-ba. Ez lehetővé teszi, hogy a felhasználó a böngésző bezárása után is megőrizze a legjobb eredményét, így a minijáték motiválóbb és újrajátszhatóbb.

2.8.10.9. Mozgás illúziója: farok törlése

- Ha a kígyó nem eszik, akkor a kód eltávolítja a test legrégebbi elemét (shift()), miközben az új fej hozzáadódik. Ettől alakul ki a mozgás illúziója: a kígyó „előrehalad”, de a hossza nem változik. Evés esetén ez a törlés nem történik meg, így a kígyó hosszabb lesz.

2.8.10.10. `show()` – megjelenítés p5.js-szel

- A show() függvény felel a játék grafikai megjelenítéséért. A megoldás p5.js eszközöket használ (createImg, image) a kígyó és az étel sprite-jainak megjelenítésére. A képek létrehozása és elrejtése (hide()) előkészíti az asseteket, majd a rajzolás során a kód kirajzolja:

  - az ételt a jelenlegi koordinátára,
  - a kígyó összes testelemét a body tömb alapján.

2.8.11. Snake control

```
function Controls() {
  document.addEventListener("keydown", function (e) {
    if (!snake) {
      return; //if snake isnt rendered yet, return
```

```
    }
    switch (e.key) {
      case "ArrowUp":
        if (snake.VelocityY !== 1) { //makes sure snake isn't
going down (down => up: self collision)
          snake.VelocityX = 0;
          snake.VelocityY = -1; //-Y => up
        }
        break;

      case "ArrowDown":
        if (snake.VelocityY !== -1) { //makes sure snake isn't
going up (up => down: self collision)
          snake.VelocityX = 0;
          snake.VelocityY = 1; //+Y => down
        }
        break;

      case "ArrowLeft":
        if (snake.VelocityX !== 1) { //makes sure snake isn't
going right (right => left: self collision)
          snake.VelocityY = 0;
          snake.VelocityX = -1; //-X => left
        }
        break;

      case "ArrowRight":
        if (snake.VelocityX !== -1) { //makes sure snake isn't
going left (left => right: self collision)
          snake.VelocityY = 0;
          snake.VelocityX = 1; //+X => right
        }
        break;
```

```
    }
  });
}
```

2.8.11.1. Irányítás és mozgásvezérlés

- Ez a kódrészlet a Snake easter egg játék irányítását valósítja meg billentyűzetes eseménykezelés segítségével. A cél az volt, hogy a játék vezérlése egyszerű és azonnal érthető legyen, miközben a logika megakadályozza az olyan hibás mozgásokat, amelyek azonnali önütközéshez vezetnének.

2.8.11.2. Billentyűzetes eseménykezelés

- A mozgásvezérlés a nyílbillentyűk segítségével történik. Minden billentyűhöz egy irányváltási logika tartozik, amely a kígyó sebességvektorait (VelocityX, VelocityY) módosítja. A megoldás a mozgást nem pozícióalapúan, hanem irányvektorok segítségével kezeli, ami letisztultabb és könnyebben karbantartható megoldást eredményez.

2.8.11.3. Önütközés megelőzése irányváltáskor

- A kód egyik legfontosabb része, hogy nem engedi a kígyót azonnal az ellentétes irányba fordulni. Például ha a kígyó lefelé halad, nem válthat azonnal felfelé, mert ez azonnali önütközést okozna. Ennek megelőzésére a rendszer minden irányváltás előtt ellenőrzi az aktuális sebességvektor értékét.

  Ez a logika biztosítja:

  - a játékmenet folyamatosságát,
  - a hibamentes mozgást,
  - a Snake játék klasszikus szabályainak betartását.

2.8.11.4. Felhasználói élmény szempontjai

- A mozgásvezérlés kizárólag egy irányt engedélyez egyszerre, így a kígyó mindig rácsszerűen halad a pályán. Ez a megoldás kiszámítható és pontos

irányítást tesz lehetővé, ami különösen fontos egy reflexekre és térbeli gondolkodásra épülő játék esetében.

2.8.12. Tetris kirájzolás

```
function drawTetromino(tetromino: typeof tetrominoRef.current)
{
      if (!tetromino) return;
      ctx.fillStyle = getTetrominoColor(tetromino.type); //fill
Tetromino according to it's color

      tetromino.shape.forEach((row, y) => {
        //loops through rows and their index (y)
        row.forEach((cell, x) => {
          //loops through columns (cells) and their index (x)
          if (cell) {
            //only draws if cell is 1, not 0
            ctx.fillRect(
              (tetromino.x + x) * BLOCK_SIZE, //(x coordinate +
row index) * block size  = PX
              (tetromino.y + y) * BLOCK_SIZE, //(y coordinate +
column index) * block size  = PX
              BLOCK_SIZE, //width
              BLOCK_SIZE //height
            );
          }
        });
      });
    }


    //Function for drawing out next random tetromino before
first
```

```
    function Next_Tetromino(tetromino: typeof
NextTetrominoRef.current) {
      if (!tetromino) return;

      const sidebar = NextcanvasRef.current; //we set the
sidebar variable to it's own canvas
      if (!sidebar) return;

      const sidebar_ctx = sidebar.getContext("2d"); //set the
rendering to 2D
      if (!sidebar_ctx) return;

      sidebar_ctx.clearRect(0, 0, sidebar.width,
sidebar.height); //perform initial clean on canvas

      sidebar_ctx.fillStyle =
getTetrominoColor(tetromino.type); //chooses the color that
applies to the tetromino

      tetromino.shape.forEach((row, y) => { //we go through
tetromino
        row.forEach((cell, x) => {
          if (cell) {
            sidebar_ctx.fillRect(
              x * BLOCK_SIZE + 25, //defining x position
              y * BLOCK_SIZE + 20, //defining y position
              BLOCK_SIZE,
              BLOCK_SIZE
            );
          }
        });
      });
```

```
}
```

2.8.12.1. Tetrominók kirajzolása és megjelenítés

- Ez a kódrészlet a Fragstore webalkalmazásban található Tetris easter egg játék grafikai megjelenítésének egy fontos részét valósítja meg. A cél a játék aktuális tetrominójának, valamint a következő tetrominó előnézetének kirajzolása HTML canvas segítségével. A megoldás TypeScript alapú, amely hozzájárul a típusbiztonsághoz és az átláthatóbb kódszerkezethez.

2.8.12.2. Aktív tetrominó kirajzolása

- A drawTetromino() függvény felelős az éppen aktív tetrominó megjelenítéséért a fő játékvásznon. A függvény paraméterként megkapja az aktuális tetrominót, majd első lépésként ellenőrzi, hogy az objektum létezik-e. Ez a védelem megakadályozza, hogy a rajzolás nem inicializált állapotban fusson le.
- A tetrominó színe a getTetrominoColor() segédfüggvény segítségével kerül meghatározásra, amely a tetrominó típusához rendelt színt adja vissza. Ez biztosítja, hogy minden elem a klasszikus Tetris megjelenéshez igazodjon.
- A tetrominó alakja egy kétdimenziós tömb (shape), amely soronként és oszloponként kerül feldolgozásra. A kód csak azokat a cellákat rajzolja ki, amelyek értéke 1, így a tényleges forma pontosan jelenik meg. A kirajzolás során a rácsalapú koordináták pixelpozíciókká alakulnak át a BLOCK_SIZE szorzó segítségével.

2.8.12.3. Pozíciószámítás és rácsalapú megjelenítés

- A rajzolás során a tetrominó aktuális pozíciója (tetromino.x, tetromino.y) összeadódik az adott cella indexével. Ez a megoldás lehetővé teszi, hogy a tetrominó mozgása egyszerűen kezelhető legyen, mivel a logikai rács és a vizuális megjelenítés szorosan összekapcsolódik. A blokkos elrendezés hozzájárul a Tetris klasszikus, jól felismerhető megjelenéséhez.

2.8.12.4. Következő tetrominó

- A Next_Tetromino() függvény a következő tetrominó előnézeti megjelenítéséért felel. Ez a funkció egy külön canvas elemet használ,

amely oldalsávként jelenik meg a játéktér mellett. A megoldás célja, hogy a játékos előre lássa a következő elemet, így tudatosabban tudjon döntéseket hozni a játék során.

- A függvény először ellenőrzi, hogy az előnézeti tetrominó és a hozzá tartozó canvas létezik-e. Ezután a rajzolási kontextus (2D context) inicializálása történik meg, majd a vászon teljes területe törlésre kerül, hogy az előző állapot ne maradjon látható.

2.8.12.5. Pozicionálás és igazítás

- Az előnézeti tetrominó kirajzolása hasonló logika alapján történik, mint a fő játéktéren, azonban itt a blokkok pozíciója egy fix eltolással (`+25`, `+20`) kerül meghatározásra. Ez az eltolás biztosítja, hogy az elem középre igazítva, esztétikusan jelenjen meg az oldalsávban, függetlenül a tetrominó alakjától.
- A színezés itt is egységesen a tetrominó típusához igazodik, így a játékos könnyen felismeri az előnézeti elemet.

2.8.13. Általános ütközésvizsgáló

```
function CheckCollision(tetro: typeof tetrominoRef.current,
offsetX = 0, offsetY = 0) {
      if (!tetro) return false;

      //Loops through rows and columns of Tetromino

      for (let y = 0; y < tetro.shape.length; y++) {
        for (let x = 0; x < tetro.shape[y].length; x++) {
          if (tetro.shape[y][x]) {
            //if filled (1), assign new x and y
            const newX = tetro.x + x + offsetX; //exp. 4 + 1 +
1 = 6
            const newY = tetro.y + y + offsetY; //exp. 4 + 0 +
0 = 5 (stays the same)
```

```
            // Checks whether it goes outside any borders on x
and y
            if (newX < 0 || newX >= COLS || newY >= ROWS)
return true;

            // Loops through already placed Tetrominos
            for (const placed of placedTetrominos.current) {
              //placed.shape[newY - placed.y] => checks if row
exists
              //placed.shape[newY - placed.y][newX - placed.x]
=> checks if cell is filled
              // && binds the logic together
              if (placed.shape[newY - placed.y] &&
placed.shape[newY - placed.y][newX - placed.x]
              ) {
                return true;
              }
            }
          }
        }
      }

      return false;
    }

    // Bottom collision => OffsetY + 1 => We check one row
ahead on Y
    function BottomCollision(tetro: typeof
tetrominoRef.current) {
      return CheckCollision(tetro, 0, 1);
    }
```

```
    // Left collision => OffsetX - 1 => We check one column to
the left on X
    function LeftCollision(tetro: typeof tetrominoRef.current)
{
      return CheckCollision(tetro, -1, 0);
    }

    // Right collision => OffsetX + 1 => We check one column to
the right on X
    function RightCollision(tetro: typeof tetrominoRef.current)
{
      return CheckCollision(tetro, 1, 0);
    }

    //Edges Function for checking edge collision
    function Edges(tetro: typeof tetrominoRef.current) {
      if (!tetro) return { minC: 0, maxC: 0 };
      const size = tetro.shape.length; //stores size of the
tetromino array, exp. 3x3 => 3
      let minC = size; //set this value as minimum column count
by default
      let maxC = 0;

      //By using minimum and maximum search, we get the
smallest and biggest column a tetromino can access
      for (let y = 0; y < size; y++) {
        for (let x = 0; x < size; x++) {
          if (tetro.shape[y][x]) {
            minC = Math.min(minC, x); //Math.min() choose the
smaller number inside the brackets
            maxC = Math.max(maxC, x); //Math.max() choose the
higher number inside the brackets
          }
```

```
        }
      }

      return { minC, maxC }; //returns the smallest and biggest
column
    }
```

2.8.13.1. CheckCollision() függvény

- A CheckCollision(tetro, offsetX, offsetY) függvény egy univerzális ellenőrző logika, amely az aktuális tetrominót vizsgálja meg, opcionális eltolással. Az offsetX és offsetY paraméterek lehetővé teszik, hogy a rendszer „előre” tesztelje a következő lépést, például egy sorral lejjebb vagy egy oszloppal oldalra.

  A függvény működése:

  - végigiterál a tetrominó shape mátrixán,
  - csak azokat a cellákat vizsgálja, amelyek valóban kitöltöttek (1),
  - kiszámolja az adott blokk következő pozícióját (newX, newY) az eltolással együtt.

- Ez a megoldás azért hatékony, mert nem a teljes mátrixot kezeli ütközőtestként, hanem csak a tényleges blokkokat vizsgálja.

2.8.13.2. Pályahatár (Border collision)

- A newX és newY kiszámítása után a rendszer elsőként azt ellenőrzi, hogy a tetrominó kilép-e a játéktérből:

  - newX < 0 → bal oldali fal
  - newX >= COLS → jobb oldali fal
  - newY >= ROWS → alsó határ (talaj)

- Ha ezek közül bármelyik igaz, a függvény azonnal true értékkel tér vissza, vagyis ütközés történt. Ez garantálja, hogy a tetrominó nem mozdulhat olyan helyre, ami már a játéktéren kívül esik.

2.8.13.3. Ütközés a már lerakott tetrominókkal

- Miután a pályahatár ellenőrzés megtörtént, a rendszer megvizsgálja az ütközést a már lerakott tetrominókkal (placedTetrominos.current). Minden lerakott elem saját shape mátrixszal és pozícióval rendelkezik, ezért a vizsgálat relatív indexeléssel történik:

    - newY - placed.y megadja, hogy a vizsgált blokk melyik sorba esne a lerakott elem mátrixában,
    - newX - placed.x megadja a megfelelő oszlopot.

- A kód először azt ellenőrzi, hogy a vizsgált sor létezik-e (placed.shape[newY - placed.y]), majd hogy a cella kitöltött-e. Ha igen, akkor a tetrominó ütközik egy már lerakott blokkal, így a függvény true értéket ad vissza.

2.8.13.4. Irány specifikus ütközésfüggvények

A kód a CheckCollision() függvényt újrahasznosítja három egyszerű segédfüggvénnyel:

- BottomCollision() → egy sorral lejjebb tesztel (offsetY = +1)
- LeftCollision() → egy oszloppal balra tesztel (offsetX = -1)
- RightCollision() → egy oszloppal jobbra tesztel (offsetX = +1)

Ez a megoldás átláthatóbbá teszi a kódot, mivel a játéklogika más részein már csak a megfelelő irányfüggvényt kell meghívni.

2.8.13.5. `Edges()` – szélek meghatározása

- Az Edges() függvény a tetrominó legbaloldalibb és legjobboldalibb kitöltött oszlopát határozza meg (minC, maxC). Ez különösen hasznos a forgatás és a fal melletti mozgások kezelésénél, mert egy tetrominó mátrixa

tartalmazhat üres széleket, amelyek miatt a forma tényleges „szélessége” eltérhet a mátrix méretétől.

A függvény:

- végigkeresi a tetrominó mátrixát,
- ahol kitöltött cellát talál, frissíti a minimum és maximum oszlopértéket,
- majd visszaadja a { minC, maxC } értékeket.

- Ezzel a rendszer pontosan tudja, hogy a tetrominó mely része fog ténylegesen ütközni a falakkal, és így jobb mozgás/forgatás-kezelést lehet kialakítani.

2.8.14. Alakzatok

```
export type TetrominoShape = number[][]; // defines a 2D array type
called TetrominoShape

export const SHAPES: Record<string, TetrominoShape> = { //Record is
a Typescript Utility that creates an object with a key (string) and
property (TetrominoShape)
    i: [
        [0,0,0,0],
        [1,1,1,1],
        [0,0,0,0],
        [0,0,0,0]
    ],
    j: [
        [1,0,0],
        [1,1,1],
        [0,0,0],
    ],
    l: [
        [0,0,1],
```

```
        [1,1,1],
        [0,0,0],
    ],
    o: [
        [1,1],
        [1,1],
    ],
    s: [
        [0,1,1],
        [1,1,0],
        [0,0,0],
    ],
    t: [
        [1,1,1],
        [0,1,0],
        [0,0,0],
    ],
    z: [
        [1,1,0],
        [0,1,1],
        [0,0,0],
    ],
};

export const SHAPE_COLORS = [
    '#00BCD4',
    '#485FE5',
    '#FF9800',
    '#FFEB3B',
    '#4CAF50',
    '#A629BC',
    '#F44336'
];
```

```
export function getTetrominoColor(type: string) {
    const SHAPE_TYPES = Object.keys(SHAPES);  // returns an array of
all the keys in the object, exp: i, j
    const index = SHAPE_TYPES.indexOf(type); // stores the index of
that Tetromino
    return SHAPE_COLORS[index]; //returns its color based on index
  }

console.log(SHAPES);
console.log(SHAPE_COLORS);


//Generating Tetromino

export function generateTetromino() {
    const types = Object.keys(SHAPES);
    const randomIndex = Math.floor(Math.random() * types.length);
//generates a random index
    const randomType = types[randomIndex]; //chooses a random
Tetromino type, exp i, j

    return{
        shape: SHAPES[randomType], //2D array of that Tetromino, exp
z: [[0,1,1], [1,1,0], [0,0,0]]
        x: 3, //x-axis
        y: 0,//y-axis
        type: randomType //exp: i, j
    };

}
```

2.8.14.1. Tetrominó alakzatok, típusok és generálás

- Ez a kódrészlet a Tetris minijáték alapvető építőelemeit határozza meg: a tetrominók alakját, típusát, színét és véletlenszerű generálását. A megoldás TypeScriptben készült, amely lehetővé teszi a típusbiztos adatkezelést, és segít elkerülni a logikai hibákat a játék során.

2.8.14.2. Tetrominó alakzatok reprezentációja

- A tetrominók alakjai kétdimenziós tömbökként (number[][]) vannak definiálva. A TetrominoShape típus deklarálja ezt a struktúrát, amely egyértelművé teszi, hogy minden tetrominó mátrix formában kerül tárolásra.
- Az egyes alakzatok (I, J, L, O, S, T, Z) klasszikus Tetris-formákat követnek. A mátrixban az 1 értékek a kitöltött blokkokat, míg a 0 értékek az üres területeket jelölik. Ez a megoldás egyszerűvé teszi a rajzolást, az ütközésvizsgálatot és a forgatást is, mivel minden művelet ugyanazon adatszerkezetre épül.

2.8.14.3. Alakzatok tárolása Record

- A SHAPES konstans egy Record<string, TetrominoShape> típusú objektum, ahol a kulcs az adott tetrominó típusa (például i, t, z), az érték pedig a hozzá tartozó alakzat mátrixa. A TypeScript Record utility típusa biztosítja, hogy minden kulcshoz egységes adattípus tartozzon, ezáltal növeli a kód megbízhatóságát és olvashatóságát.

2.8.14.4. Színek kezelése és hozzárendelése

- A tetrominók megjelenéséhez egy külön színlista (SHAPE_COLORS) tartozik, amely hexadecimális színkódokat tartalmaz. A getTetrominoColor() függvény az adott tetrominó típusát használja fel arra, hogy meghatározza a megfelelő színt.
- A szín kiválasztása indexalapú módon történik: a tetrominó típusa alapján megkeresi annak helyét a SHAPES kulcsai között, majd az azonos indexű színt adja vissza a színlistából. Ez biztosítja, hogy minden tetrominó következetes és jól felismerhető színnel jelenjen meg a játék során.

2.8.14.5. Véletlenszerű tetrominó generálása

- A generateTetromino() függvény felel az új tetrominók létrehozásáért. A függvény először lekéri az összes elérhető tetrominó típust, majd véletlenszerűen kiválaszt egyet közülük. A kiválasztott típus alapján a rendszer visszaadja:

  - az adott tetrominó alakját (2D mátrix),
  - a kezdő pozíciót a játéktéren (x = 3, y = 0),
  - valamint a tetrominó típusát.

- Ez a megoldás biztosítja, hogy minden új elem kiszámíthatatlanul jelenjen meg, miközben a kezdőpozíció egységes marad, így a játékmenet következetes és kiegyensúlyozott.

2.8.15. Admin panel

```
switch ($action) {
    case 'add':
        $stmt = $conn->prepare("INSERT INTO datas (name, game_pic,
release_date, genre, platforms, prize, isDiscount, discountPerc) VALUES (?, ?,
?, ?, ?, ?, ?, ?, ?)");
        if (!$stmt) die("Prepare failed: " . $conn->error);

        $stmt->bind_param("sssssdii",  $name, $game_pic, $release_date,
$genre, $platforms, $prize, $isDiscount, $discountPerc);

        if ($stmt->execute()) {
            $message =  "Game added successfully!";
            $type = "info";
        } else {
            die("Execute failed: " . $stmt->error);
        }
        break;

    case 'update':
        if (empty($id)) {
            die("ID is required for updating a game.");
        }

        $id = intval($id);

        // Create an associative array of column => value
        $data = [
```

```
            'name'        => $name,
            'game_pic'    => $game_pic,
            'release_date'=> $release_date,
            'genre'       => $genre,
            'platforms'   => $platforms,
            'prize'       => $prize,
            'isDiscount'  => $isDiscount,
            'discountPerc'=> $discountPerc
        ];

        // Remove empty values
        foreach ($data as $key => $value) {
            if ($value === "" || $value === null) {

                if (!is_numeric($value)) {
                    unset($data[$key]);
                }
            }
        }

        if (!empty($data)) {
            $sql_parts = [];
            foreach ($data as $key => $value) {
                // If numeric, no quotes; else escape string
                if (is_numeric($value)) {
                    $sql_parts[] = "$key=$value";
                } else {
                    $sql_parts[] = "$key='" . $conn-
>real_escape_string($value) . "'";
                }
            }

            $sql = "UPDATE datas SET " . implode(", ", $sql_parts) . " WHERE
id=$id";
            $conn->query($sql);
            $message = "Game updated!";
            $type = "success";
        } else {
            echo "Nothing to update!";
        }
        break;

    case 'delete':
        if (empty($id)) {
            die("ID is required to delete a game.");
        }
        $stmt = $conn->prepare("UPDATE datas SET available = 0 WHERE id = ?");
        $stmt->bind_param("i", $id);
        $stmt->execute();
```

```
        $stmt->affected_rows > 0 ? $message = "Game deleted successfully!" :
$message = "No game found with that ID.";
        $type = "danger";
        break;

    default:
        if ($action !== '') {
            echo "Invalid Action";
        }
        break;

}
```

Az admin panel backendje egy switch ($action) szerkezetre épül, amely az admin felületen kiválasztott művelet alapján dönti el, hogy új játékot adunk hozzá**,** meglévő játékot frissítünk, vagy törlünk (nálunk soft delete formában). A megoldás előnye, hogy egyetlen vezérlési ponton kezelhető az összes admin művelet, így átláthatóbb és könnyebben bővíthető a rendszer.

2.8.15.1. Új játék hozzáadása

Az add ágban egy prepared statement (előre fordított SQL) kerül használatra, ami biztonságosabb, mint a sima string-összefűzés, mert csökkenti az SQL injection esélyét.

- A lekérdezés beszúrja a játék adatait a datas táblába:
  name, game_pic, release_date, genre, platforms, prize, isDiscount, discountPerc
- A bind_param("sssssdii", ...) rész meghatározza a típusokat:
  - s = string (szöveg)
  - d = double (ár, lebegőpontos)
  - i = integer (pl. isDiscount, százalék)
- Siker esetén egy visszajelző üzenet készül: “Game added successfully!”.

2.8.15.2. Játék frissítése

A frissítésnél az első lépés az ID ellenőrzése, mivel update-et csak egy konkrét rekordra lehet végrehajtani. Az id intval()-lal számmá konvertálódik, így csökken a hibás inputból adódó gond.

A frissítési logika fontos része, hogy a kód:

- összegyűjti az összes lehetséges mezőt egy $data asszociatív tömbbe,
- majd kiszűri az üresen hagyott mezőket, így nem írja felül őket nullával vagy üres stringgel.

Ezután a rendszer dinamikusan felépíti az SQL SET részét:

- ha az érték numerikus → idézőjelek nélkül kerül be (price=29.99)
- ha szöveg → escape-eli és idézőjelek közé rakja (name='...')

Végül lefut az UPDATE datas SET ... WHERE id=$id, és siker esetén megjelenik a *"Game updated!"* üzenet.
Ha minden mező üres volt, a rendszer jelzi: *"Nothing to update!"*.

2.8.15.3. delete

delete – Soft delete (nem törli ténylegesen)

A delete ág szintén ellenőrzi az ID-t, majd **nem törli fizikailag** a rekordot, hanem egy úgynevezett *soft delete* megoldást használ:

UPDATE datas SET available = 0 WHERE id = ?

Ez azért hasznos, mert:

- az adat nem vész el végleg,
- később visszaállítható,
- statisztikákhoz vagy audit célra megmarad.

A kód a végén vissza is jelzi, hogy történt-e módosítás:

- ha talált rekordot → *"Game deleted successfully!"*
- ha nem → *"No game found with that ID."*

2.8.15.4. Hibás művelet kezelése

Ha az $action olyan értéket kap, amit nem ismer a rendszer, a default ág fut le, és jelzi: "Invalid Action". Ez segít kiszűrni a hibás kérésküldéseket vagy rosszul hívott endpointokat.

3. Felmerült akadályok

- Videogames adatbázis datas táblája tartalmaz többértékű mezőket, amiket több kód átírás ás adatbázis szerkesztés során sem sikerült külön táblákba szedni illetve biztosítani a weboldal megfelelő működését. Ezt sajnos nem tudtuk meg valósítani, mert ehhez az egész adatbázist újra kellett volna dolgozni és minden adatot az új séma szerint feltölteni a projekt akkori állapotában ez már nem volt sajnos megvalósítható.

- Reszponzivitási problémák. Ezekre jó példa a navbar problémák vagy esetleg kartyák kiíratása kisebb képernyőn (például.: telefon vagy táblagépes nézet). Ezeket CSS-sen belül media quaryket használtunk. Ezeket maximum szelesség alapján néztük és ez alapján osztottuk fel. Ezekre jó példa, hogy a telefonos nézetet 400px-eltől számoltuk és onnan volt reszponzív. A mini gameken belül ezek nehezebbek voltak, mert ott a map és minden megjelenítése javascripten belül történt, így azon belül kellett ifekkel és más módszerekkel eltávolítani vagy átalakítani elemeket.

- A pacman minigame-ben kilehet menni a map két oldalán de nincs le lockolva az x tengelyen (y tengelyen is lehet mozogni, ez azt eredményezni hogy a player ki tud menni a map határain kívűl). Ezt azzal oldottuk meg, hogy a mapon kívül, ha kiment a player a tunelen akkor lelockoltuk a mozgását az y-on tengelyen addig ameddig bennem ért a map másik oldalára.

- A szellemek AI – ja úgy van kivitelezve, hogy képtelenek a kezdő helyükről (dobozukról) kijönni. Megoldás az lenne, hogy a kiinduló irányok választásánál minden szellemet felfele toljuk. Eredetileg a mozgás random volt és, hogy ha az első mozgásuk nem felfele volt akkor egy végtelen körforgásba kerültek és köröztek a kezdő helyen belül, így minden ghost első mozgását felre állítottuk, így nem tudtak bebuggolni és sikeresen kijutottak a kezdő helyről.

- A checkoutnál nem történik tényleges kártyás / paypalos tranzakció (API)

- A pac man egyik powerupja – A teleport nem működik egyelőre megfelelően, mert képes blokkokba teleportálni a játékost. Ezt a játék frissítési funkciójában egy iffel avagy egy elágazással tudtuk megoldani, hiszen a teleport függvény az egész mapot nézte, így azokat a elemeket is ahova más elem volt rakva. Így az elágazással ellenőriztük, hogy a hely ahova teleportálja a Pac Mant vagy üres mező vagy pedig pellet avagy a pályán lévő kis pontok.

- A snake game-ben a kígyó (player) képes saját magán keresztül menni, de nem hal meg. Ezt egy collision logic avagy egy ütközési logikával ellenőrizzük. Ezen belül bekérjük a kígyó testét és megnézzük, hogy a fej avagy amit irányítunk nincs ugyan azon a mezőn amin már van a kígyónak az egyik testrésze, ha már ott van akkor meghívunk egy freeze függvényt ami megállítja a játékot, visszaállítja a kígyó eredeti méretét és eredeti helyét, megjeleníti a game over menüt és leállítja a játék folyamatokat. A gameover menün, ha megnyomjuk a Play again gombot akkor meghívja a Restart függvényt ezzel újra indítjuk a játékot.

- A felvehető kaja hitboxa és mérete révén, a pálya margójára is generálódhat. Ezt végül abban a függvényben oldottuk meg ami az étel megevését nézi és abban az étel random kordinátára helyezzük el, ehhez kell 2 változó a foodX és foodY ezek a kettő kordináta érték. Az X kordniátát a Math.random (Add egy random decimális számot 0 és 1 között) függvényt beszorozzuk a szélesség és a hitbox különbségével ezzel garantáljuk ne tudjon a kifolyni a mapról az X kordinátán az étel. At Y kordonítani is hasonlóan generálódik viszont itt aza különbség, hogy a szélesség helyet a magasság van, így a képlet (0 és 1 közötti szám) szorozva magasság és hitbox különbsége.

- A kígyó alap irányításai tükrözve vannak. Ez a hiba javítása egyszerű volt. Azért mert van egy controls függvény amely mindig fut és egy switchel ellenőrzi, hogy a játékos meg e nyom egy nyilat és ha igen akkor az alapján változtatja meg a kígyó sebességét

abba az irányba itt van X és Y változó és ezek változtatásával indul el abba az irányba. Például, ha fel szeretnék menni akkor az X-en a sebesség 0 és Y-on -1 itt jön fel a probléma, hiszen eredetileg azt hittük 1 lesz itt a jó érték csak utólag jöttünk rá, hogy lentről lefelé nézi a játék az irányítást nem fordítva ezért kellett az értéket is megfordítani

- A High Score System nem megfelelően működik. A score system alapján oldottuk meg, amely szerint, ha a player megeszik egy ételt akkor lefut egy függvény amely megnöveli eggyel a scoret és ugyan ezen a függvényen belül egy elágazás ellenőrzi, hogy a mostani score nagyobb, mint az eddig vagy előző high score és ha igen akkor a mostani score lesz a high score és, hogy ez újrafrissítés után is megmaradjon local storageban eltároljuk ezt a változót. Majd a game canvasban ugyan ezt a változót megjelenítjük a high scorenak fenntartott elemben.

- A snake játék border és food collision logikájának megírása. Ezt egy elágazással nézzük amelyben nézzük, hogy a food X és Y kordinátája egyenlő a snake X és Y kordinátájával. Azonban ezzel az volt a probléma, így önmagával, hogy a collison logika működött, de a food a játék borderébe is tudott újra spawnolni, így lehetetlen volt felszedni. Ezért a food kapott egy hitboxot amellyel megtudtuk azt oldani, hogy a canvas határain belül tudjuk tartani és fixen egy cellába spawnoljon az étel és, hogy beleférjen a cellába az étel valamennyivel kisebb, mint maga a cella.

- A játékok custom assetjeinek betöltése különféle keretrendszereken keresztül. A snake játékhoz egy p5.js keretrendszert használunk aminek egyedi előre megírt függvényei vannak ezzel megkönyíti egy játék fejlesztését ilyenek (pl.: Az update ami framenként frissíti a játékot: show is ami jelen esetben nekünk a kígyó testének kirajzolásában segít a frissítés után ezzel lesz látható az új testrész). A show függvény fontos az egész játékban nem csak a kígyó testét hanem az ételek megjelenítését is ez a függvéyn kezeli. Ezen a függvényen belül meghívtunk egy szintén p5.js-es függvényt a createaImg függvényt amely a kép generálásért felelős. Itt rajzolódik ki a kígyónak a

teste az assetsekből kiszedett képből ugyanígy generálódik az étel avagy az alma képe is. Majd egy for looplal végigmegyünk a kígyó testén és annyiszor fog lefutni amilyen hosszú a kígyó, így tudja legenerálni az összes testrészt a kígyónak.

- A kígyó alapvetően nyilakkal mozog, viszont tegyük fel ha felfele tart és megnyomjuk a lefele nyilat, beleütközik saját testébe. Ezt szintén a controls függvényben oldottuk meg a switchen belül amikor ellenőrizzük, hogy ha a lenyomott nyíl előtti nyíl vele ellentétes (pl.: felső nyíl majd alsó nyíl) akkor ezt kiszűrjük egy elágazással ezt nem egyelővel ellenőrizzük már az előbb említett X és Y sebességek alapján (pl.: ha a lenyomott nyíl az a fel gomb akkor ellenőrizzük, hogy a kígyó előző sebessége az Y-on tengelyen ne legyen 1 avagy lefelé megy a kígyó, ha ez igaz akkor az irány változtatás megtörténik, ha nem avagy az irány ellenkező a mostanival akkor átlép a másik elágazás részbe ami megszakítja az irányváltoztatást).

- A tetrominók alakzatainak eltárolása, azok x és y tengelybeli kezdeti értékének definiálása, illetve az alakzathoz tartozó színek megadása.

- A játék elindítása gombnyomásra (space bar)

- Egy fő függvény létrehozása amit frame-en ként meghívunk.

- Tetrominó alakzatok kitöltése a maguk színjével, majd kiírtatása a canvas képernyőjére. Ez egy Tetromino.tsx-ben történik. Itt vannak előre meghatározott mátrixokban alakzatok, innen kapja meg a random alakzatot a program. Egy listában vannak az előre meghatározott színek. Végül ezt átadja a tetris.tsxnek ahol beimportáljuk és ott adjuk át a canvasnak.

- A sidebar elkészítése a UI – on, majd kirajzolni a megfelelő színnel a következő alakzatot és meghatározni annak pozícióját a sidebaron belül.

- A tetromino collision logika megcsinálása a grid jobb és bal oldalán, az alján. Ugyanígy a már letett tetrominók esetében.

- A collision logika egyszerűsítése. Külön függvények létrehozása a jobb, bal oldali és a talaj collision-höz. Ezt végül egy újra gondolt logikával oldottuk meg. Egy ütközés avagy Collision ellenőrzővel oldottuk meg amelyet végül több fajta függvényben hívtuk meg. Erre jó példa a BottomCollision amely az alsó ütközést nézte. Ezen meghívunk 3 paramétert, az első az alakzat illetve 2 offset az x és az y. Itt az y az egy mivel nulla az a játék elején a keret alja a játék közben a sor amire érkezik. X meg 0 lesz, mert ezt itt nem nézzük. Ezeket majd még 2 különböző függvény nézi avagy a bal és a jobb oldali ellenőrzés.

- Egy megtöltött sor esetében az adott sor törlése. Újabb felmerült probléma, hogy egyszerre nem tudunk több telített sort eltávolítani. Ehhez végül egy ciklus kellett amely lefutáskor ellenőrzi, hogy ha megtel egy sor azt kitörli és mivel egy ciklus végig fut minden soron és addig nem megy tovább a játék ameddig miden megtelt sort ki nem törölte és a felette lévő sorokat lejebb vitte, így nem maradtak üres sorok.

- A tetromino alakzatok forgatása matematikai mátrix felhasználásával. Ezt érvényesíteni minden fokra. Erre jó példa az a logika amely nem hagyja, hogy az alakzatok kibuggoljanak a jaték canvasán kívülre ezt úgy csinálja meg, hogy az alakzat mátrixának a mérete nem változik az elforgása közben.

- A Controls fejlesztése: Meg kell vizsgálni a jobb és bal széleket, hisz amikor forgatás történik, az adott alakzat jobb és bal határai is változnak. Ugyanezt a logikát át kell ültetni a talaj collision logikára is.

- A discount logika átalakítása, ugyanis jelenleg a játékok leárazása úgy történik, hogy egy adott játék véletlenszerűen (5 – 90%) között kap egy leárazást és ezt az értéket meg is tartja (nem reális megoldás). Ezt egy jobb tovább fejlesztett logikával oldottuk meg. Amely most már nem csak random kiválasztott egy játékot, hanem most már a fejlesztő vagy a store adminja tudja meghatározni a leárazás mértékét, így például egy új játék nem 90%-kal lesz leárazva hanem csak 10-el.

- A wish list termékeinek eltárolása az oldal ujratöltése után. A wish list szív ikon megfelelő toggle logikájának kivitelezése. Ezt egyszerűen a local storagel lett eltárolva, így nem az oldaltól függött a tárolás hanem a felhasználó gépében tárolta ezt az adatot. A szív ikon egy svg volt javascripten belül a toggle logikán belül, ha bevolt nyomva az a gomb akkor lefutott egy elágazás ami megnézte a kettő közül melyik állapotban van bevan kapcsolva vagy pedig nincs, ha nem volt akkor az svg színét egy string változóban megtudtuk változtatni és visszaadni a html/php-nek. Fordítva visszaszínezte az eredeti színére az ikont.

- A filter rendszer megfelelő működése a learázott játékok esetében (ABC, ár szerinti rendezés, illetve ár range). Ezeket végül finomítottabb algoritmusokkal oldottuk meg például egy csúszka a kiadási évnek, így lehetet év szerint keresni a csúszkán szereplő adatot meg könnyen visszatudtuk adni egy javascript fájlnak. Az árat egy árrés alapján oldottuk meg volt egy minimális érték és egy maximális (pl.: 20$ minimumnak és maximumnak 50$) ezt szintén visszaadtuk a javascriptnek és onnan egy metódus megoldotta a keresést.

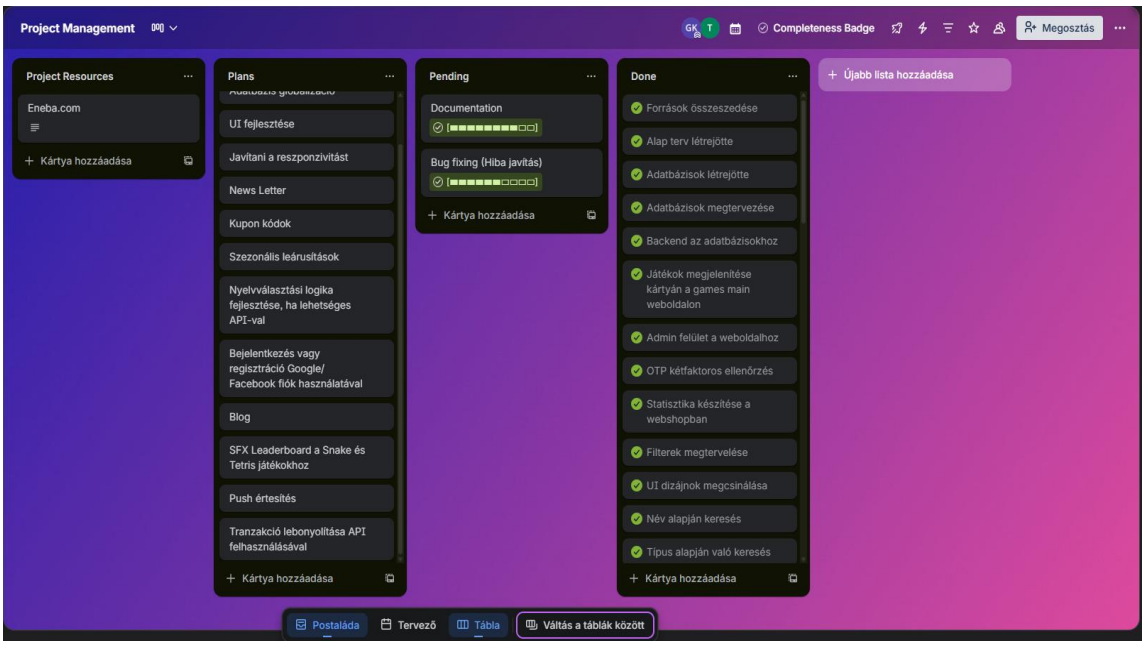
- A live stock feature működtetése firebase cloud service felhasználásával

- A shopping cart adatainak átvitele az URL – en keresztül a tranzakciós oldalakra

- Az alap ár (amerikai dollárban megadva) átváltása más pénznemekbe, azok igazítása a filterekhez, illetve azok megfelelő felvétele, tárolása a shopping cartban és azon túl. Ehhez egy API-t használtunk amely 5 percenként frissíti az árfolyamokat, így egy metóduson belül a select avagy az oldalon található pénznem lista elemét és annak nevét és ezt, így egy cikluson belül kikeresi az adott pénznemet és ezzel átváltja abba pénznembe a termék árát.

4. Közös munka és Platformok

4.1. Trello



37. ábra - Trello Layout

A Trellót én és a csapattársam a feladatok felosztására, valamint a projekt feladatainak nyomon követésére és azok haladási fokának meghatározására használtuk. Nagy segítséget jelentett számunkra a Trello táblaszerű felépítése és a különböző beépített funkciói, amelyek megkönnyítették a csapatmunkát és az átlátható tervezést.

Ilyen hasznos funkció volt például a „Pending” listában található kártyák alatti **Completeness Badge**, amely jól mutatta, hogy egy adott feladat milyen mértékben van kész. Ennek köszönhetően könnyen nyomon tudtuk követni, hogy az elkezdett feladatokra még mennyi időt kell fordítanunk, és így hatékonyabban tudtunk prioritási sorrendet felállítani.

Ahogy látható, összesen **négy különböző listát** használtunk a projekt során. Az első listában a projekt nyersanyagai kaptak helyet, mint például források vagy ihletet adó weboldalak. Itt például megtalálható az Eneba.com is, amely egy link segítségével közvetlenül elérhető volt számunkra.

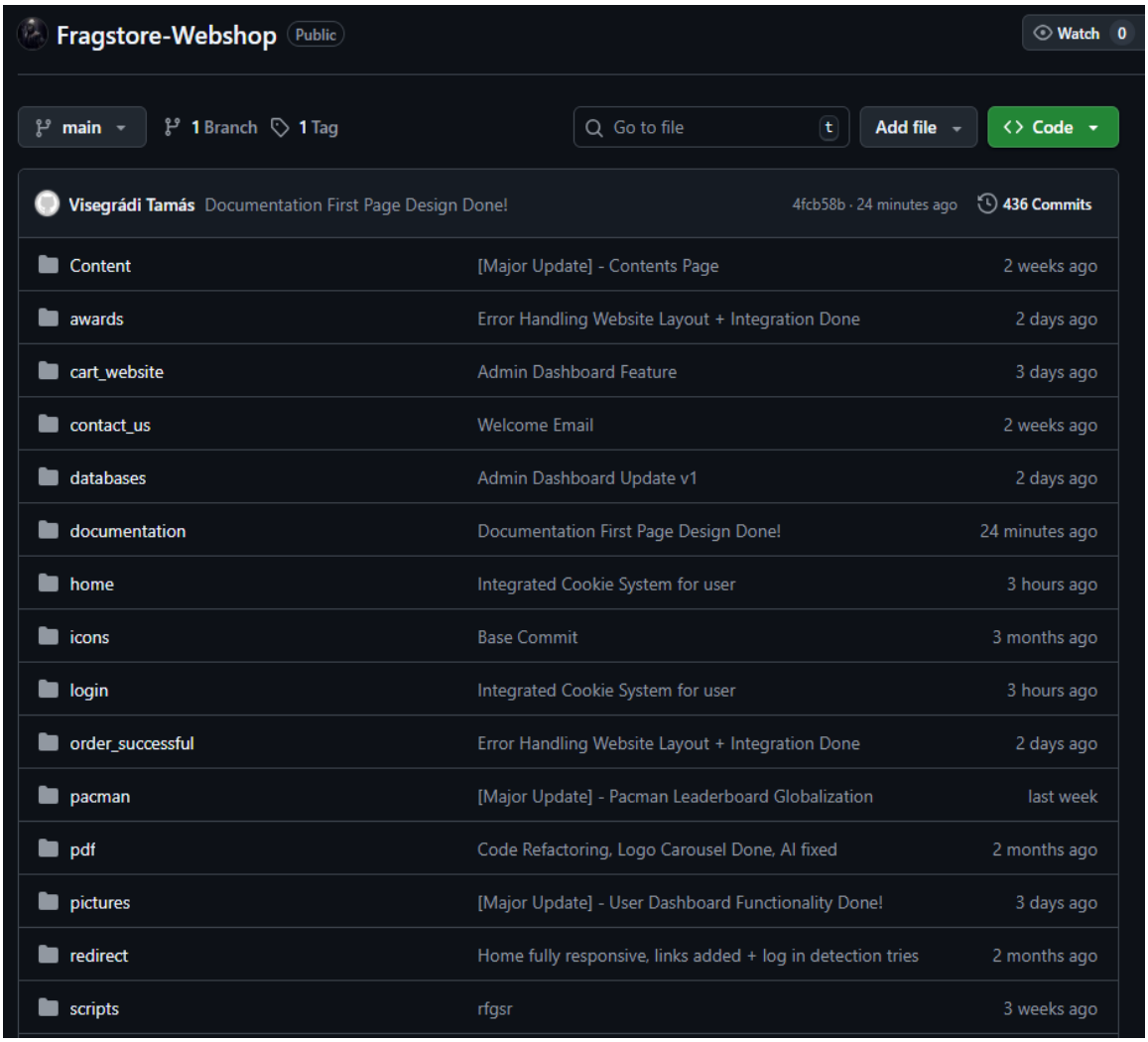
A második lista a **Plans** nevet viseli, és a továbbfejlesztési lehetőségeket tartalmazza. Ezek között szerepelnek egyszerűbb feladatok, mint például a felhasználói felület

dizájnjának finomítása, de összetettebb elképzelések is, mint egy blog vagy egy hírlevél (newsletter) rendszer kialakítása.

A harmadik lista a **Pending**, amelyről már korábban is szó esett. Ennek a lényege, hogy az éppen folyamatban lévő feladatok itt jelennek meg, minden esetben egy Completeness Badge segítségével jelezve az aktuális készültségi állapotot.

Az utolsó lista a **Done**, vagyis a kész feladatokat tartalmazza. Ide kerültek minden olyan előre meghatározott feladatok, amelyek már teljes mértékben elkészültek, így egyértelműen láthatóvá vált a projekt haladása és az elvégzett munka mennyisége.

4.2. Github



38. ábra - Github Repository

A GitHubot a projektünk elsődleges verziókezelő rendszerének használtuk. Számunkra ez bizonyult a legegyszerűbb és legátláthatóbb megoldásnak a fejlesztés során. Nagy előnyt jelentett, hogy az általunk használt Visual Studio Code fejlesztőkörnyezet alapértelmezetten tartalmaz egy GitHub-kezelő felületet, amely jelentősen megkönnyítette a munkát. A commitok, a pull és push műveletek, valamint az egyszerűbb merge-ek néhány gombnyomással elvégezhetők voltak, terminál használata nélkül.

Fontos szempont volt számunkra a GitHub letisztult felülete és könnyű kezelhetősége. Bár a több branch használata is egy hasznos funkció, ezt a projekt során végül nem alkalmaztuk, mivel túl későn kezdtük volna el bevezetni, így abban a fázisban már nem lett volna hatékony vagy indokolt. Ennek ellenére a rendszer így is jól támogatta a közös munkát.

Kifejezetten hasznosnak találtuk a commitolási rendszert, mivel pontosan nyomon követhető volt, hogy ki mikor és mennyit dolgozott a projekten, valamint az is, hogy mikor történt az utolsó módosítás. A commit history áttekinthető és könnyen kezelhető, ami nagy segítséget jelentett a fejlesztés során, illetve a projekt előrehaladásának visszakövetésében is.

A GitHub használata mellett szólt az is, hogy már több mint négy éve tanuljuk és használjuk a Git verziókezelést, így a projekt során nem okozott nehézséget a kezelése. Ez lehetővé tette számunkra, hogy a verziókezelésre ne akadályként, hanem hatékony segítségként tekintsünk a fejlesztési folyamat során.

4.3. Discord

A Discord számunkra a fő, illetve legfontosabb kommunikációs alkalmazás volt a projekt során. Régóta használjuk, és mivel más célokra is rendszeresen igénybe vesszük, így a beüzemelése nem jelentett különösebb nehézséget. Mindketten jártasak voltunk a Discord működésében, valamint a szerverek és csatornák használatában, így gyorsan és hatékonyan tudtuk kialakítani a projekt számára megfelelő környezetet.

A Discord funkciói teljes mértékben megfeleltek a projekt kaliberének. Előnyei közé tartozik a letisztult felület, a könnyű kezelhetőség, valamint számos olyan kényelmi funkció, amely megkönnyíti a csapatmunkát. Számunkra a legfontosabb funkció a

hangcsatornákon belüli képernyőmegosztás és streamelési lehetőség volt, mivel ennek segítségével közösen tudtunk tervezni, illetve hibákat keresni a fejlesztés során.

A képernyőmegosztás lehetőséget adott arra is, hogy egy-egy nagyobb, már elkészült projektrészletet élőben meg tudjunk mutatni egymásnak, így azonnali visszajelzést kaphattunk. A Discord által biztosított hang- és képminőség megfelelő volt, a streamek tiszták és folyamatosak voltak, ami nagyban hozzájárult a hatékony közös munkához.

Egy ilyen kommunikációs alkalmazás használata azért volt különösen fontos számunkra, mert így az iskolán kívül is tudtunk együtt dolgozni a projekten. Ez lehetővé tette, hogy például iskolai szünetek alatt se legyenek hosszabb kimaradások a fejlesztésben. Ennek köszönhetően nem estünk ki a munkafolyamatból, és sikerült megőriznünk a motivációnkat a projekt teljes időtartama alatt.

5. Továbbfejlesztések

- **Bundels:**
  - A bundle funkció lehetővé tenné több termék együttes, kedvezményes áron történő megvásárlását. Ez növelné a felhasználók vásárlási kedvét, valamint az átlagos kosárértéket is. A bundle-ök kialakítása történhetne előre definiált csomagokkal vagy dinamikusan, a felhasználó választása alapján. Ez a megoldás különösen hasznos lehet akciók vagy szezonális események során.
- **Adatbázis Globalizáció:**
  - Az adatbázis globalizációja lehetővé tenné több nyelv és régió kezelését egyetlen rendszerben. Ez magában foglalná a terméknevek, leírások, pénznemek és dátumformátumok lokalizálását. A megoldás elősegítené a nemzetközi felhasználók kiszolgálását és az oldal globális elérhetőségét. Hosszú távon ez egy stabil alapot adna a terjeszkedéshez.
- **UI fejlesztés:**
  - A felhasználói felület további fejlesztése növelné az oldal átláthatóságát és esztétikai értékét. Finomabb animációk, egységesebb színpaletta és jobb komponens-elrendezés javíthatná a felhasználói élményt. A cél egy modernebb, letisztultabb megjelenés kialakítása lenne. Ez hozzájárulna az oldal professzionálisabb összképéhez.

- **Javított reszponzivitás:**
    - A reszponzivitás további fejlesztésével az oldal még jobban alkalmazkodna különböző képernyőméretekhez. Ez különösen fontos mobil- és tabletfelhasználók esetében. A cél a töréspontok finomhangolása és az interaktív elemek jobb kezelhetősége kisebb kijelzőkön. Ezáltal az oldal minden eszközön kényelmesen használható lenne.
- **News Letter:**
    - A hírlevél funkció lehetőséget adna a felhasználók rendszeres tájékoztatására újdonságokról, akciókról és frissítésekről. A feliratkozók célzott tartalmat kaphatnának érdeklődési körük alapján. Ez erősítené a felhasználói elköteleződést és visszatérést. Emellett marketing szempontból is hatékony eszköz lenne.
- **Kupon kódok:**
    - A kuponkód rendszer lehetővé tenné kedvezmények biztosítását meghatározott feltételek mellett. Ezek lehetnek egyszer használatos, időszakos vagy felhasználóhoz kötött kuponok. A funkció ösztönözné a vásárlásokat és növelné az akciók hatékonyságát. A kuponok könnyen kombinálhatók lennének más promóciókkal is.
- **Szezonális leárusítások:**
    - A szezonális leárazások bevezetése lehetőséget adna időszakos akciók kezelésére, például ünnepekhez vagy eseményekhez kötve. A rendszer automatikusan csökkentené az árakat egy meghatározott időintervallumban. Ez növelné az oldal aktivitását és a vásárlási hajlandóságot. A felhasználók számára egyértelmű és átlátható kedvezményeket biztosítana.
- **Bejelentkezés vagy regisztráció Google vagy Facebook fiók használatával:**
    - A külső szolgáltatásokkal történő bejelentkezés leegyszerűsítené a regisztrációs folyamatot. A felhasználók gyorsabban és kevesebb adat megadásával tudnának fiókot létrehozni. Ez csökkentené a belépési akadályokat és növelné a regisztrációk számát. A megoldás modern és felhasználóbarát élményt biztosítana.
- **Blog:**
    - Egy blog rendszer lehetőséget adna hírek, fejlesztési naplók és frissítések megosztására. Ez erősítené a közösségi jelenlétet és az oldal hitelességét. A

blog tartalmai segítenének a felhasználók tájékoztatásában és bevonásában. Emellett keresőoptimalizálási (SEO) szempontból is előnyös lenne.

- **SFX Leaderboard a Snake és Tetris játékokhoz:**
  - A ranglista funkció megjelenítené a Snake és Tetris játékok legjobb eredményeit. Ez versenyhelyzetet teremtene a felhasználók között, növelve a játékok újrajátszhatóságát. A leaderboard adatbázisban tárolná a pontszámokat és felhasználóneveket. Ez hosszú távon növelné az oldal interaktivitását.
- **Push értesítése:**
  - A push értesítések segítségével a felhasználók azonnali tájékoztatást kaphatnának fontos eseményekről. Ilyenek lehetnek akciók, új termékek vagy rendszerüzenetek. Ez növelné a visszatérési arányt és az oldal aktivitását. A funkció személyre szabható lenne felhasználói beállítások alapján.
- **Tranzakció lenonyolítása API felhasználásával:**
  - A fizetési API-k integrálása lehetővé tenné a valós tranzakciók kezelését. Ez magában foglalná a biztonságos fizetést, a rendelés-visszaigazolást és a számlázást. A megoldás professzionális webshop élményt biztosítana. Emellett megfelelne a modern e-kereskedelmi elvárásoknak és biztonsági követelményeknek.

6. Összefoglalás

A közös munka számunkra nem volt idegen, hiszen már évek óta együtt dolgozunk különböző projektekben és más informatikai területeken is. Ennek köszönhetően megvolt az összhang közöttünk, és jóval kevesebb konfliktus vagy nézeteltérés fordult elő, mint egy kevésbé összeszokott csapat esetében. A meglévő tapasztalataink nagyban hozzájárultak ahhoz, hogy hatékonyan és gördülékenyen tudjunk együttműködni a projekt során.

Rengeteget tanultunk a projekt megvalósítása alatt. Korábban még nem vettünk részt ilyen nagy terjedelmű és komplex projektben, ezért az egész folyamat új élményt jelentett számunkra. Az egyik legfontosabb tanulság mindkettőnk számára a vezetői szerep megismerése volt, valamint az empátia és a megértés fontossága. Megtanultuk, hogy akkor is fenn kell tartani az effektív közös munkát, ha a másik félnek éppen rosszabb napja van. Szintén fontos tapasztalat volt a saját határaink felismerése. A projekt elején rengeteg funkcionalitást terveztünk, és naivan azt gondoltuk, hogy mindegyiket meg

tudjuk valósítani, sőt akár túl is tudjuk szárnyalni az elképzeléseinket. Az idő előrehaladtával azonban éreztük a nyomást, és világossá vált, hogy a lendület és a motiváció idővel csökkenni fog. Ennek ellenére a tudatos limitálásnak köszönhetően mindig maradt bennünk elegendő energia ahhoz, hogy tovább dolgozzunk a projekten, még ha kisebb lépésekben is. Így jutottunk el a projekt végső, kiforrott formájához.A projekt nagyban hozzájárult a kritikus gondolkodásunk fejlődéséhez is. A célunkat végig szem előtt tartottuk, és a kivitelezést már előre átgondoltuk, ami segített meghatározni a projekt irányát, és csökkentette az elakadások számát. Ez a munka kiváló lehetőséget biztosított több programozási nyelv, keretrendszer és API megismerésére és alkalmazására. Jó példa erre a Google Fordító API használata, vagy a p5.js, amely egy JavaScript alapú keretrendszer, és játékfejlesztés során egyszerűsített, beépített függvényekkel segítette a munkánkat.A dizájnelemek használatában is sokat fejlődtünk, mivel számos új módszert és technológiát alkalmaztunk az oldalak elkészítése során. Ilyen volt például a GSAP, amely az animációk megvalósításában nyújtott nagy segítséget. Egyszerű használatának köszönhetően sokkal hatékonyabban és látványosabban tudtunk animálni, mint pusztán CSS vagy vanilla JavaScript alkalmazásával.Fontos tanulság volt számunkra az is, hogy a projekt megkezdése előtt még nagyobb hangsúlyt kellett volna fektetnünk a tervezésre, mivel a fejlesztést viszonylag hamar elkezdtük. Ennek következményeként utólag több elem eltávolítására és teljes újratervezésekre is szükség volt. Erre jó példa a fordítási rendszer, amelyet kezdetben JSON fájlok és data tagek segítségével valósítottunk meg, majd később rájöttünk, hogy egy fordítási API használata sokkal hatékonyabb megoldás: kevesebb kódot igényel, és lényegesen több nyelv támogatását teszi lehetővé.A projekt elején az is előfordult, hogy a munka lendülete miatt külön dolgoztunk, kevesebb kommunikációval, ami bizonyos megoldásoknál elakadásokhoz vezetett. Később azonban a folyamatos tervezés és rendszeres megbeszélések hatására ezek a problémák megszűntek. Ilyen volt például az adatbázis kialakítása, amelyről kezdetben nem volt teljes egyetértés, de végül mindketten hatékonyan tudtunk vele dolgozni. Ugyanez igaz egyes oldalak felépítésére is, amelyeket közös megbeszélések alapján átalakítottunk, hogy mindkettőnk számára megfelelőek legyenek. Ennek köszönhetően megtanultunk kompromisszumokat kötni, és ezzel elkerülni a felesleges konfliktusokat.

Összegzésként elmondható, hogy a csapat teljes mértékben elégedett az elvégzett munkával, és a számunkra elvárt minőséget sikerült elérnünk. Rengeteget köszönhetünk

egymásnak, valamint a segítőinknek és mentorainknak. A projekt kezdetén nem gondoltuk volna, hogy az addigi együttműködésünket is tovább tudjuk fejleszteni, mégis sikerült, ami számunkra az egyik legnagyobb eredménye ennek a munkának.

7. Források

https://github.com/VTom21/Fragstore-Webshop

https://www.eneba.com

https://www.g2a.com

https://store.playstation.com

https://www.xbox.com

8. Ábrajegyzék:

1. ábra - Hero Section 11
2. ábra - About Us 13
3. ábra - Logo Carousel 15
4. ábra - Giftcard Section 17
5. ábra - Genres Section 19
6. ábra - FAQ 21
7. ábra - games main 23
8. ábra - Filtering 24
9. ábra - Currency Conversion 25
10. ábra - Wishlist 26
11. ábra - Shopping Cart 26
12. ábra - Footer 28
13. ábra - Log In 29
14. ábra - OTP Verification 31
15. ábra - Sign Up 32
16. ábra - Transaction 33
17. ábra - Order Summary 34
18. ábra - Delivery Method 35
19. ábra - Payment Method 37
20. ábra - Pac Man Menu 38
21. ábra - Pac Man field 39
22. ábra - Pac Man Game Over 40
23. ábra - Pac Man Achievements 41
24. ábra - Snake Menu 42
25. ábra - Snake game 43
26. ábra - Snake Game Over 44
27. ábra - Tetris Game 45
28. ábra - Tetris Game Over 46
29. ábra - Admin Panel 48
30. ábra - Database Plan 65

31. ábra - Firebase Databases 78
32. ábra - Leaderboard 78
33. ábra - Delivery 79
34. ábra - Home Figma 83
35. ábra - Login / Sign up 83
36. ábra - Main webshop Figma 84
37. ábra - Trello Layout 151
38. ábra - Github Repository 152